Minnesota State University, Mankato

EE 584 Final Project Report

Design and Implementation of an 8-Bit Pipelined Vedic Multiplier Using 90nm CMOS Technology

Oluwaferanmi Arowoshola
12-14-2025

Design and Implementation of an 8-Bit Pipelined Vedic Multiplier Using 90nm CMOS Technology

Oluwaferanmi Arowoshola

12/14/2025

Table of Contents

1. **Introduction** .....

2. **Overall Design Architecture** .....
    - 2.1 Architectural Overview .....
    - 2.2 Hierarchical Decomposition .....
        - 2.2.1 8×8 Pipelined Vedic Multiplier .....
        - 2.2.2 4×4 Vedic Multiplier .....
        - 2.2.3 2×2 Vedic Multiplier .....
    - 2.3 Arithmetic Building Blocks .....
        - 2.3.1 Adders .....
        - 2.3.2 Register Design .....
    - 2.4 Pipeline Placement Strategy .....

3. **Circuit-Level Design and Transistor Sizing** .....
    - 3.1 Design Methodology and Technology Assumptions .....
    - 3.2 Primitive Logic Gates .....
        - 3.2.1 Inverter (INV) .....
        - 3.2.2 NAND Gate (NAND2) .....
        - 3.2.3 AND Gate (AND2) .....
    - 3.3 XOR Gate Design .....
    - 3.4 Adder Circuits .....
        - 3.4.1 Half Adder (HA) .....
        - 3.4.2 Full Adder (FA) .....
        - 3.4.3 Ripple Carry Adders .....
    - 3.5 Transmission-Gate D Flip-Flop (TGATE_DFF) .....
        - 3.5.1 Transmission Gate (TGATE) .....
        - 3.5.2 TGATE D Flip-Flop Structure .....
    - 3.6 Design Trade-Off Summary .....

4. **Layout Implementation and Physical Design Strategy** .....
    - 4.1 Layout Design Methodology .....
    - 4.2 Primitive Cell Layout .....

4.3 Mid-Level Block Layout .....
4.4 Top-Level Layout Integration .....
4.5 Pin Placement Strategy .....
4.6 Metal Layer Utilization .....
4.7 Physical Verification .....
4.7.1 Design Rule Check (DRC) .....
4.7.2 Layout Versus Schematic (LVS) .....
4.7.3 Parasitic Extraction .....
4.8 Layout Challenges and Design Improvements .....

5. **Simulation, Verification, and Testing Methodology** .....
5.1 Verification Strategy Overview .....
5.2 Simulation Environment and Setup .....
5.2.1 Clock Configuration .....
5.3 Pre-Layout Functional Simulation .....
5.3.1 Input and Output Signals .....
5.3.2 Functional Correctness .....
5.4 Pipelining Behavior and Timing Interpretation .....
5.5 Physical Verification as Part of Testing .....
5.6 Absence of Post-Layout Simulation .....
5.7 Summary of Verification Results .....

6. **Design Analysis and Performance Evaluation** .....
6.1 Area Analysis .....
6.2 Timing Analysis .....
6.2.1 Clock Frequency .....
6.2.2 Critical Path Identification .....
6.2.3 Propagation Delay .....
6.2.4 Timing Trade-Offs .....
6.3 Power Considerations .....
6.3.1 Dynamic Power Consumption .....
6.3.2 Static Power Consumption .....
6.3.3 Power–Performance Trade-Off .....
6.4 Scalability Considerations .....
6.5 Summary of Performance Evaluation .....

7. **Design Decisions, Challenges, and Lessons Learned** ....
7.1 Key Design Decisions .....
7.1.1 Selection of Vedic Multiplier Architecture .....
7.1.2 Use of Pipelining .....
7.1.3 Choice of Transmission-Gate DFFs .....
7.1.4 Use of Ripple Carry Adders .....
7.2 Major Design Challenges .....
7.2.1 4×4 Vedic Multiplier Layout Complexity .....

7.2.2 Routing Congestion in Top-Level Layout .....
7.3 Verification-Driven Design Refinement .....
7.4 Lessons Learned .....
7.5 Opportunities for Improvement .....

8. **Conclusion** .....

9. **References** .....

1. Introduction

The continuous demand for high-speed and area-efficient arithmetic units has made multiplier design an important topic in modern digital VLSI systems. Multipliers are basic building blocks in signal processing, cryptography, machine learning accelerators, and general-purpose processors, where performance is limited a lot by multiplication latency and throughput. As operand widths increase, conventional multiplication approaches can cause significant delays due to deep combinational logic and long-carry propagation paths [2]

This project presents the design and implementation of an 8-bit pipelined Vedic multiplier using full-custom CMOS design techniques. The primary objective is to show how Vedic multiplication principles combined with pipelining can improve computational throughput while maintaining structural regularity and scalability. The design is implemented at the transistor level using a **90 nm CMOS process**, verified through schematic simulation, layout, and physical verification steps including Design Rule Check (DRC) and Layout Versus Schematic (LVS) [3].

Vedic multiplication, derived from the ancient Indian mathematical system, is based on the Urdhva Tiryakbhyam (vertical and crosswise) algorithm. Unlike conventional array or Booth multipliers, Vedic multipliers generate partial products in parallel, reducing the logical depth of the multiplication operation. This inherent parallelism makes its architecture quite good for high-speed digital implementations, especially when combined with pipelining techniques. Prior research has shown that Vedic multipliers offer advantages in speed and regular layout structure when implemented in CMOS technologies [1].

To further enhance performance, pipelining is introduced into the multiplier architecture. By putting registers between major computational stages, the overall critical path is reduced, allowing the circuit to operate at higher clock frequencies. While pipelining increases register count and switching activity, it significantly improves throughput, making it a practical trade-off for performance-oriented designs.

This report details the architectural decomposition, circuit-level implementation, layout strategy, and verification of the proposed 8×8 pipelined Vedic multiplier, highlighting key design decisions, challenges encountered, and performance considerations.

2. Overall Design Architecture

2.1 Architectural Overview

The proposed design implements an 8×8 pipelined Vedic multiplier using a hierarchical and modular approach. The architecture decomposes the multiplication operation into smaller reusable building blocks, improving design clarity, verification efficiency, and layout scalability. Figure 1 illustrates the top-level block diagram of the multiplier.

Figure 1a: Top-level block diagram of the 8×8 pipelined Vedic multiplier

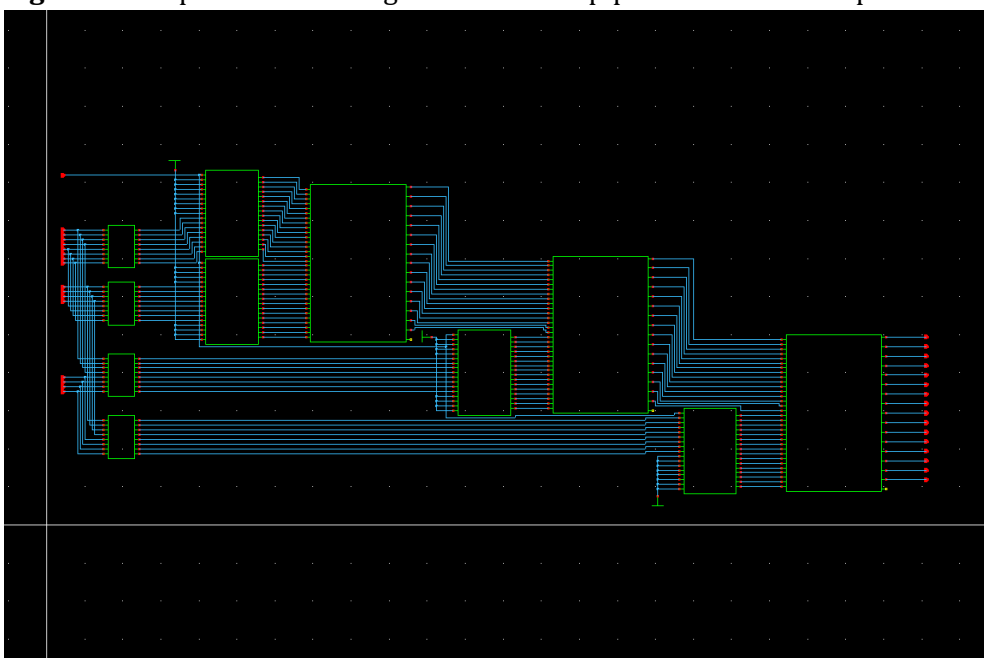


Figure 1b: Top-level block diagram of the 8×8 pipelined Vedic multiplier (inputs)

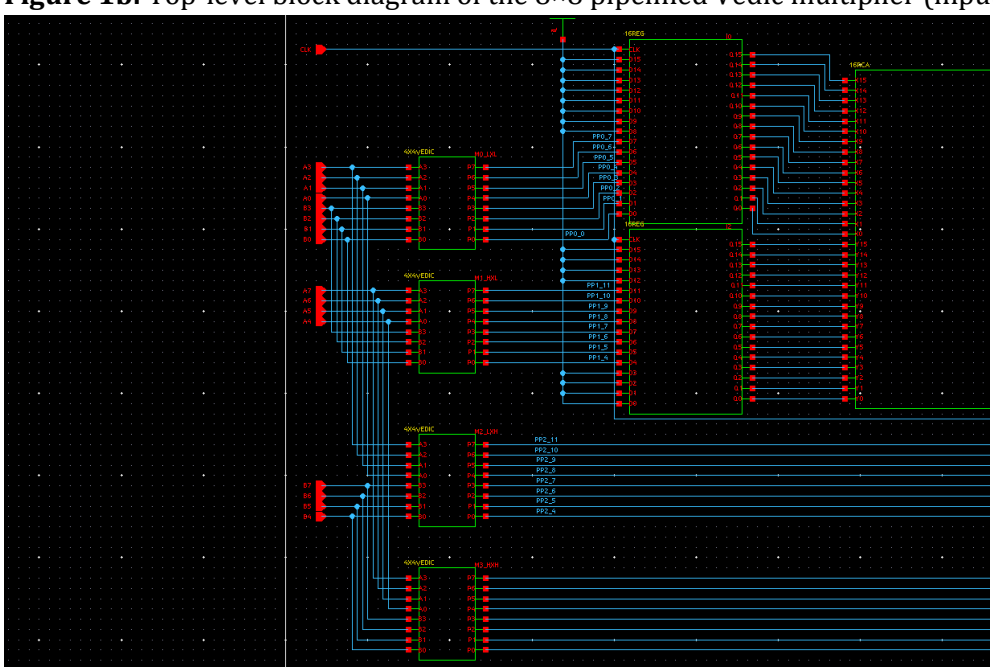


Figure 1c: Top-level block diagram of the 8×8 pipelined Vedic multiplier (outputs)

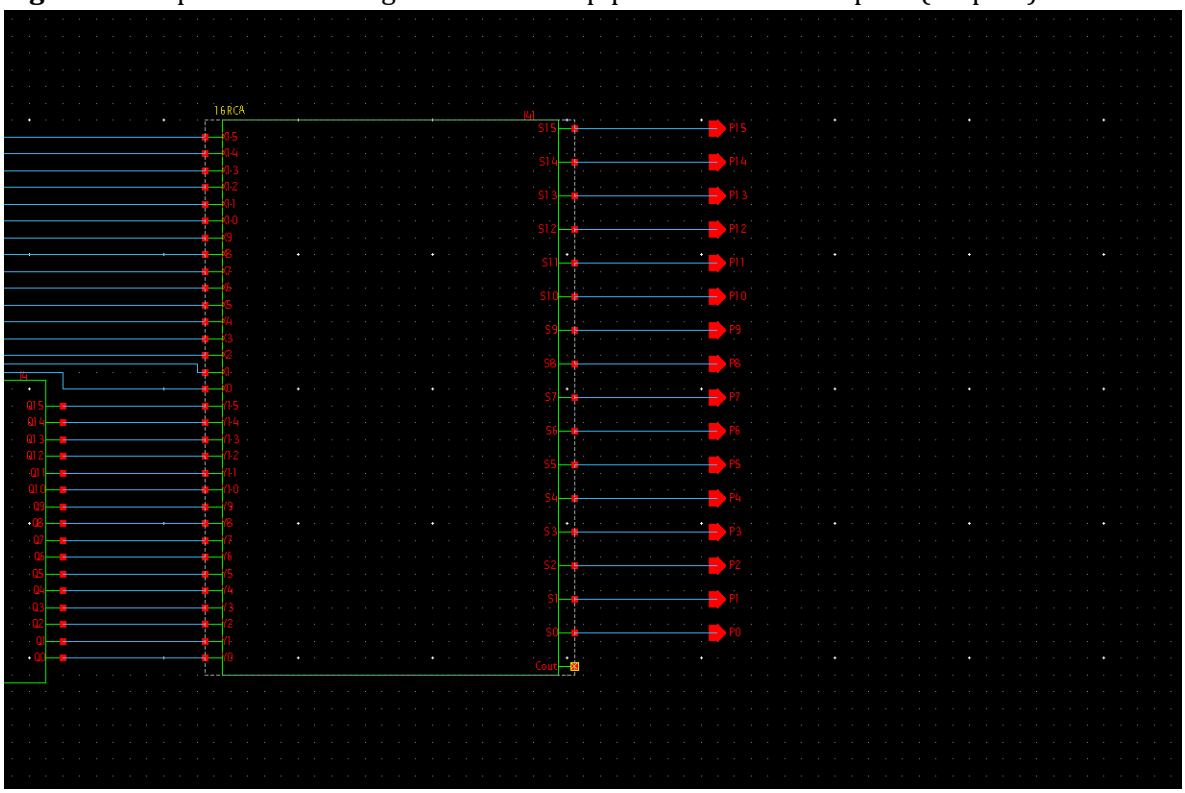


Figure 1d: 8×8 pipelined Vedic multiplier symbol

At the highest level, the design consists of:

- Four 4×4 Vedic multiplier blocks
- Four 16-bit register banks (16REG)
- Three 16-bit ripple carry adders (16RCA)

The pipeline registers are strategically placed between the 4×4 Vedic multiplier outputs and the following 16-bit addition stages. This placement breaks the long combinational paths associated with partial product accumulation and carry propagation, thereby reducing the effective critical path [1].

2.2 Hierarchical Decomposition

2.2.1 8×8 Pipelined Vedic Multiplier

The top-level module, 8X8VEDIC_PIPELINED, is constructed as follows:

- 4 × 4X4VEDIC blocks generate intermediate partial products
- 4 × 16REG pipeline registers store intermediate results
- 3 × 16RCA blocks perform final summation of partial products

The use of pipeline registers enables new input operands to be accepted every clock cycle after the pipeline is filled, significantly improving throughput [1].

2.2.2 4×4 Vedic Multiplier

Each 4X4VEDIC block is built hierarchically using smaller arithmetic units:

- 4 × 2X2VEDIC blocks
- 3 × 8RCA adders

This decomposition leverages the regular structure of Vedic multiplication, allowing efficient reuse of lower-level blocks while simplifying routing and verification.

Figure 2a: Internal structure of the 4×4 Vedic multiplier

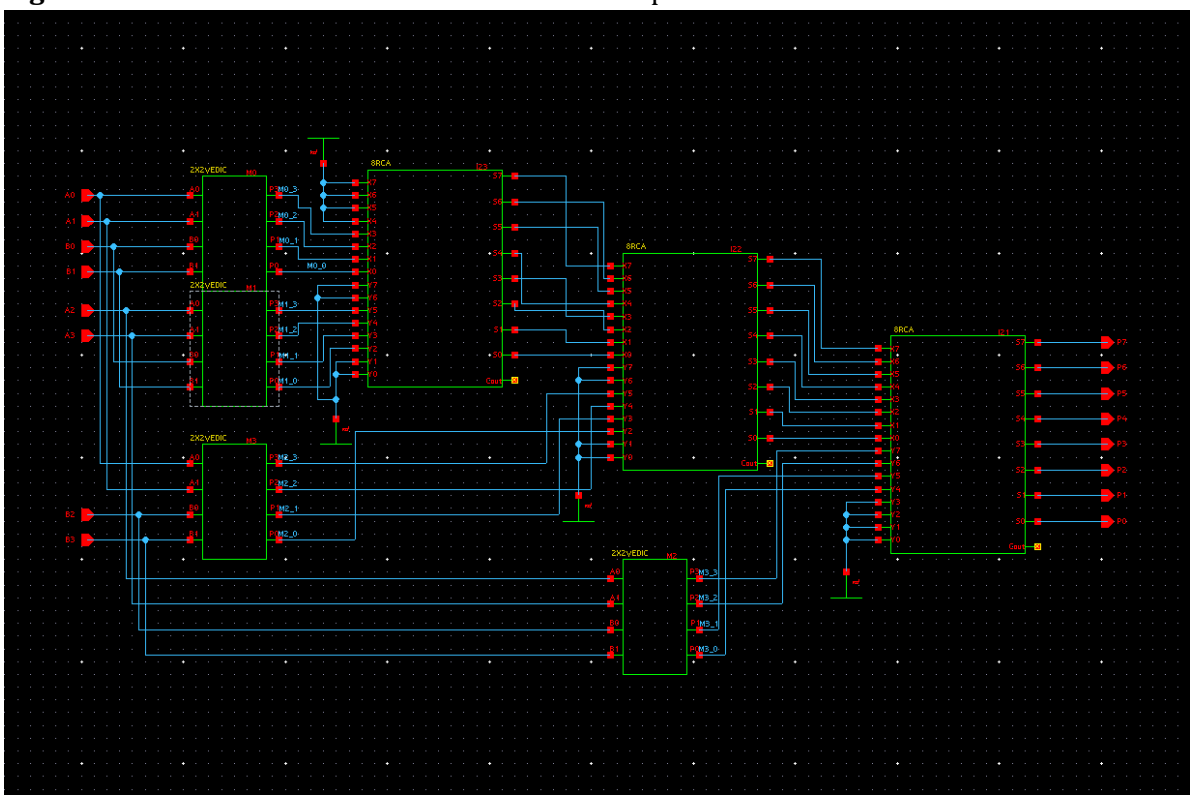
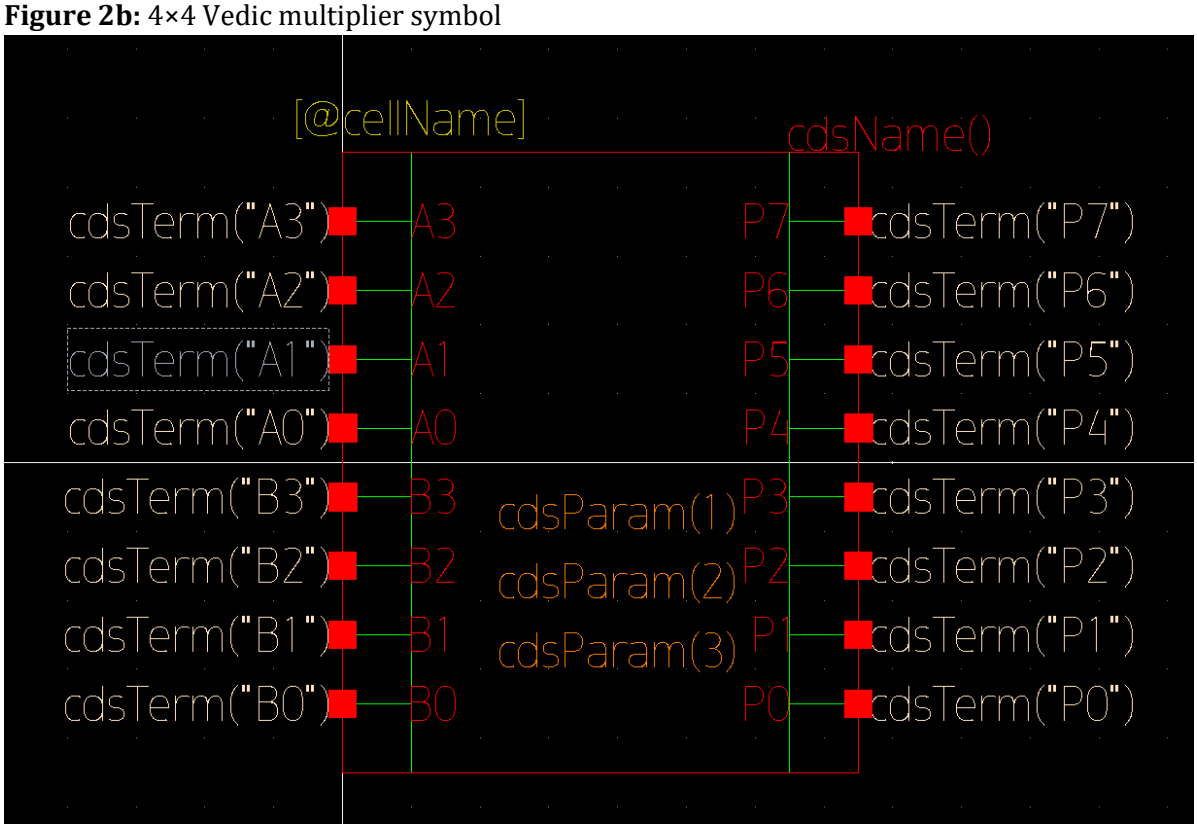


Figure 2b: 4×4 Vedic multiplier symbol



2.2.3 2×2 Vedic Multiplier

The 2X2VEDIC block forms the fundamental arithmetic unit of the design and consists of:

- 4 × AND2 gates
- 2 × Half Adders (HA)

Each half adder combines partial products generated by the AND gates, producing sum and carry-outputs that propagate upward in the hierarchy.

Figure 3a: Internal structure of the 2×2 Vedic multiplier

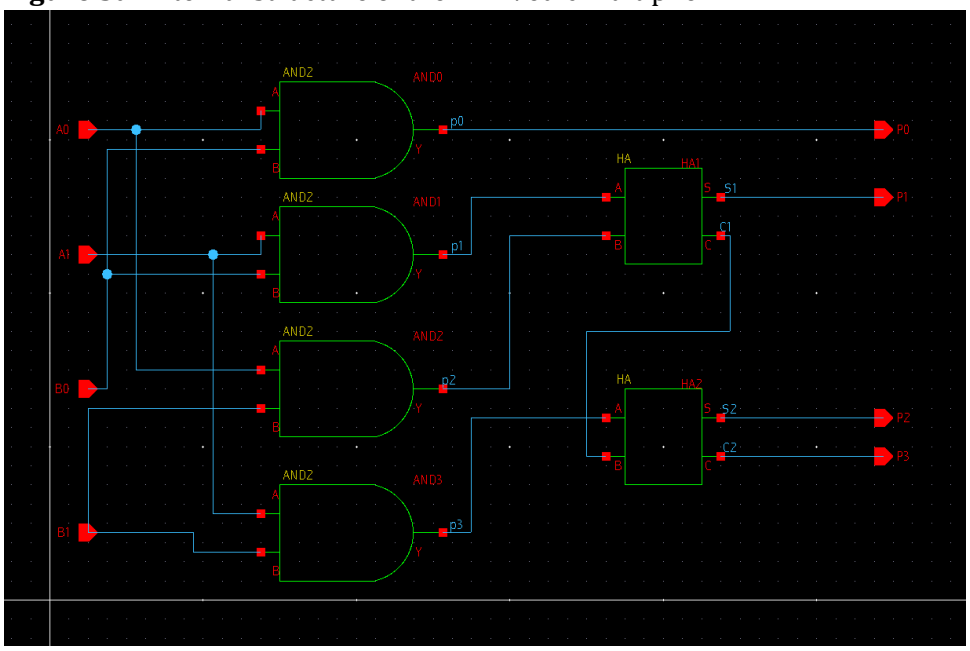
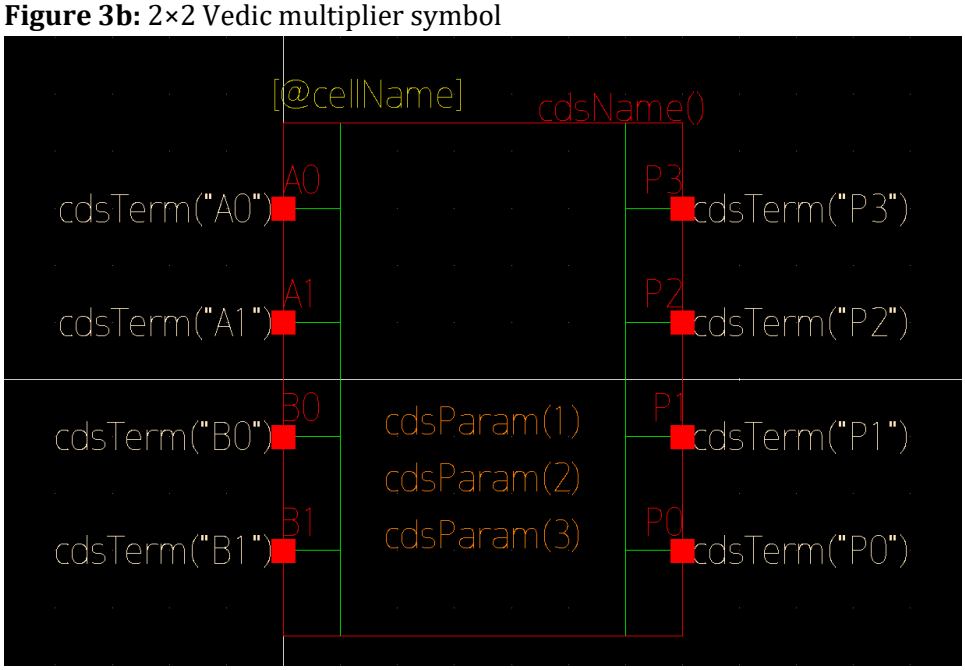


Figure 3b: 2×2 Vedic multiplier symbol



2.3 Arithmetic Building Blocks

2.3.1 Adders

- Half Adder (HA): Implemented using one XOR gate and one AND gate
- Full Adder (FA): Implemented using:
    - 2 XOR gates
    - 2 AND gates
    - 1 OR gate

The 8-bit ripple carry adder (8RCA) consists of one HA followed by seven FAs, while the 16RCA consists of one HA followed by fifteen FAs. Ripple carry adders were selected due to their simplicity, predictable timing behavior, and ease of layout integration, particularly when pipelining limits the carry propagation to a single stage.

2.3.2 Register Design

Pipeline registers are implemented using transmission-gate D flip-flops (TGATE_DFF). Each 16REG block consists of sixteen TGATE_DFF instances, enabling synchronized storage of 16-bit intermediate results.

Figure 4a: 16REG schematic

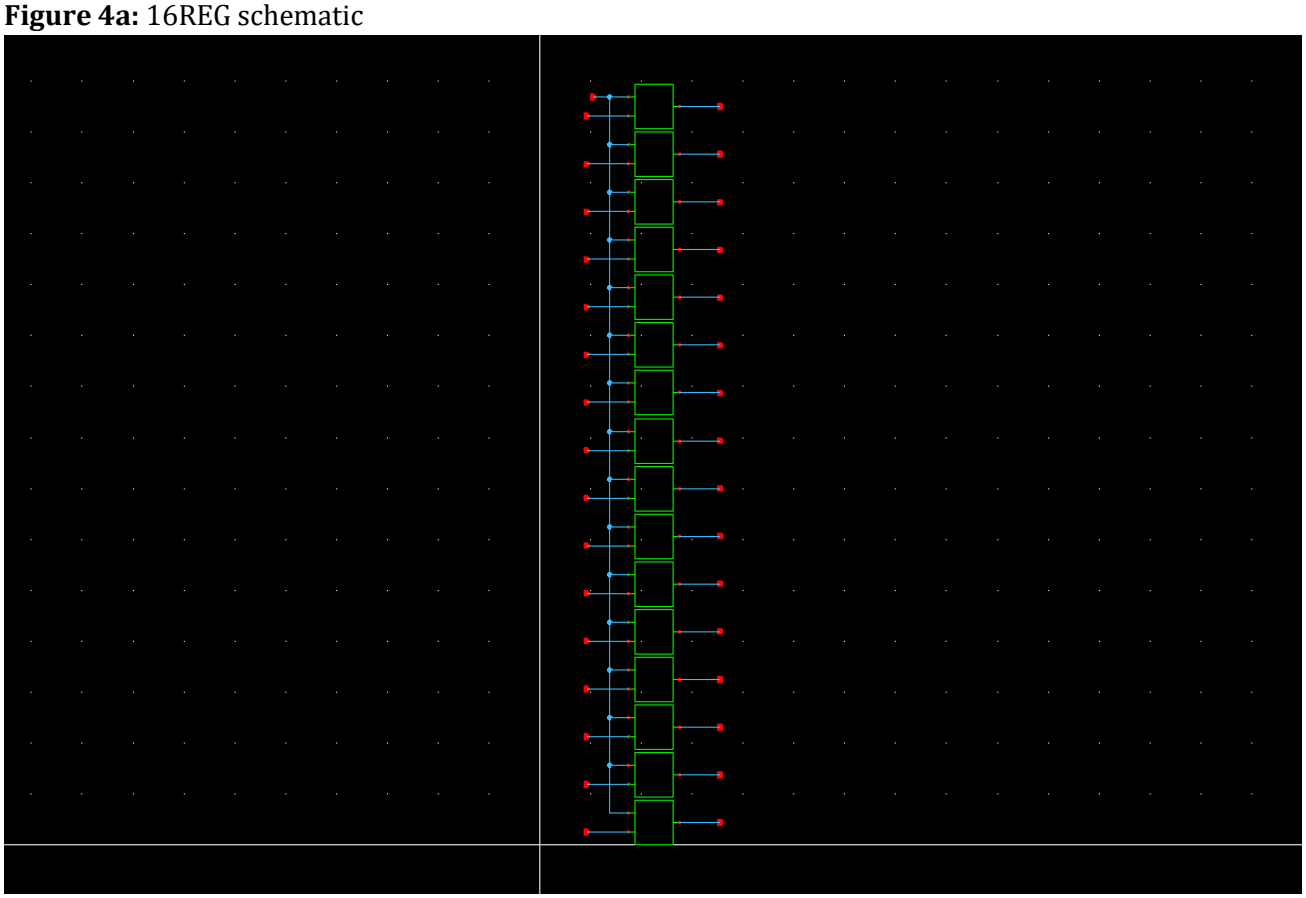


Figure 4b: 16REG schematic (top half)

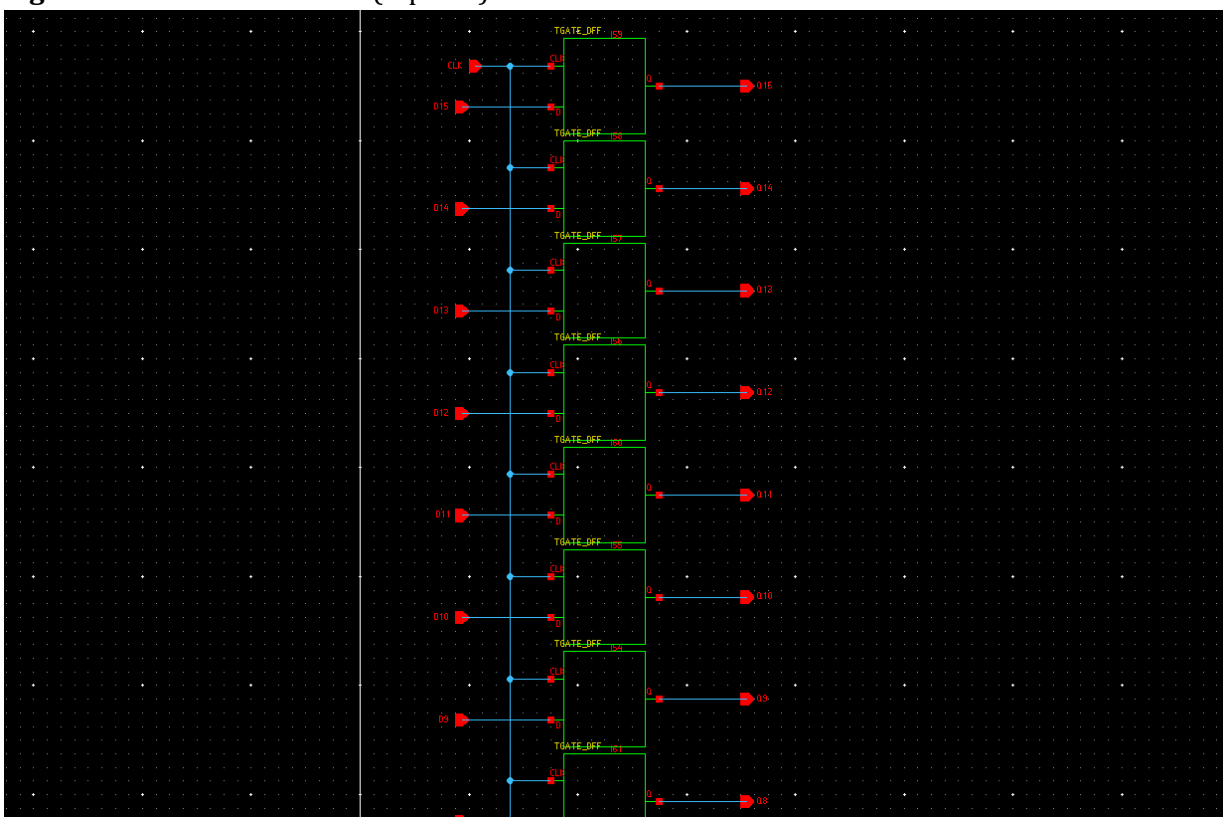


Figure 4c: 16REG schematic (bottom half)

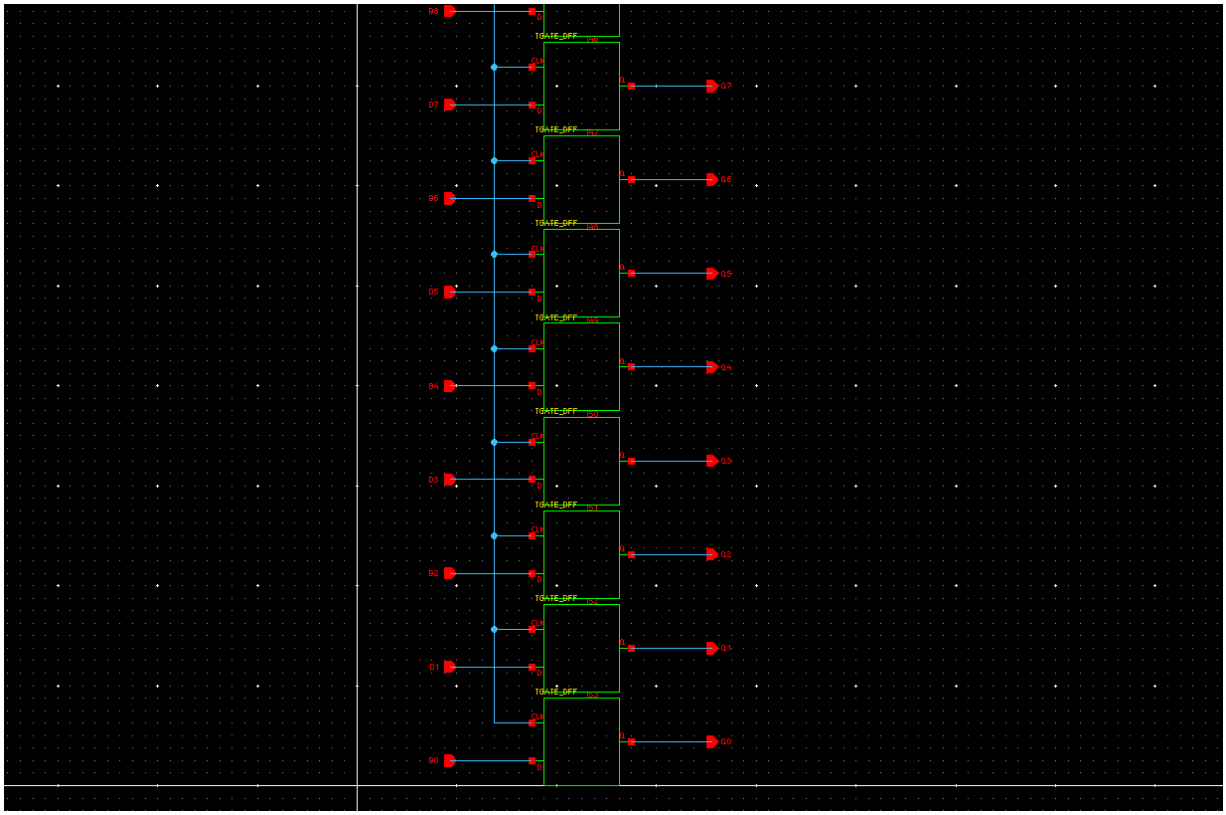


Figure 4d: 16REG symbol

Transmission-gate-based flip-flops were selected for their full rail-to-rail signal swing, reduced static power consumption, and efficient operation at low supply voltages compared to ratioed or dynamic latch designs.

2.4 Pipeline Placement Strategy

The pipeline registers are placed between the 4×4 Vedic multiplier blocks and the 16-bit ripple carry adders. This placement ensures that:

- The longest combinational paths are isolated within a single pipeline stage
- Carry propagation does not span multiple arithmetic levels
- The clock frequency is primarily limited by one RCA stage rather than the entire multiplier

This design choice reflects a deliberate trade-off between increased register count and improved throughput, aligning with performance-driven VLSI design principles [3].

3. Circuit-Level Design and Transistor Sizing

3.1 Design Methodology and Technology Assumptions

All circuits in this project were implemented using full-custom CMOS design techniques in a **90 nm process technology** with a nominal supply voltage of 1.2 V. The design approach followed a strict bottom-up methodology, beginning with primitive logic gates and progressing hierarchically toward complex arithmetic and sequential blocks. Each primitive cell was first verified at the schematic level before being reused in higher-level modules.

All simulations were performed using a testbench, with Prime Wave with HSPICE, which provides a balanced representation of nominal device behavior and is appropriate for functional verification and comparative design evaluation [2].

3.2 Primitive Logic Gates

3.2.1 Inverter (INV)

The CMOS inverter serves as a fundamental building block throughout the design, appearing in logic gates, transmission gates, and register structures. The inverter was sized with the following parameters:

- **NMOS:** W = 0.5 µm, L = 0.1 µm
- **PMOS:** W = 1.0 µm, L = 0.1 µm

Figure 5a: CMOS inverter schematic with transistor dimensions

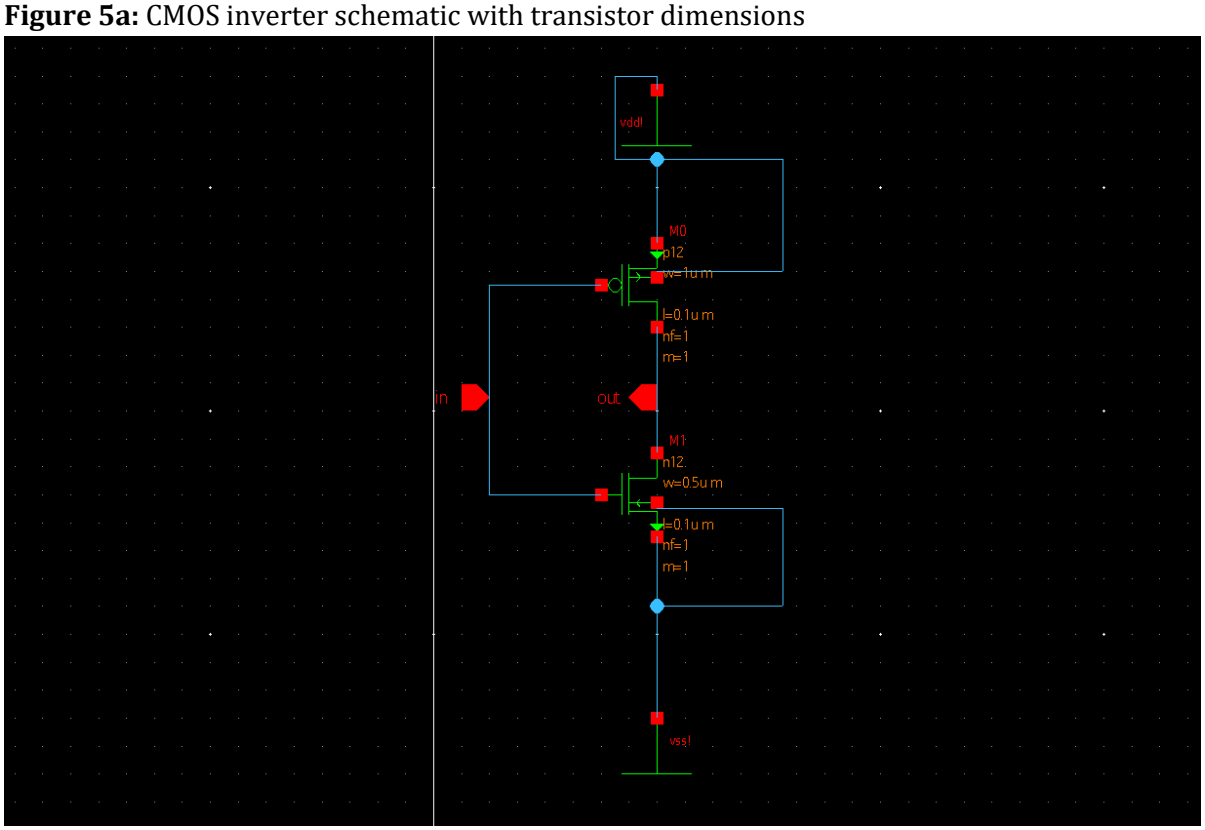
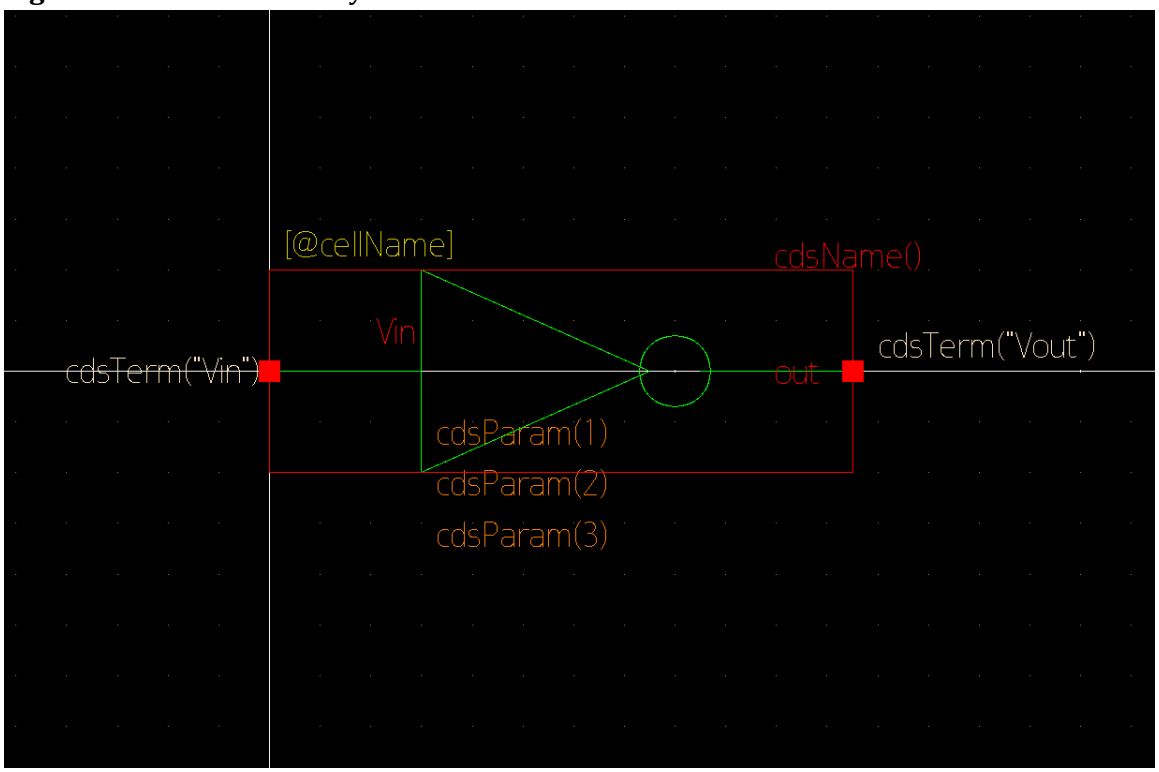


Figure 5b: CMOS inverter symbol



The PMOS-to-NMOS-width ratio of **2:1** was selected to compensate for the lower hole mobility in PMOS devices, resulting in more balanced rise and fall times. Maintaining minimum channel length improves switching speed while remaining compliant with the 90 nm design rules [4].

3.2.2 NAND Gate (NAND2)

The two-input NAND gate is used extensively as a primitive logic element and serves as the foundation for constructing AND gates and other combinational logic.

Transistor dimensions:

- **NMOS:** W = 1.0 µm, L = 0.1 µm
- **PMOS:** W = 1.0 µm, L = 0.1 µm

Figure 6a: Two-input CMOS NAND gate schematic

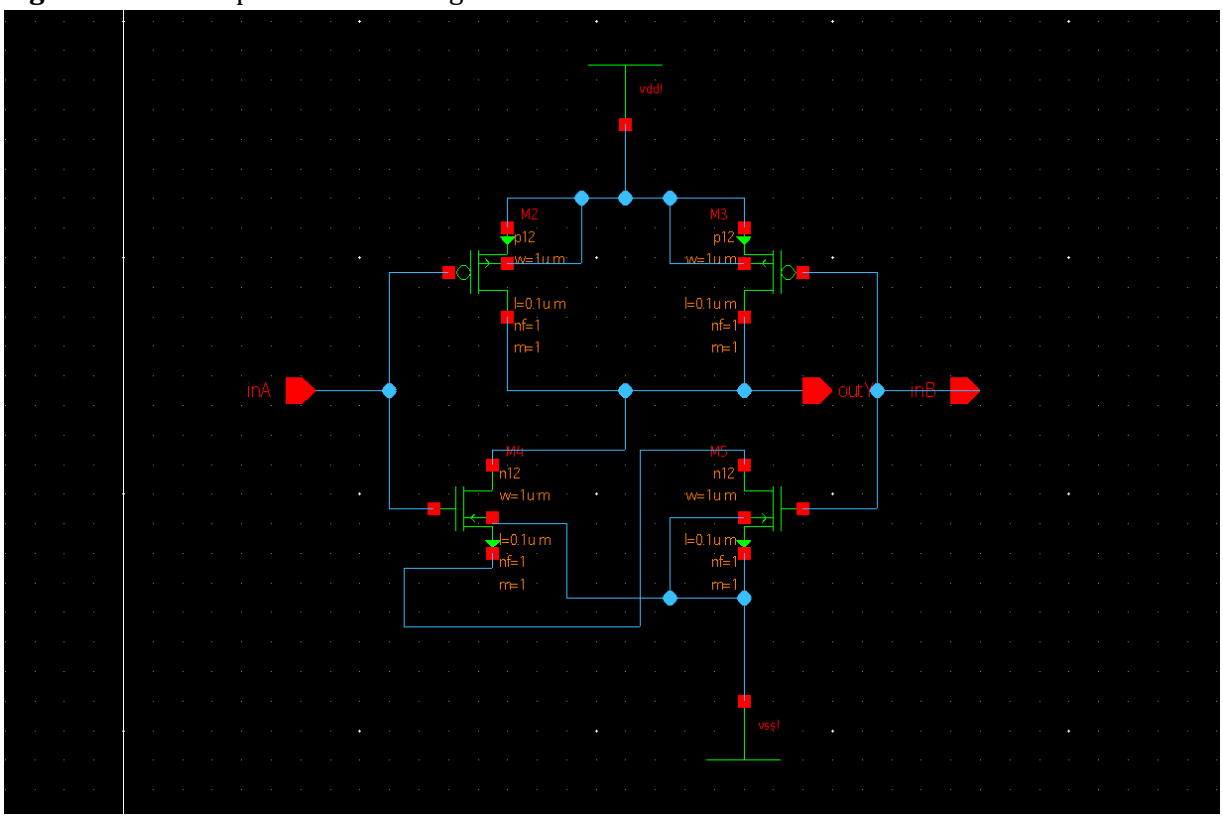
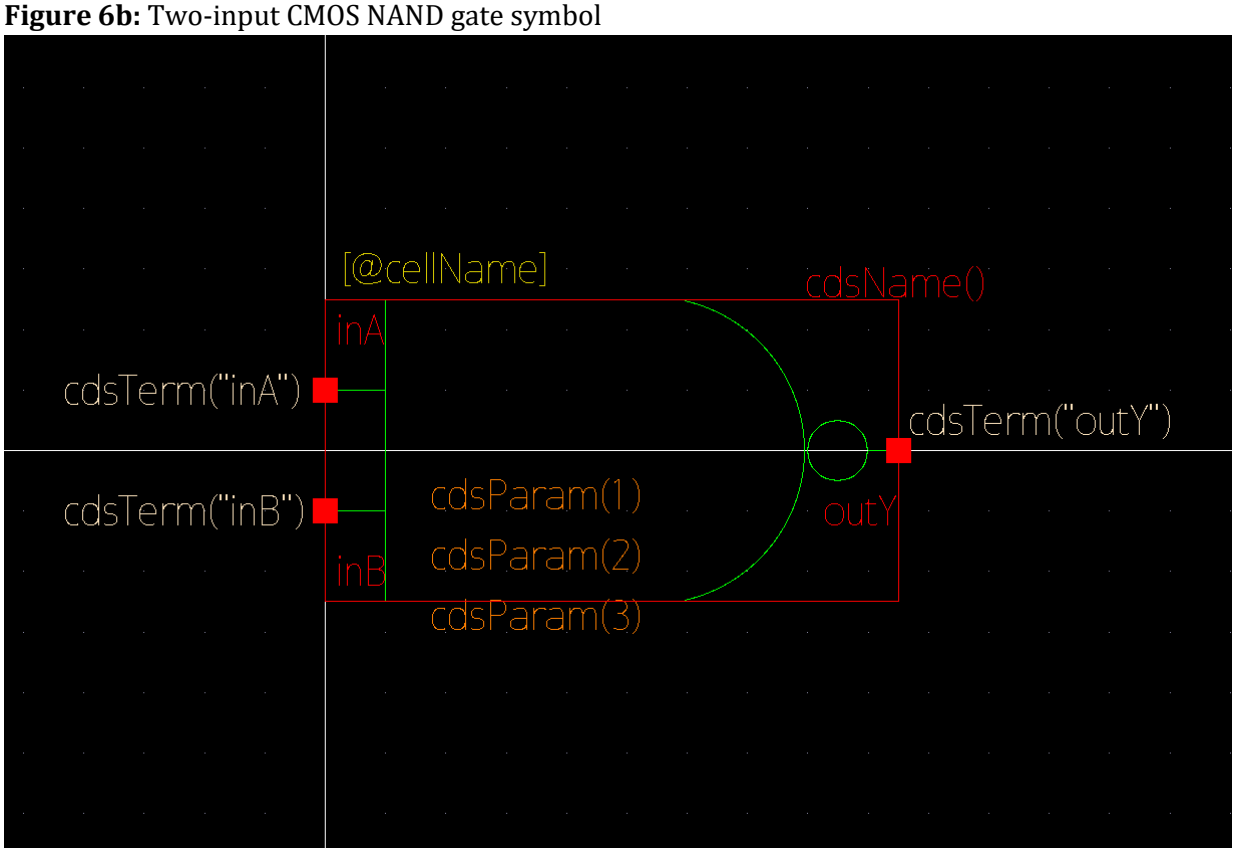


Figure 6b: Two-input CMOS NAND gate symbol



The NMOS devices are connected in series, while the PMOS devices are connected in parallel. To mitigate the increased resistance caused by series-connected NMOS transistors, the NMOS width was increased relative to the inverter NMOS. Equal PMOS sizing ensures sufficient pull-up strength under worst-case input combinations [4].

3.2.3 AND Gate (AND2)

The AND gate was implemented using a **NAND gate followed by an inverter**, rather than a direct CMOS AND structure. This approach offers several advantages:

- Simpler and more robust transistor-level implementation
- Improved noise margins
- Reuse of well-characterized primitive cells

Figure 7a: AND gate implemented using NAND + INV

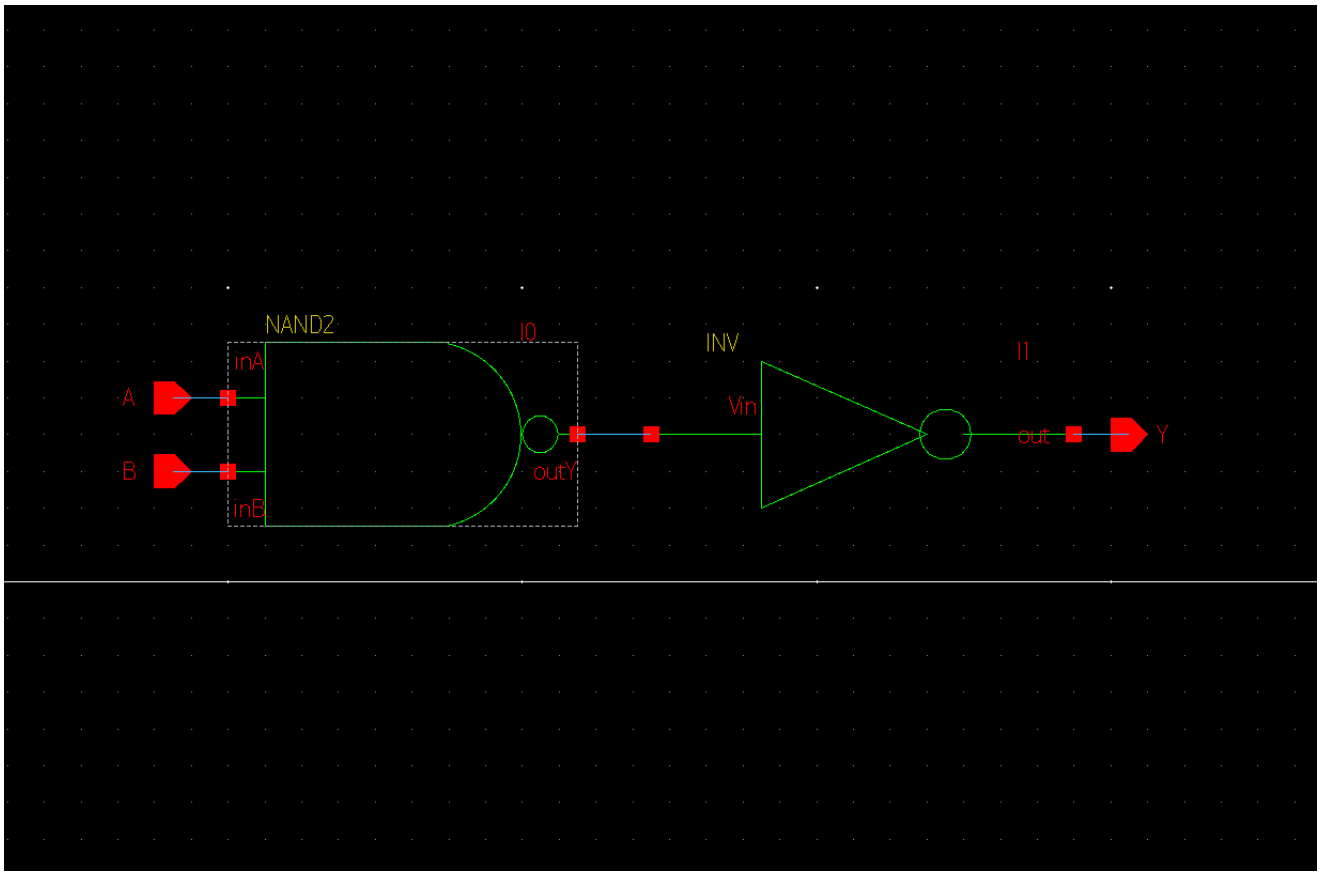
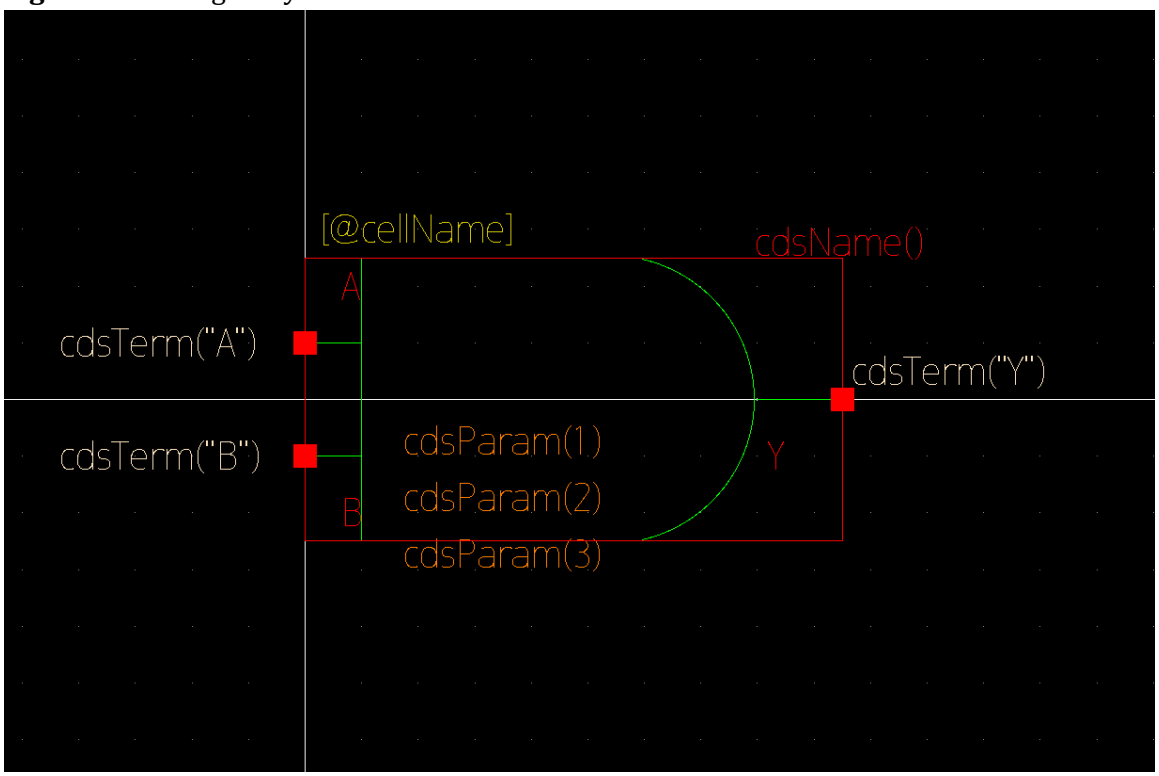


Figure 7b: AND gate symbol



This design choice prioritizes reliability and modularity over minimal transistor count, which is appropriate for a hierarchical full-custom design [4].

3.3 XOR Gate Design

The XOR gate is a critical component in both half adders and full adders, directly influencing the delay and power consumption of the arithmetic blocks.

Transistor dimensions:

- **NMOS:** W = 0.5 μm, L = 0.1 μm
- **PMOS:** W = 1.0 μm, L = 0.1 μm

Figure 8a: XOR gate schematic

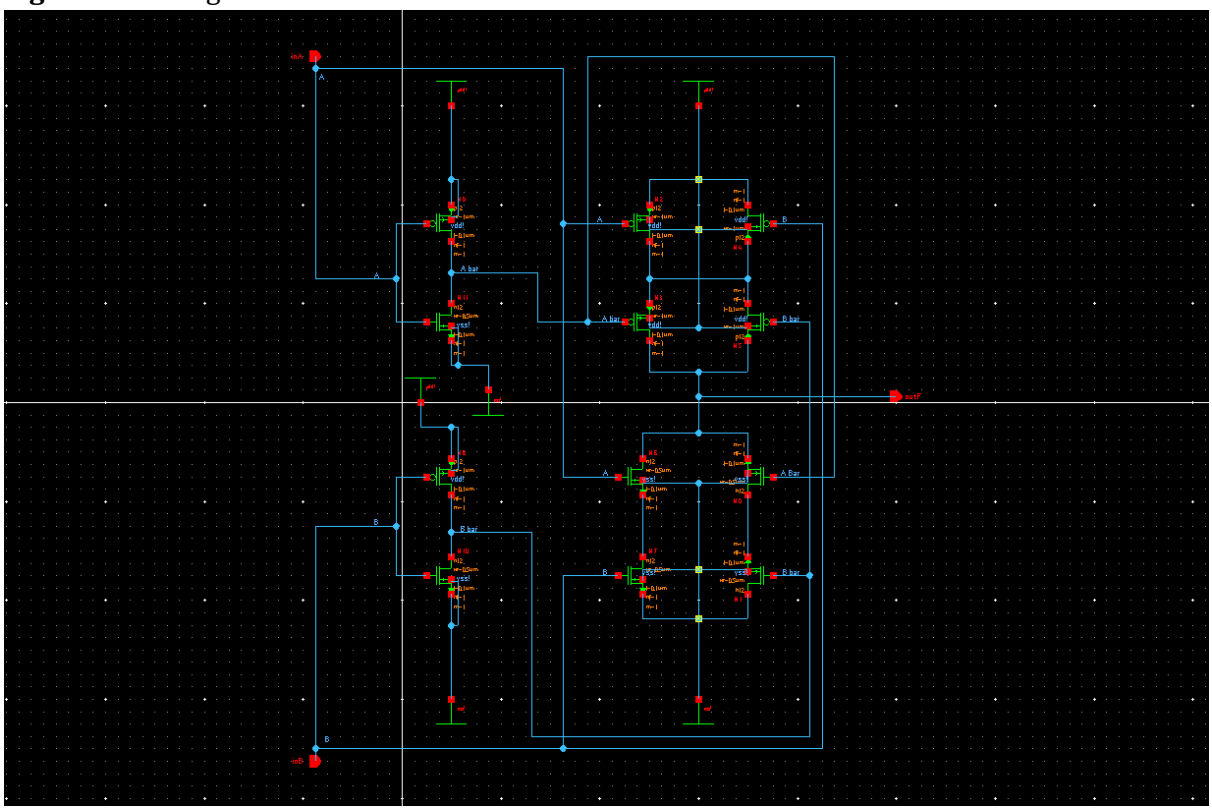
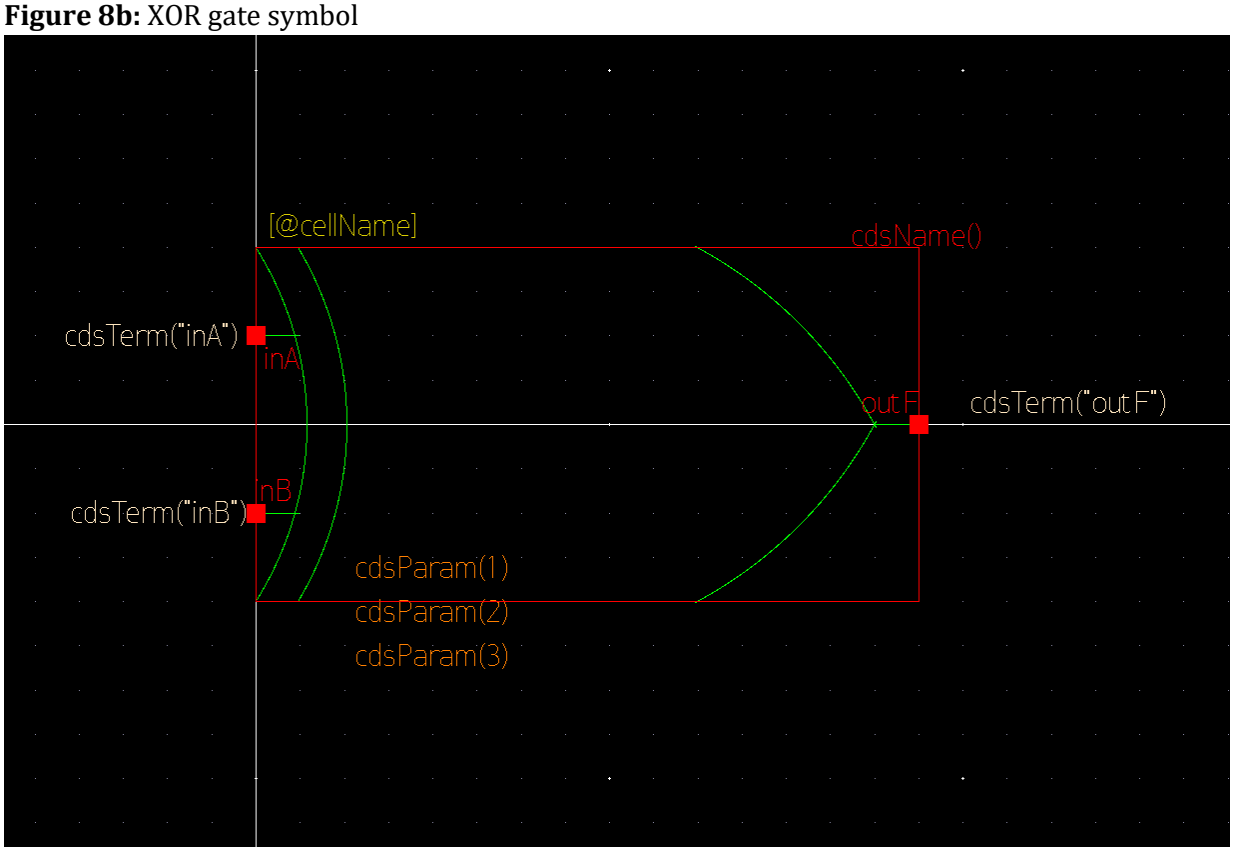


Figure 8b: XOR gate symbol



The XOR gate was designed to maintain full logic swing and correct functionality across all input combinations. Balanced transistor sizing was chosen to reduce asymmetry in rise and fall delays, which is particularly important in arithmetic circuits where timing mismatches can propagate through multiple stages.

3.4 Adder Circuits

3.4.1 Half Adder (HA)

The half adder is composed of:

- One XOR gate (sum output)
- One AND gate (carry output)

Figure 9a: Half adder schematic

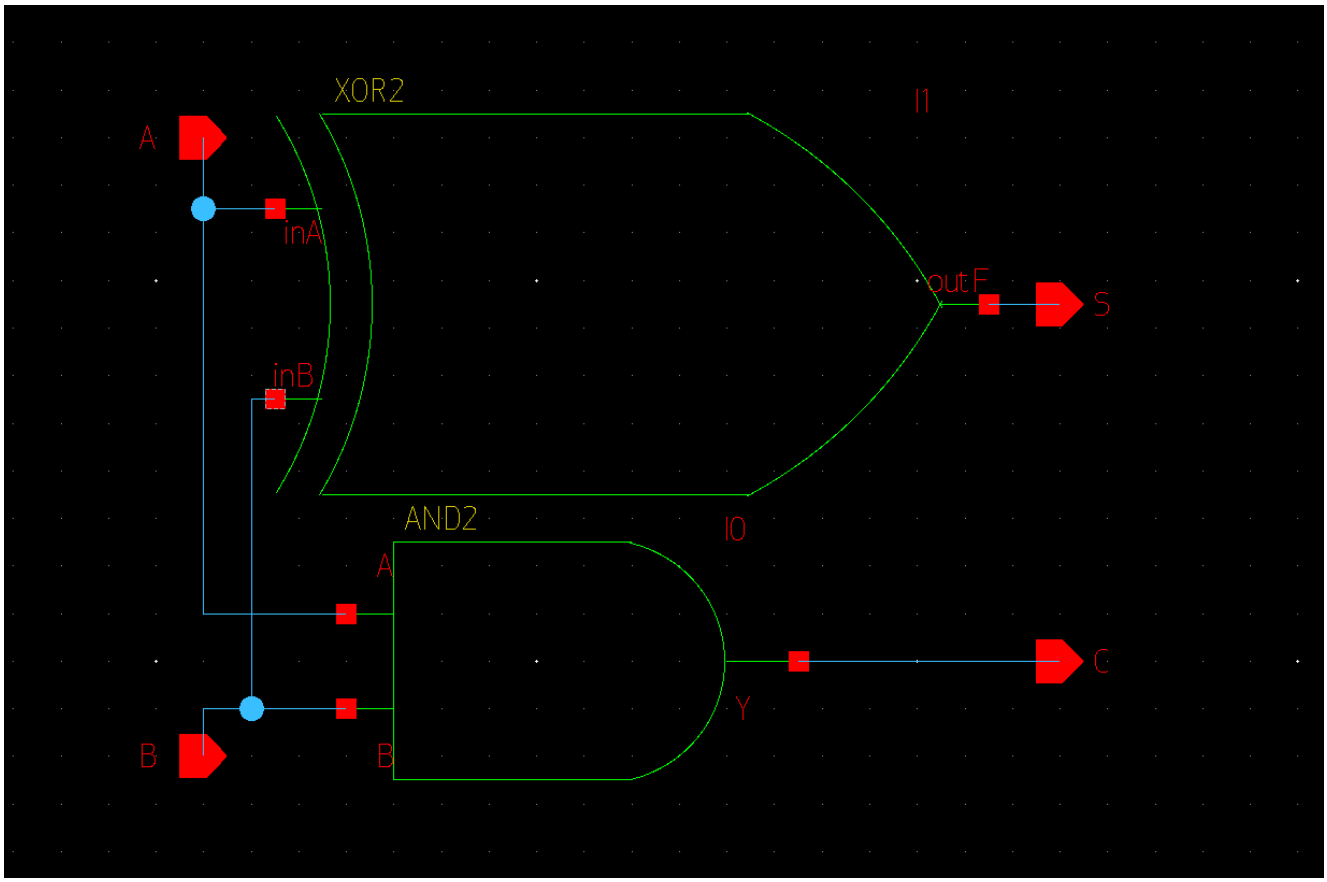
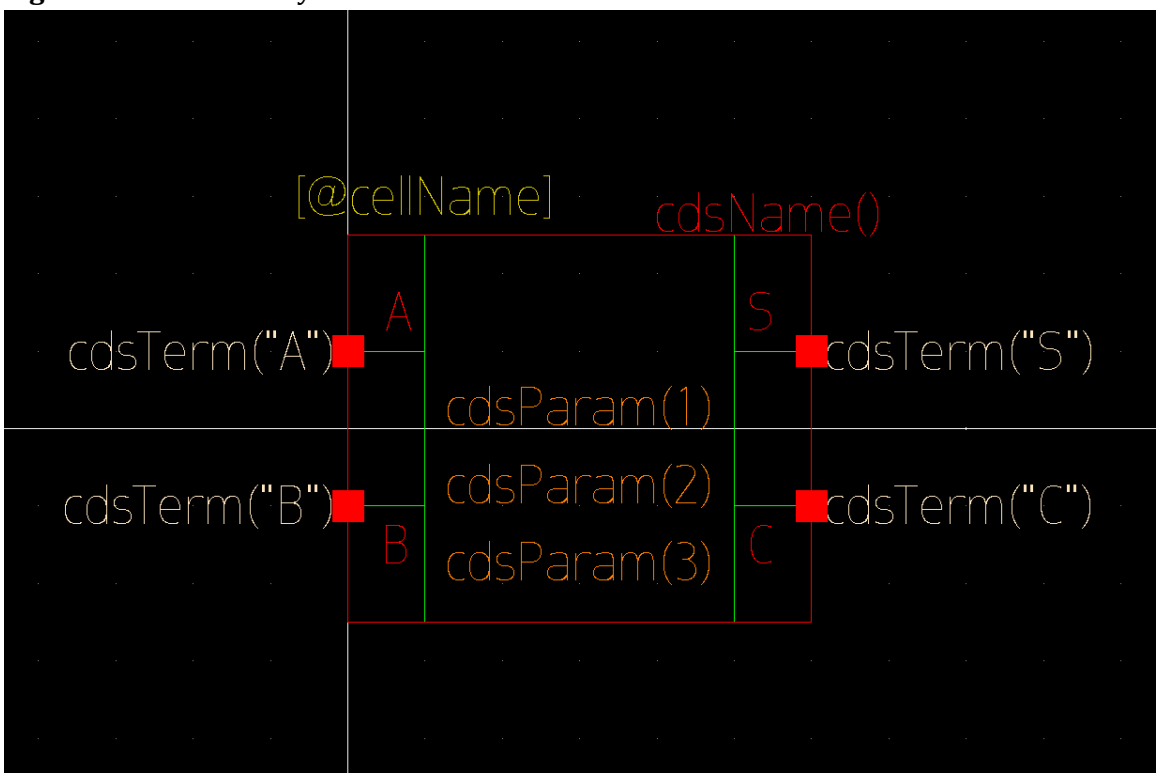


Figure 9b: Half adder symbol



This modular construction simplifies verification and allows consistent reuse across different adder widths.

3.4.2 Full Adder (FA)

The full adder is constructed using:

- **2 XOR gates**
- **2 AND gates**
- **1 OR gate** (implemented using NOR + INV)

Figure 10a: Full adder schematic

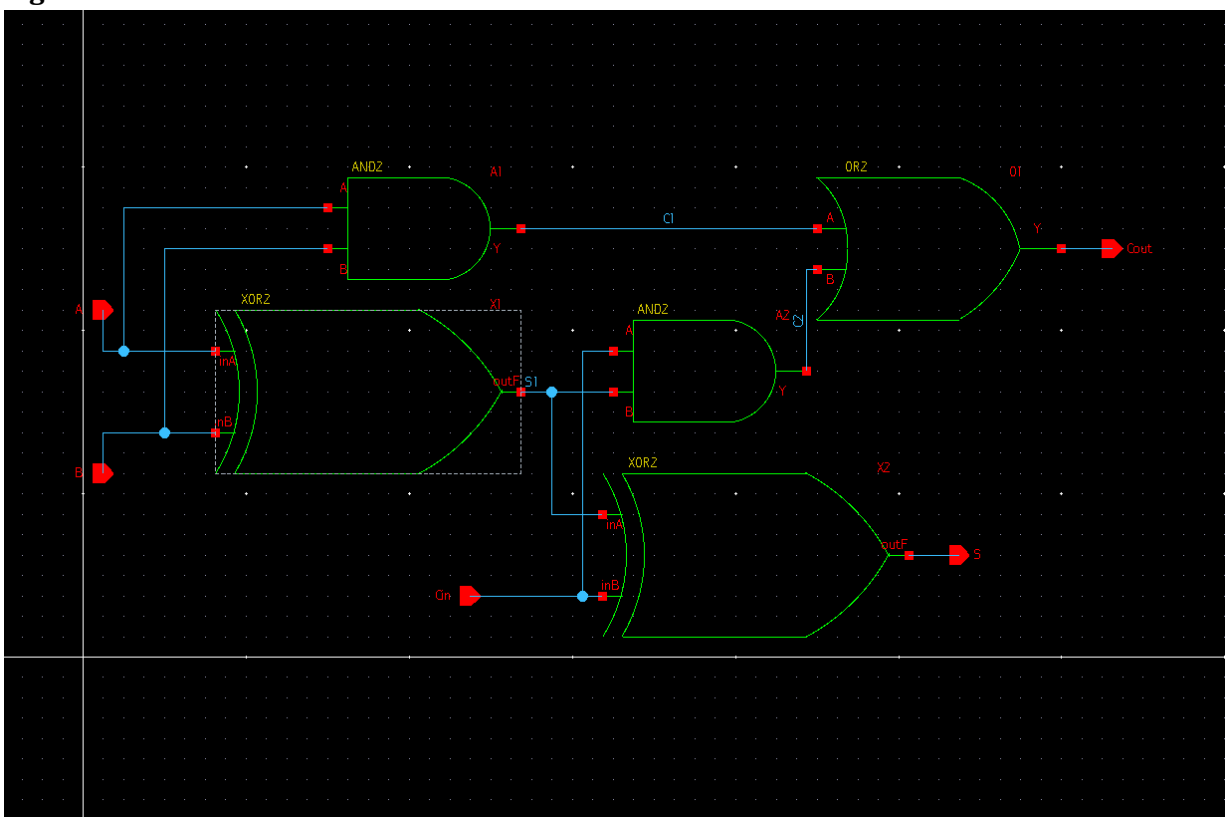
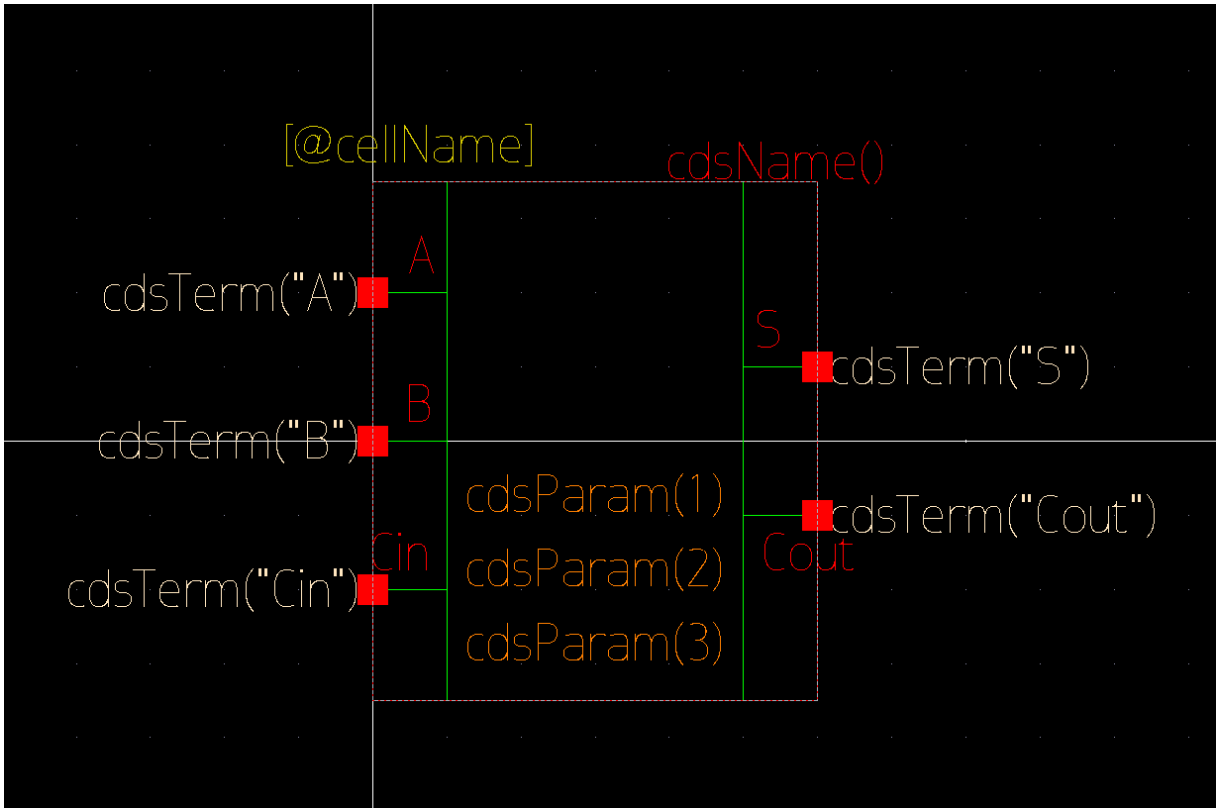


Figure 10b: Full adder symbol



The OR gate was implemented as a **NOR gate followed by an inverter**, consistent with the design philosophy used for the AND gate.

NOR gate transistor dimensions:

- **NMOS:** W = 0.5 µm, L = 0.1 µm
- **PMOS:** W = 2.0 µm, L = 0.1 µm

The larger PMOS width compensates for the series-connected PMOS network in the NOR gate, ensuring acceptable pull-up strength and balanced switching behavior.

Figure 11a: NOR schematic

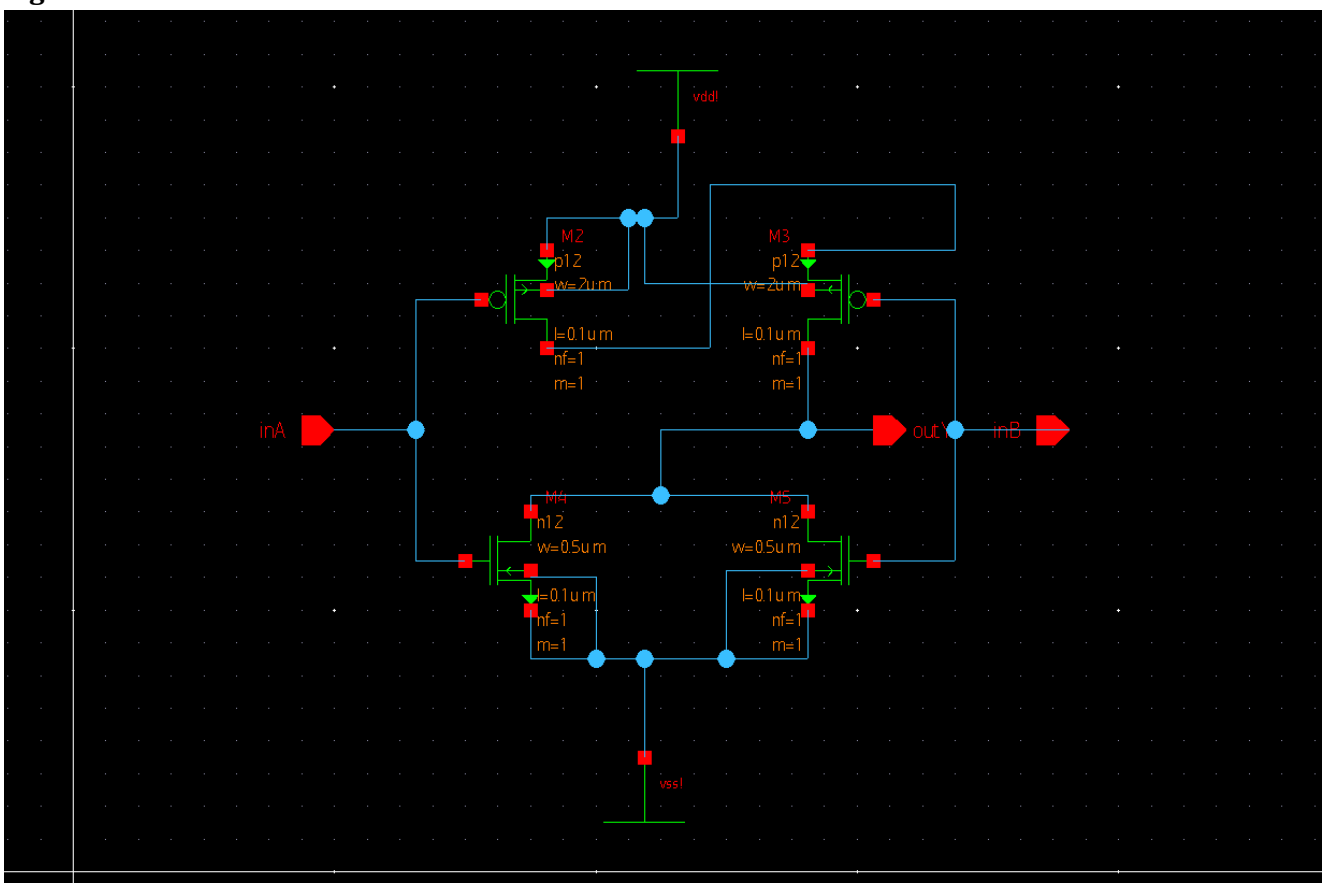


Figure 11b: NOR symbol

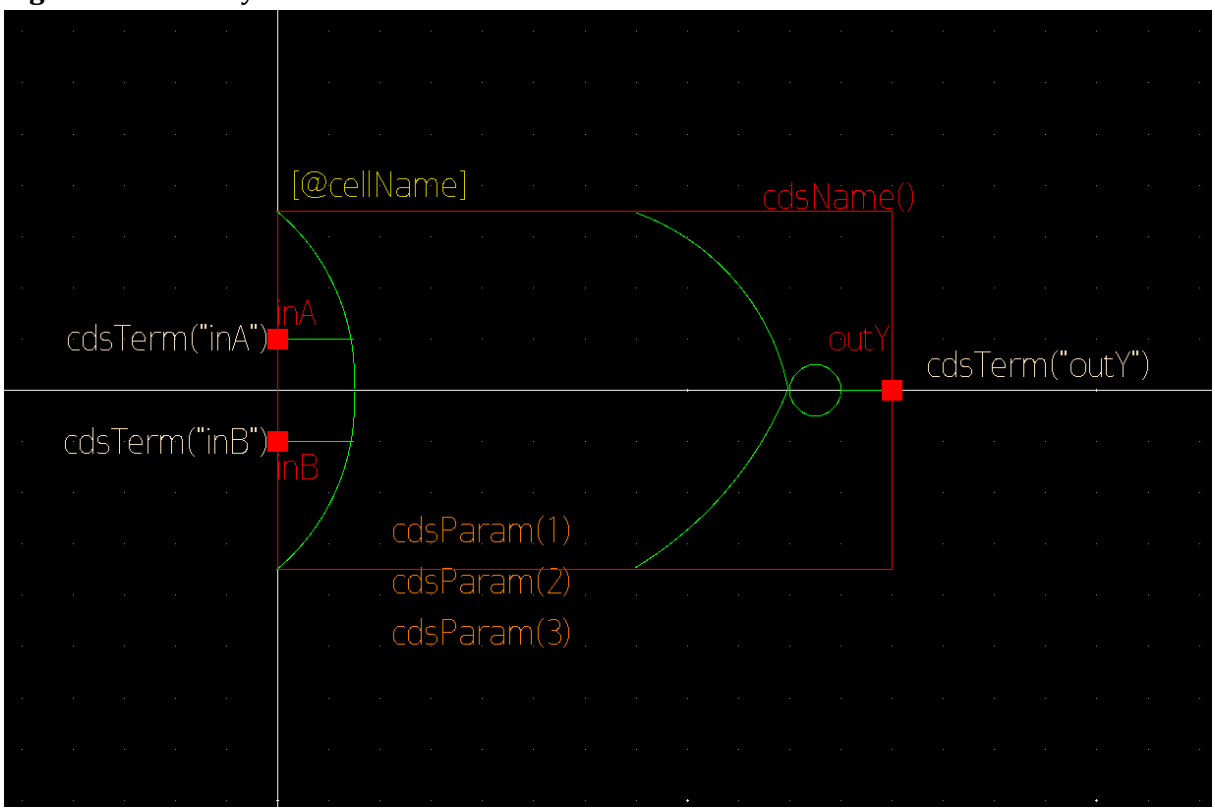


Figure 12a: OR schematic

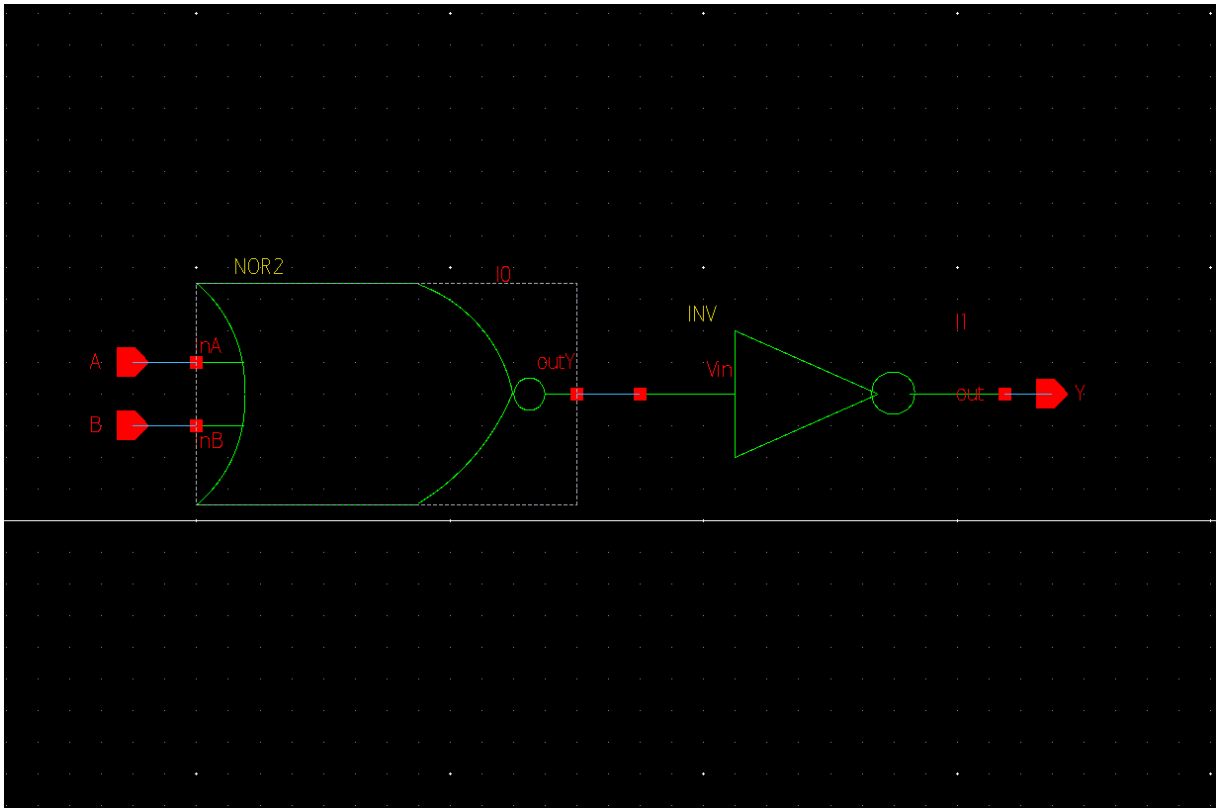
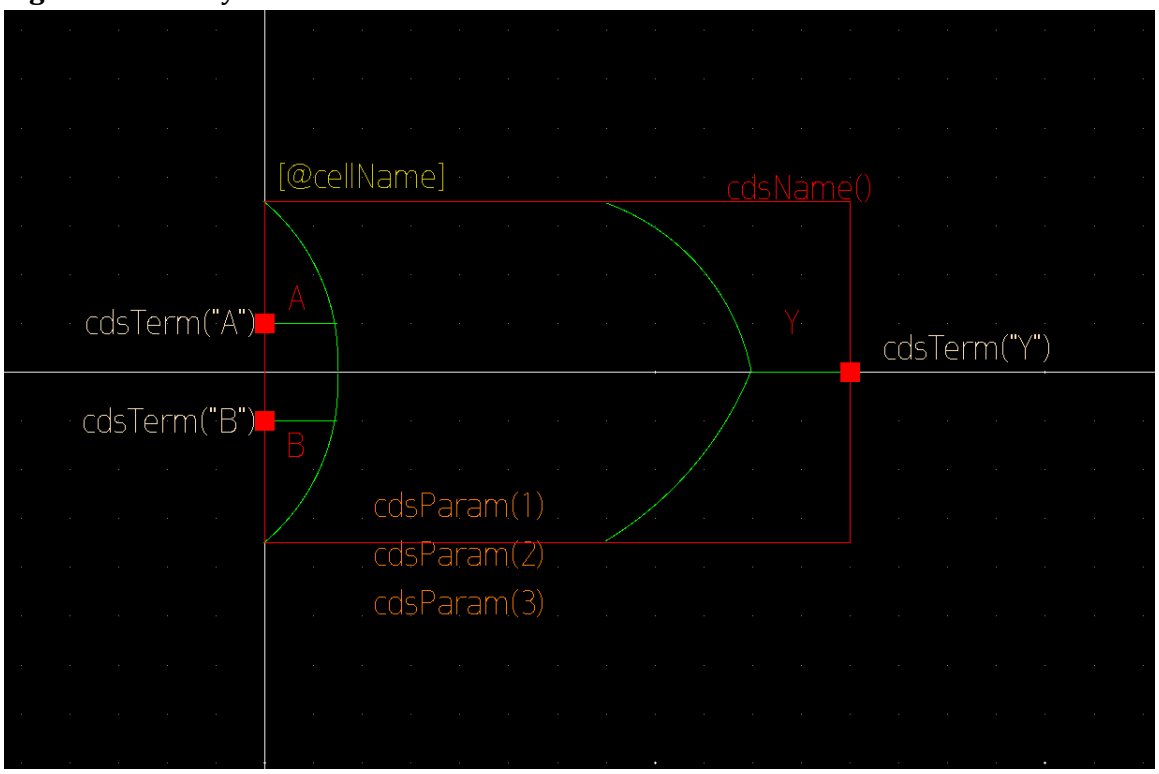


Figure 12b: OR symbol



3.4.3 Ripple Carry Adders

Ripple carry adders were selected for both 8-bit and 16-bit addition stages due to their simplicity, predictable timing behavior, and ease of layout integration.

- **8RCA:** 1 HA + 7 FA
- **16RCA:** 1 HA + 15 FA

Although ripple carry adders exhibit linear carry propagation delay, the use of pipelining confines this delay to a single pipeline stage, making the overall timing acceptable for the target clock frequency.

Figure 13a: 8RCA schematic

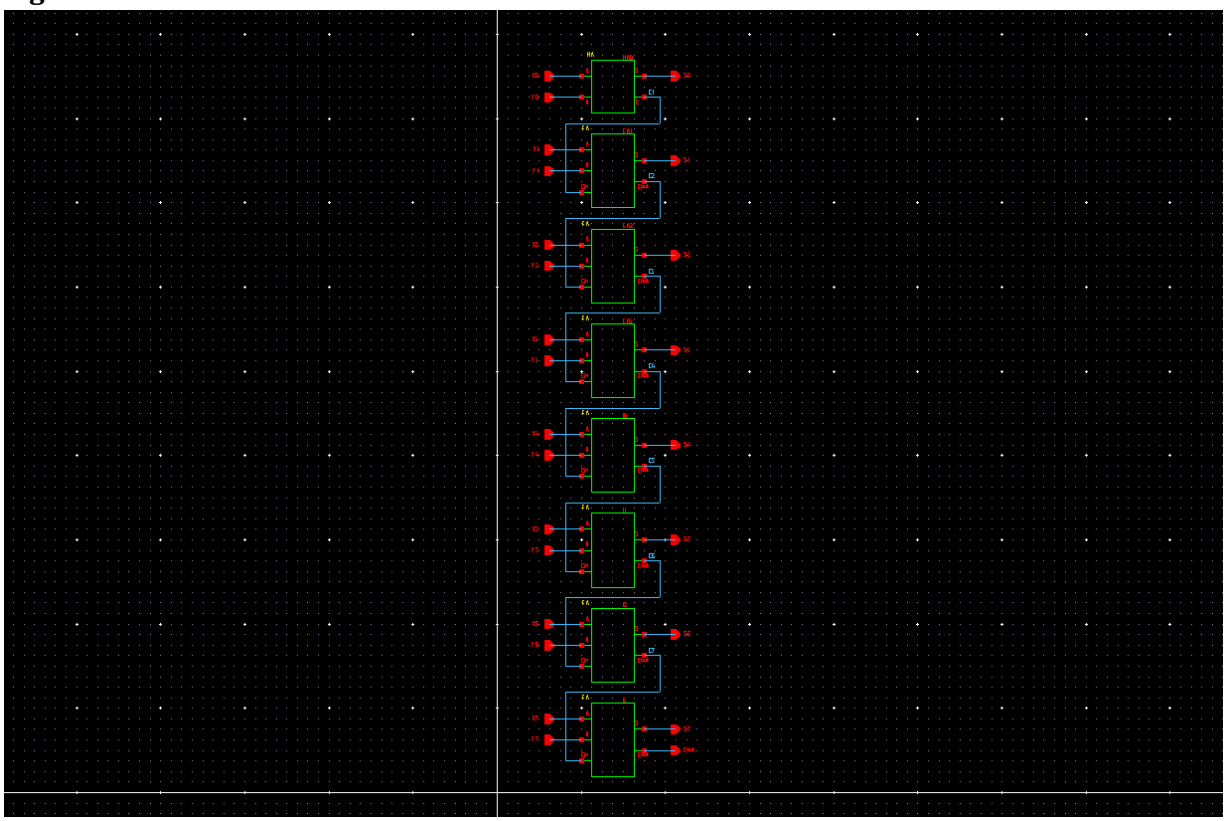


Figure 13b: 8RCA symbol

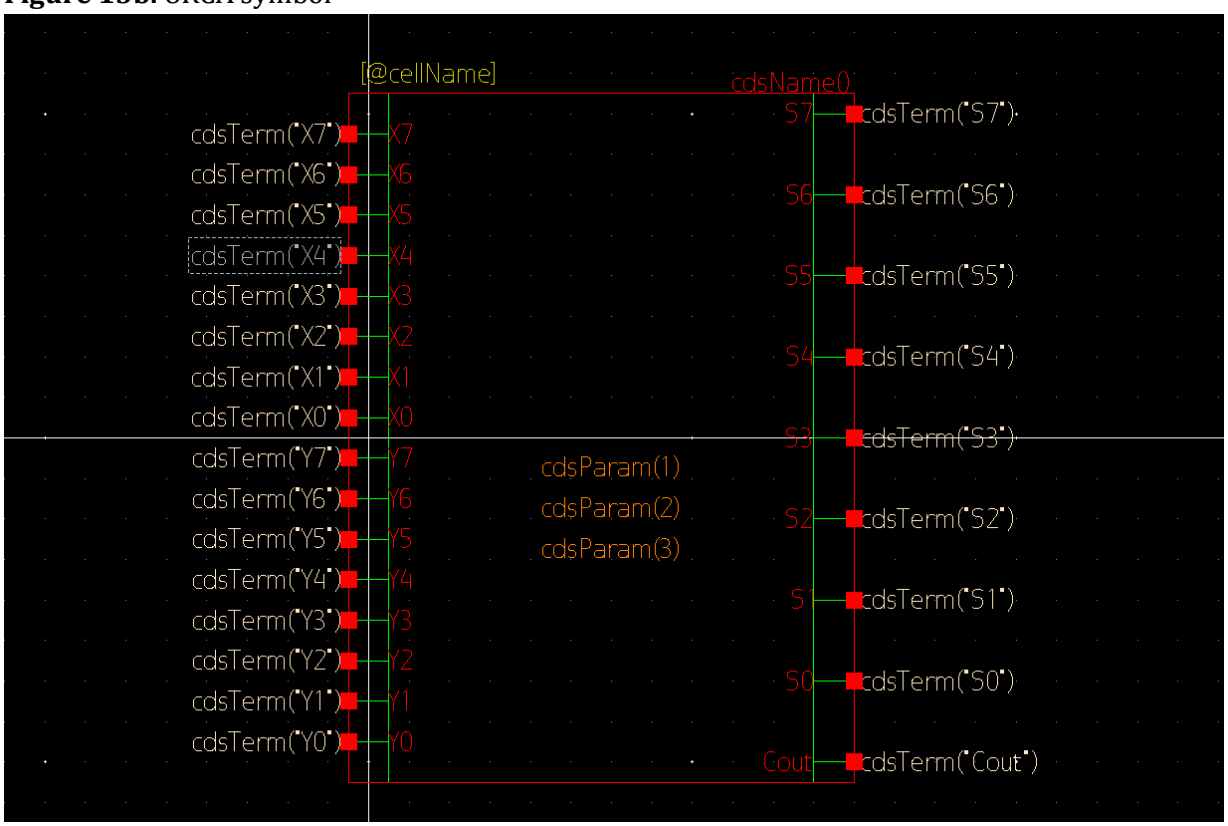


Figure 14a: 16RCA schematic

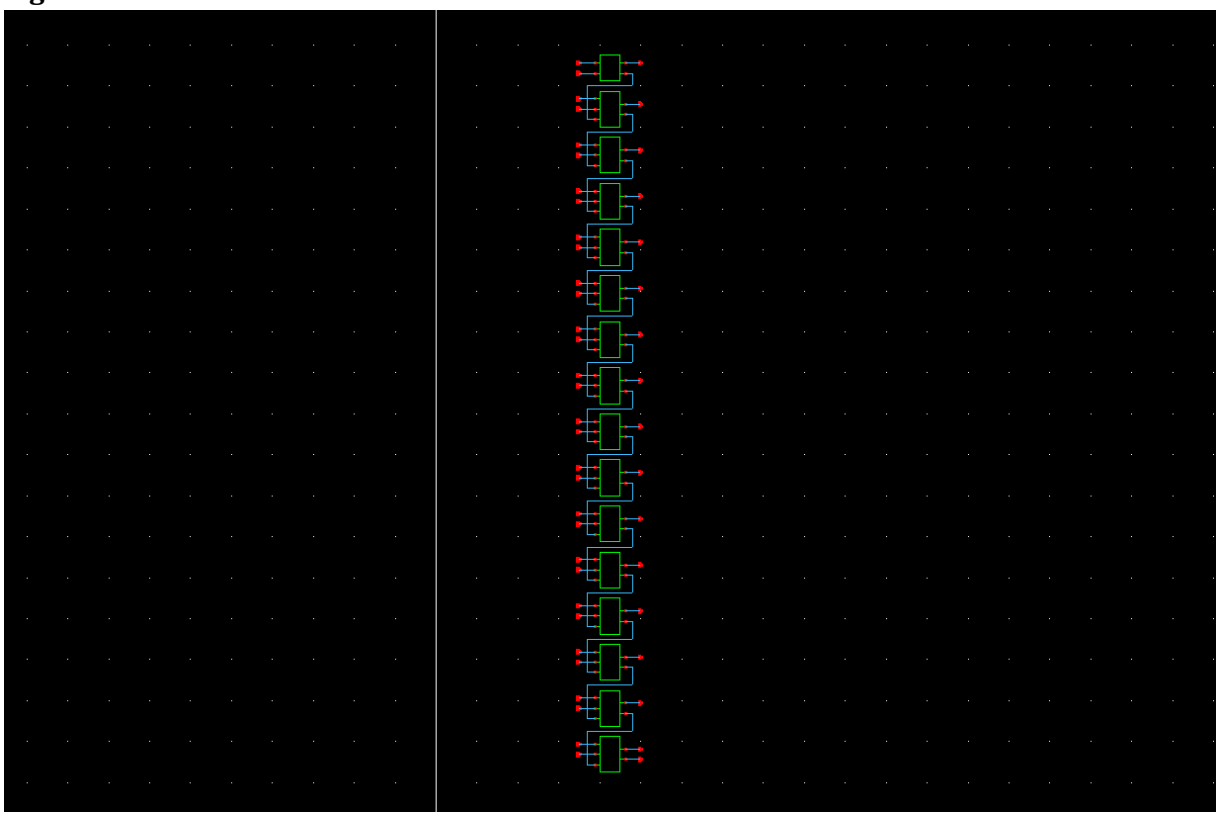


Figure 14b: 16RCA schematic (top half)

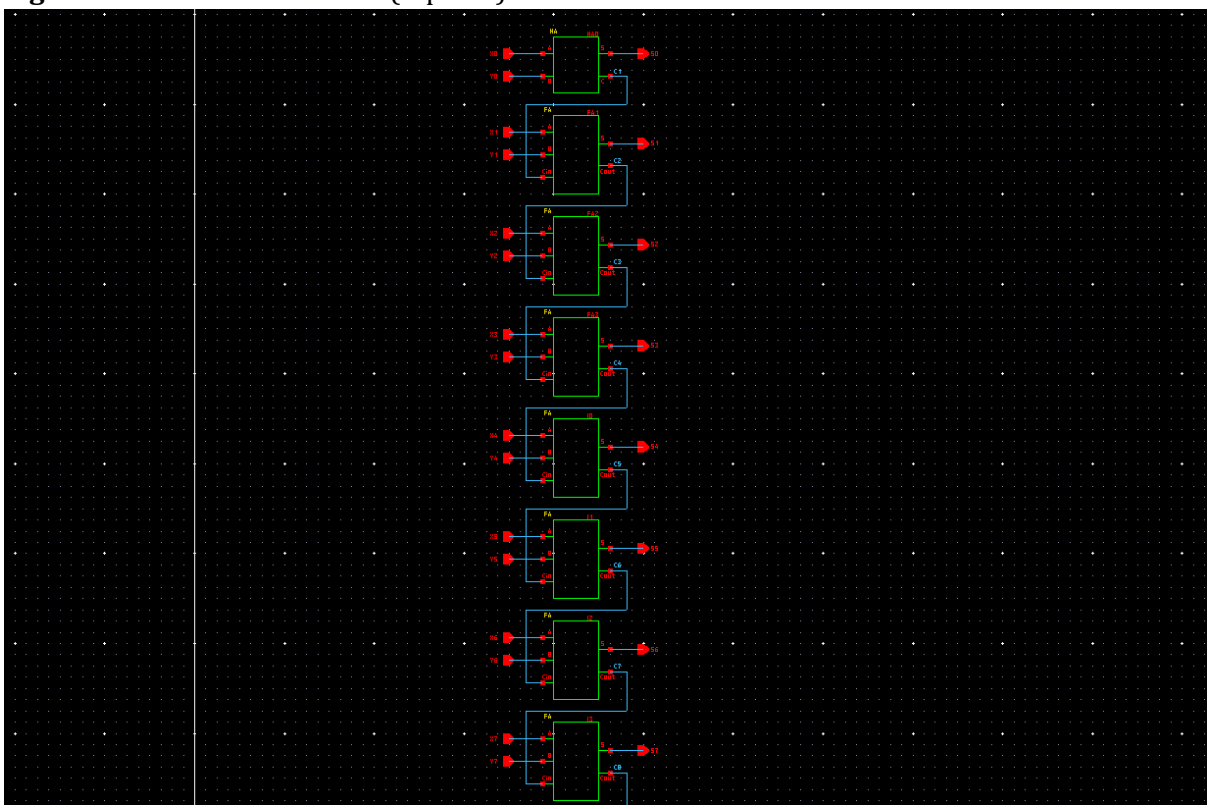


Figure 14c: 16RCA schematic (bottom half)

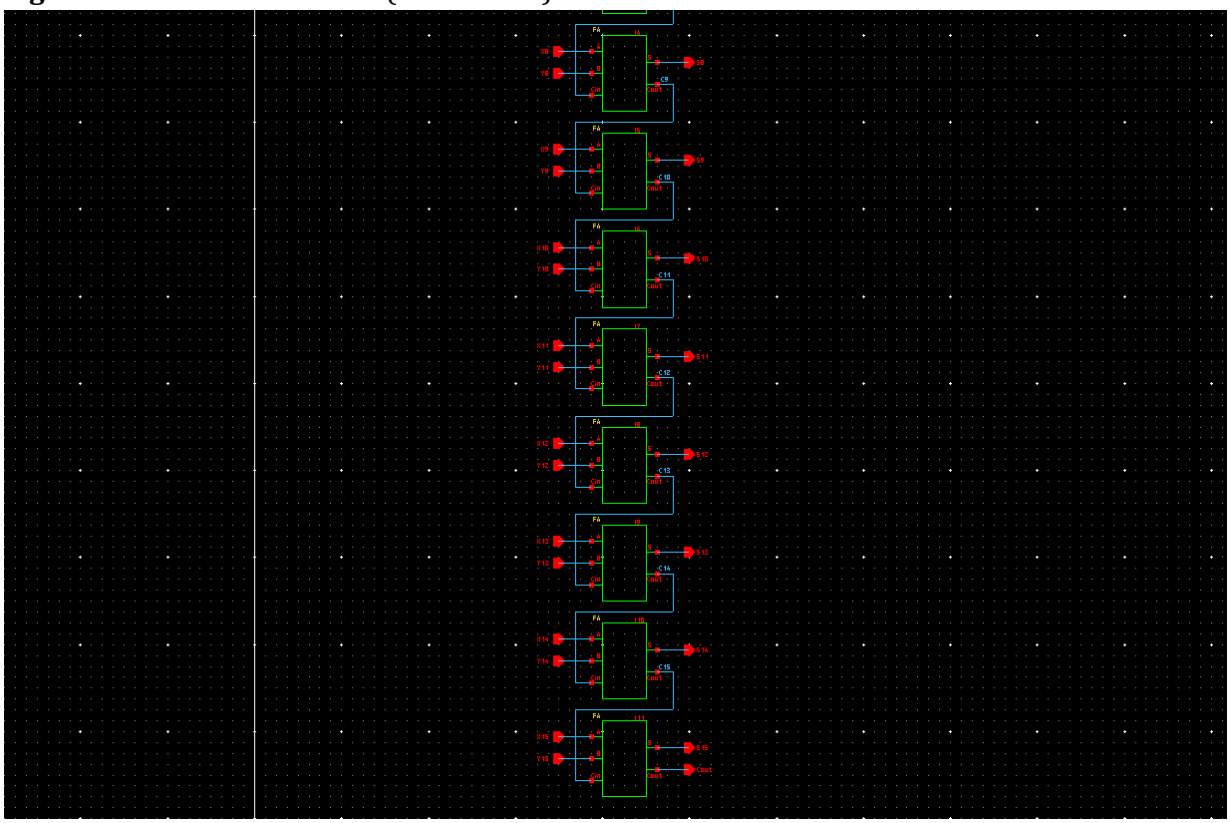
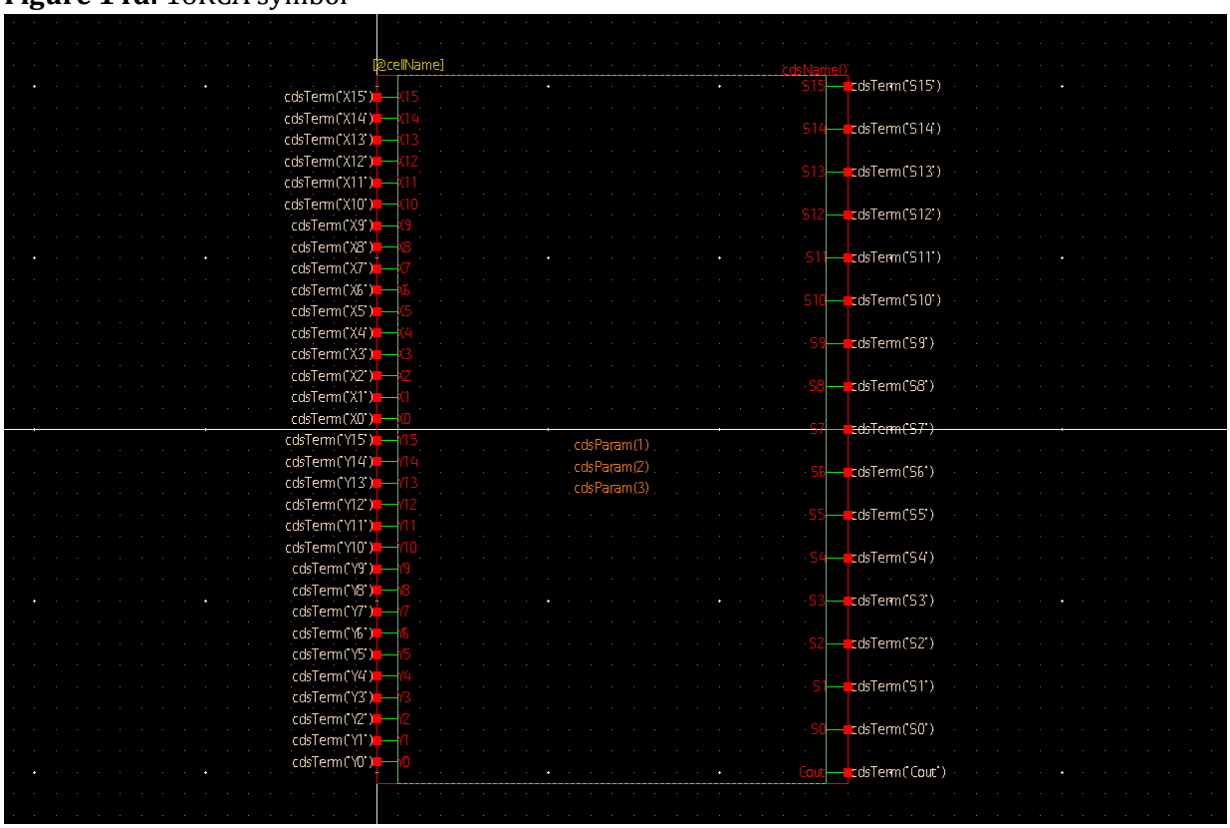


Figure 14d: 16RCA symbol



3.5 Transmission-Gate D Flip-Flop (TGATE_DFF)

Pipeline registers are implemented using transmission-gate-based D flip-flops, which provide robust operation and full voltage swing [5].

3.5.1 Transmission Gate (TGATE)

Transistor dimensions:

- **NMOS:** W = 0.5 µm, L = 0.1 µm
- **PMOS:** W = 1.0 µm, L = 0.1 µm

Figure 15a: Transmission gate schematic

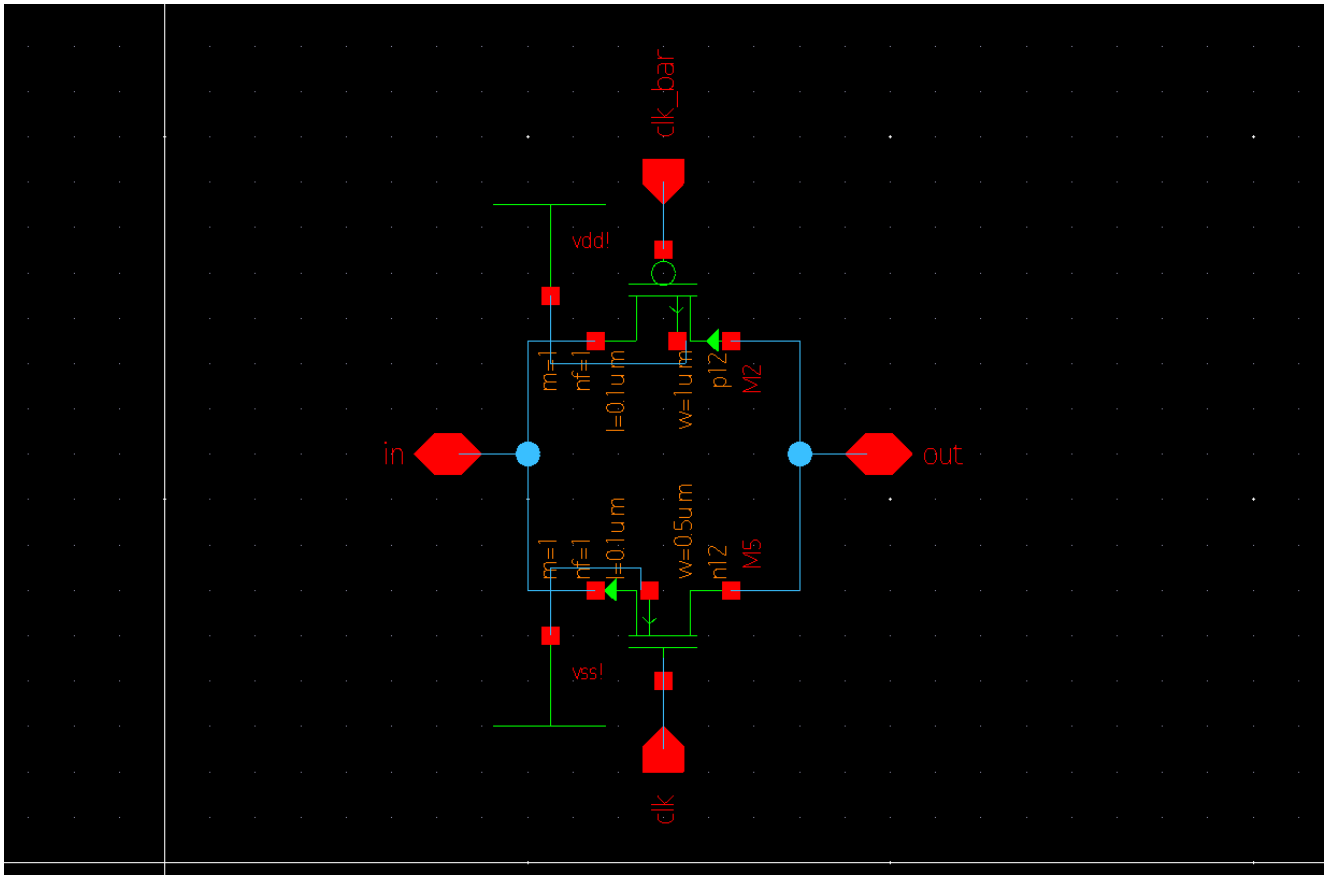
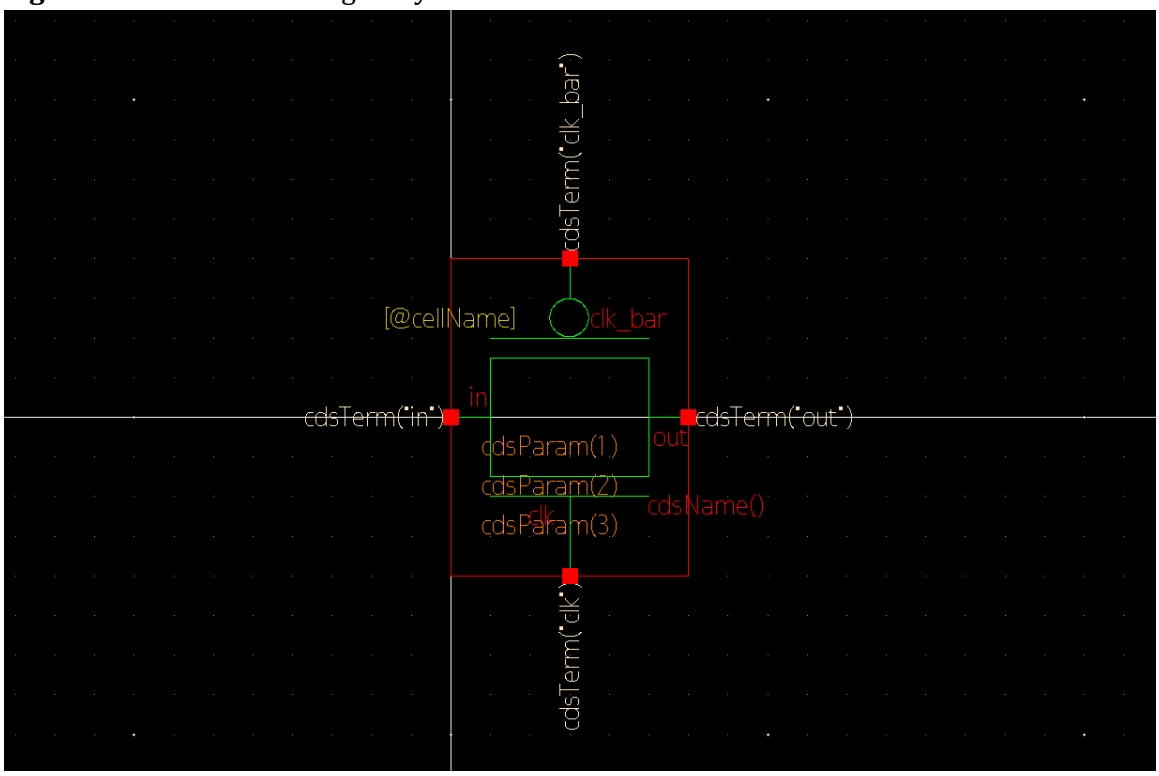


Figure 15b: Transmission gate symbol



The complementary NMOS and PMOS devices ensure low on-resistance for both logic-high and logic-low signals, avoiding threshold voltage degradation commonly observed in pass-transistor-only designs [5].

3.5.2 TGATE D Flip-Flop Structure

Each **TGATE_DFF** consists of:

- Two transmission gates
- One inverter for storage and regeneration

Figure 16a: TGATE-based D flip-flop schematic

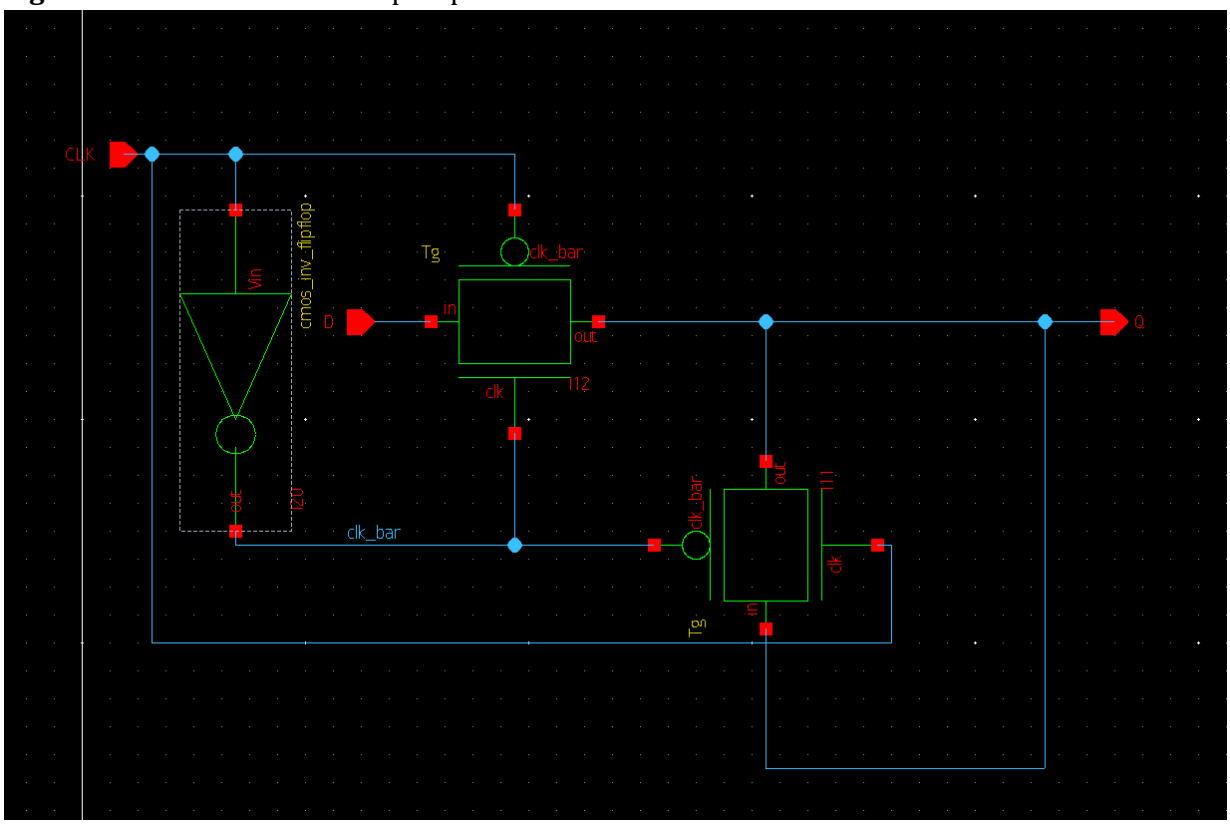
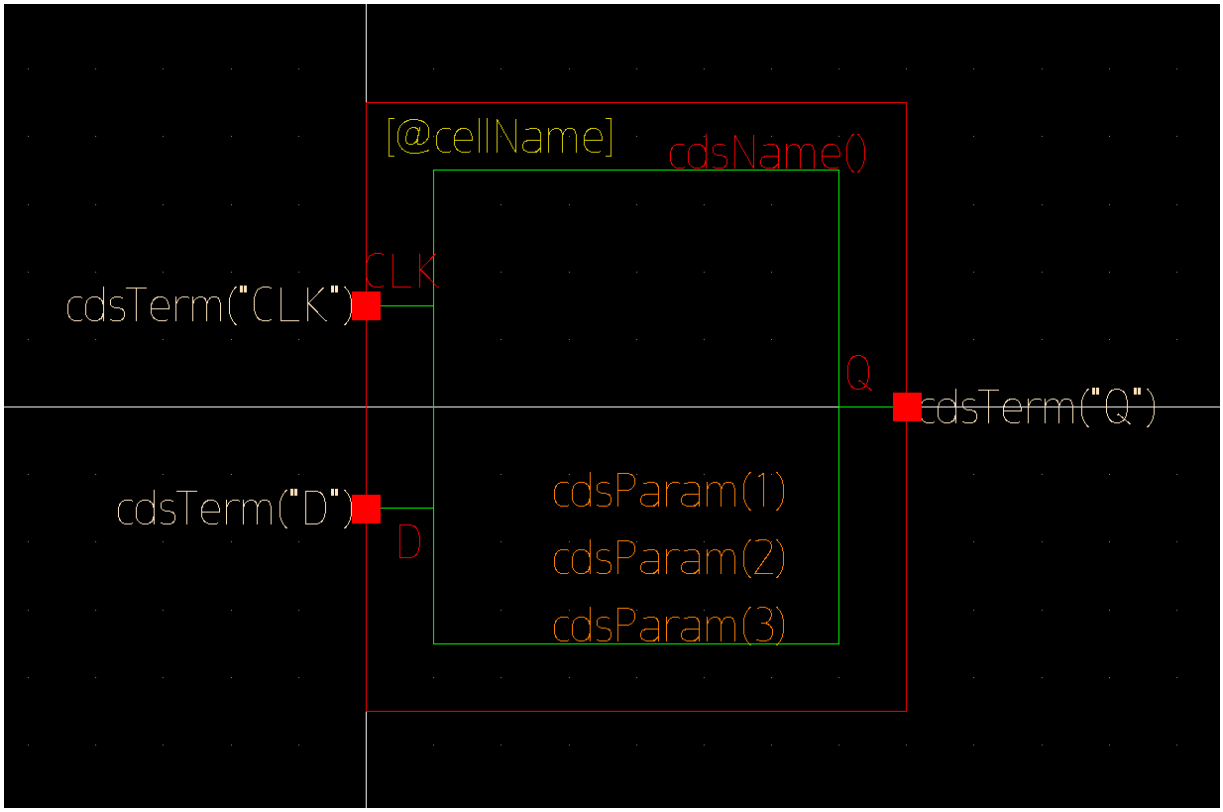


Figure 16b: TGATE-based D flip-flop symbol



Transmission-gate flip-flops were selected over static master–slave designs because they:

- Provide full rail-to-rail output swing
- Reduce static power consumption
- Exhibit robust timing behavior at low supply voltages

Each **16REG** block instantiates sixteen TGATE_DFF cells to store 16-bit intermediate results between pipeline stages [5].

3.6 Design Trade-Off Summary

At the circuit level, the design prioritizes:

- Correctness and robustness over minimal transistor count
- Modularity and reuse across hierarchical levels
- Predictable timing behavior suitable for pipelined operation

While more aggressive optimizations (e.g., carry-lookahead adders or dynamic logic) could reduce delay, the chosen architecture provides a balanced trade-off between performance, layout complexity, and design effort, making them well-suited for a full-custom educational VLSI project.

4. Layout Implementation and Physical Design Strategy

4.1 Layout Design Methodology

The physical layout of the 8×8 pipelined Vedic multiplier was implemented using a hierarchical full-custom layout methodology. Each circuit block was laid out and verified independently before being instantiated at higher levels of hierarchy. This approach reduces debugging complexity, ensures correctness at every level, and promotes reuse of verified layout cells.

The layout process followed a strict sequence:

1. Primitive cell layout (logic gates and flip-flops)
2. Mid-level block layout (adders and registers)
3. Top-level integration and routing
4. Physical verification using DRC and LVS

At every stage, functional equivalence between schematic and layout was maintained by preserving transistor sizing, connectivity, and pin naming consistency [2].

4.2 Primitive Cell Layout

Primitive cells such as INV, NAND2, XOR, NOR2, OR2, AND2, and TGATE were laid out first, as they form the foundation of all higher-level blocks.

Figure 17: Primitive inverter layout with labeled pins and power rails

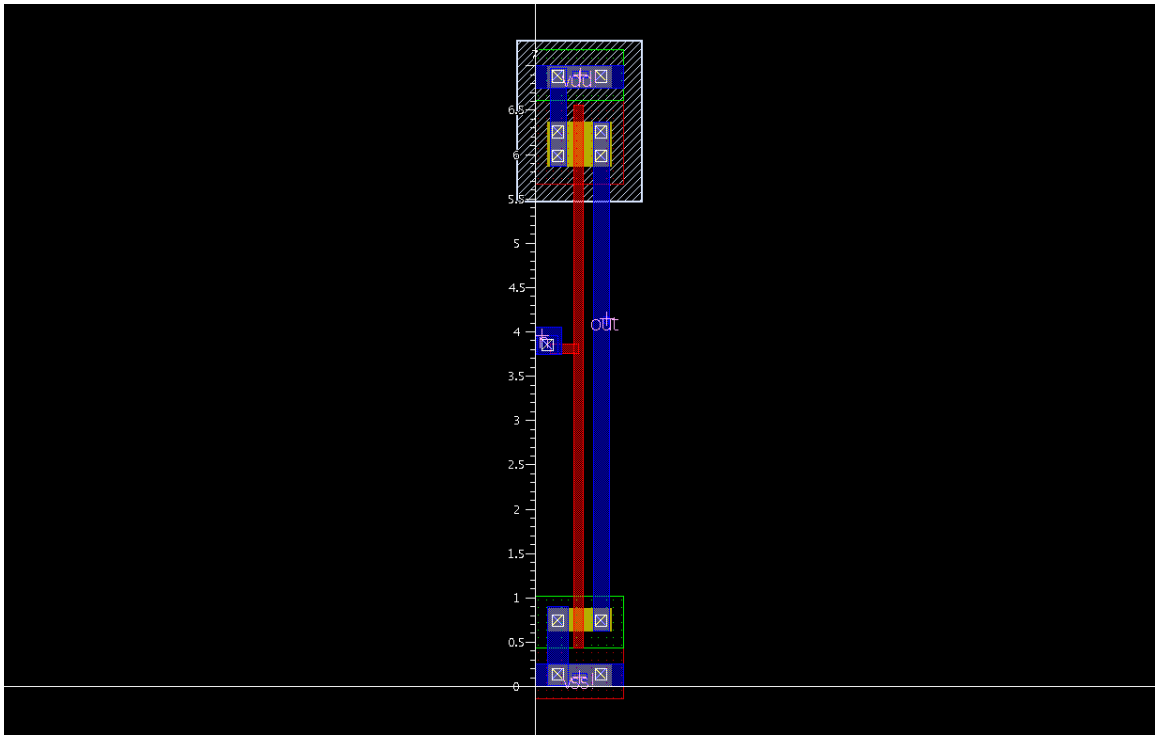


Figure 18: Primitive NAND layout with labeled pins and power rails

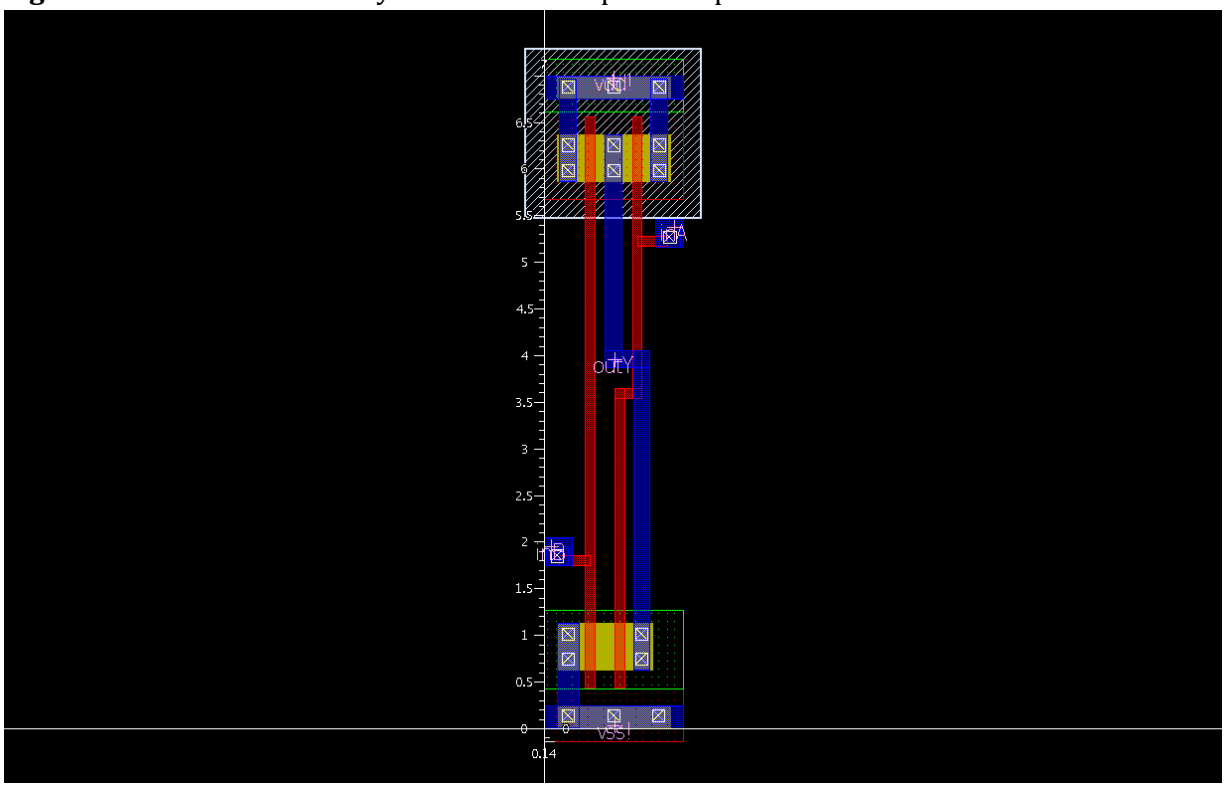


Figure 19: Primitive XOR layout with labeled pins and power rails

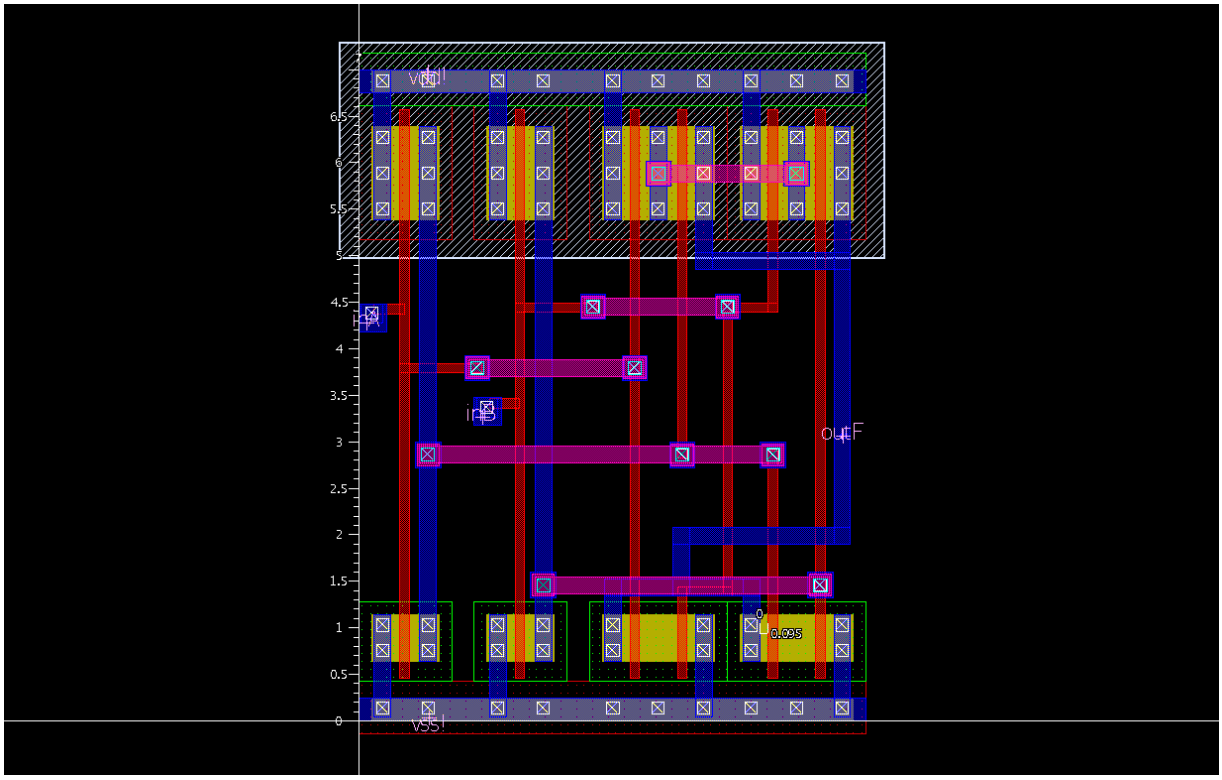


Figure 20: Primitive NOR layout with labeled pins and power rails

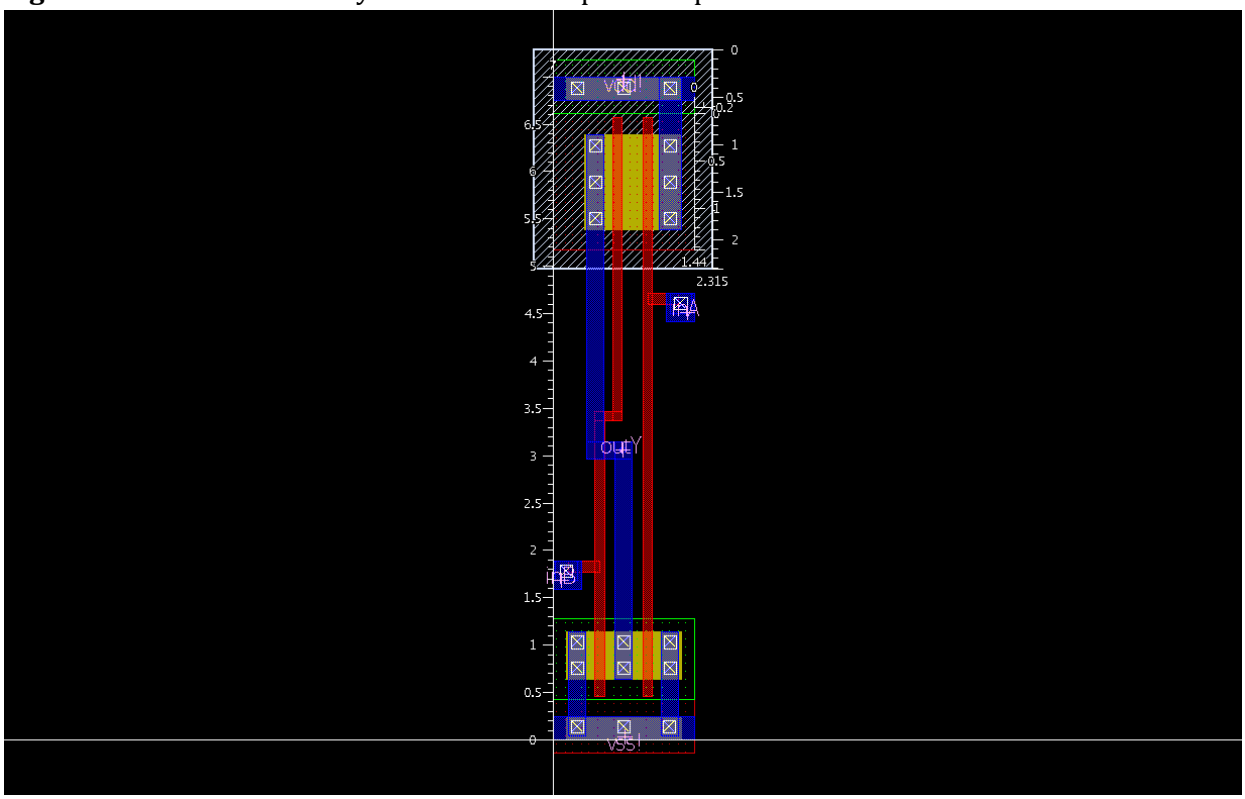


Figure 21: Primitive OR layout with labeled pins and power rails

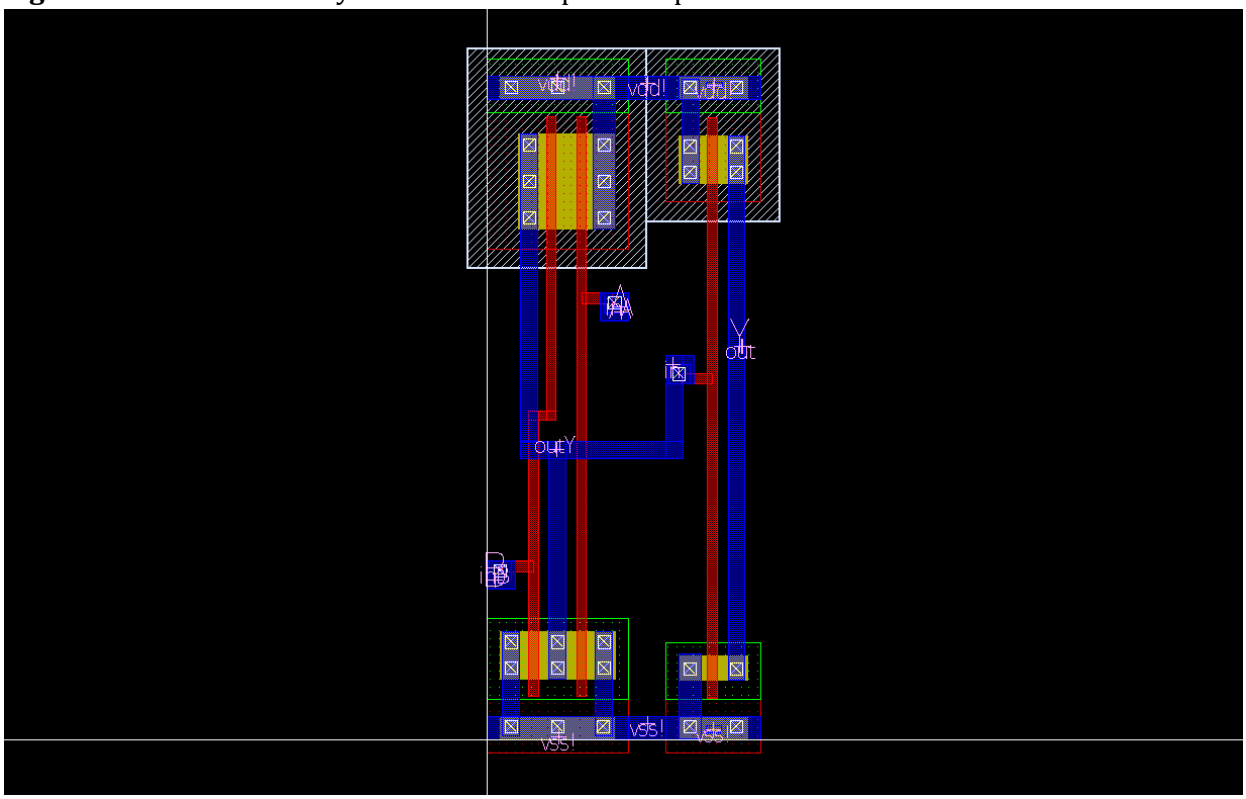


Figure 22: Primitive AND layout with labeled pins and power rails

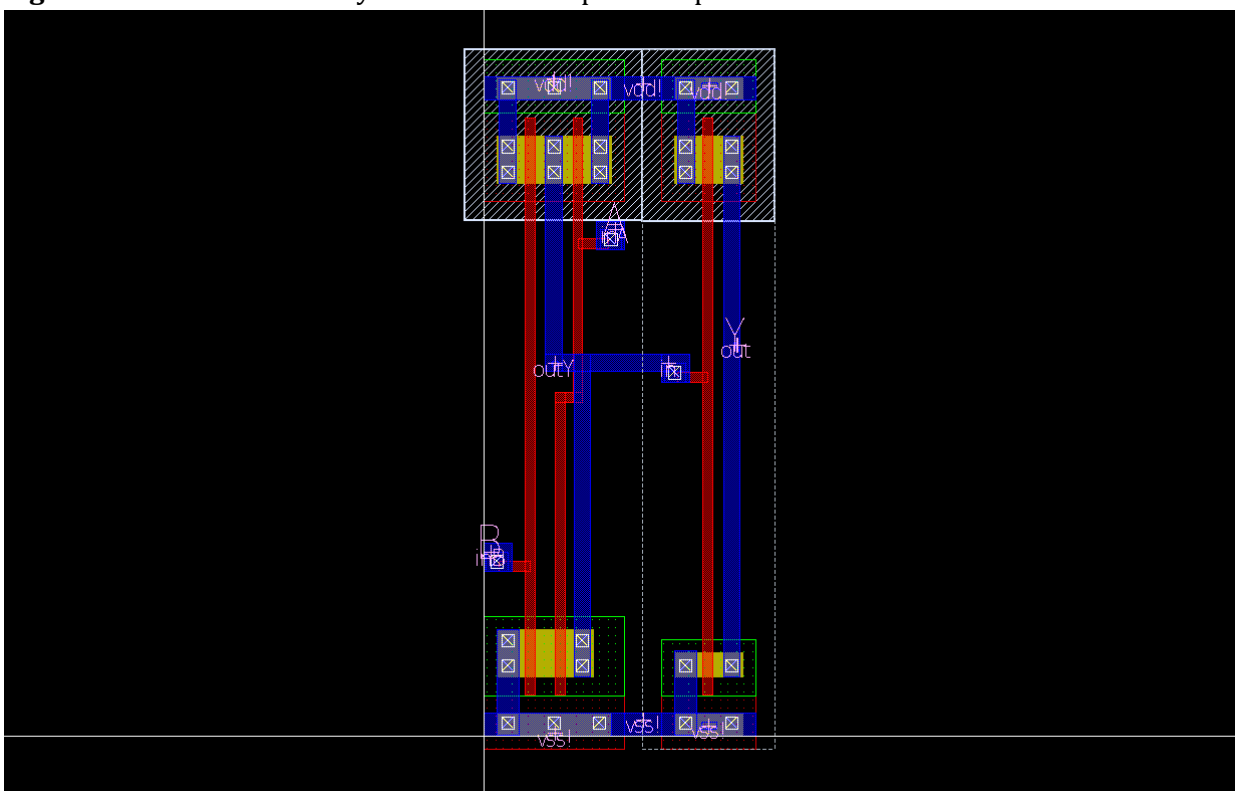
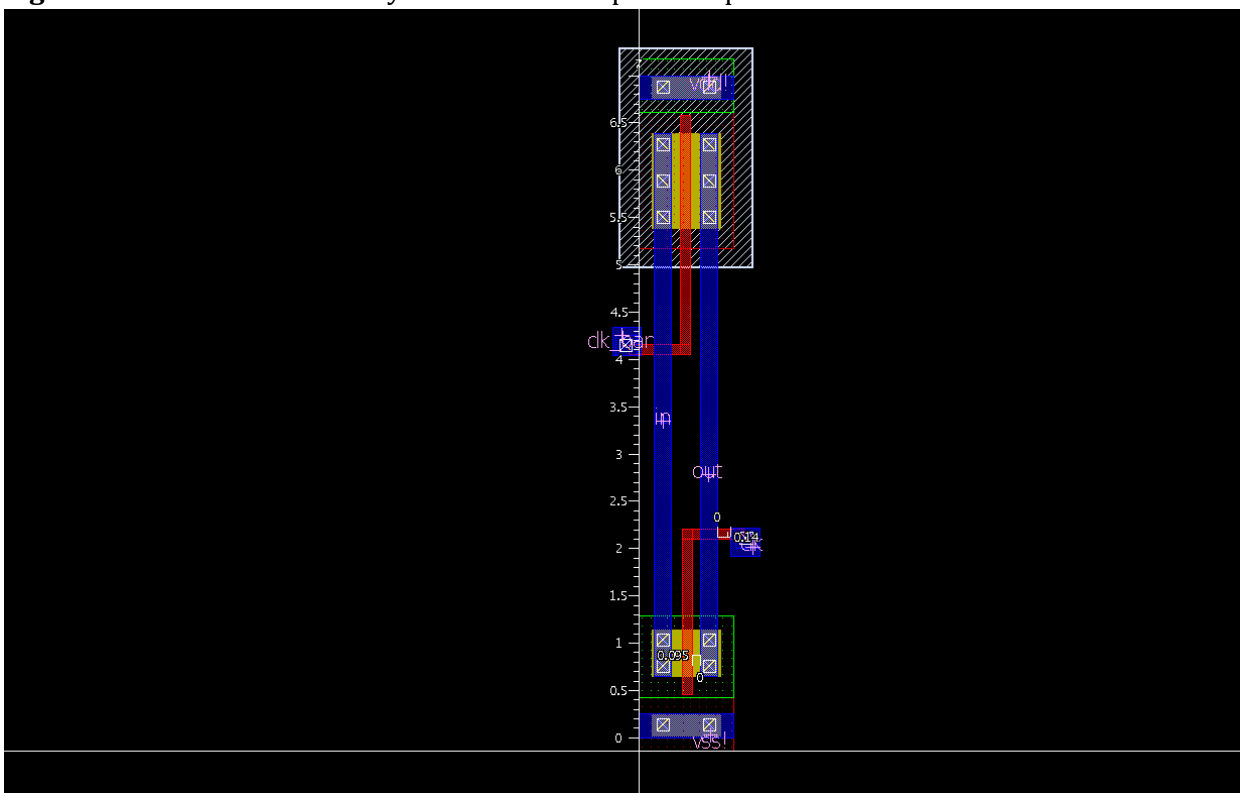


Figure 23: Primitive TGATE layout with labeled pins and power rails



Key layout principles applied to primitive cells include:

- Consistent cell height to facilitate horizontal abutment
- Aligned VDD and VSS rails to enable power continuity across adjacent cells
- Pins placed at cell boundaries to simplify inter-cell routing
- Compact diffusion sharing where possible to reduce area and parasitic capacitance

Metal 1 (M1) was primarily used for intra-cell routing, while contacts and vias were placed conservatively to ensure DRC compliance and reliable connectivity.

4.3 Mid-Level Block Layout

Mid-level blocks, including Half Adders (HA), Full Adders (FA) and Transmission-Gate D Flip-Flops (TGATE_DFF), were constructed by instantiating verified primitive cells and routing them within a bounded layout region.

Figure 24: Half adder layout showing internal routing

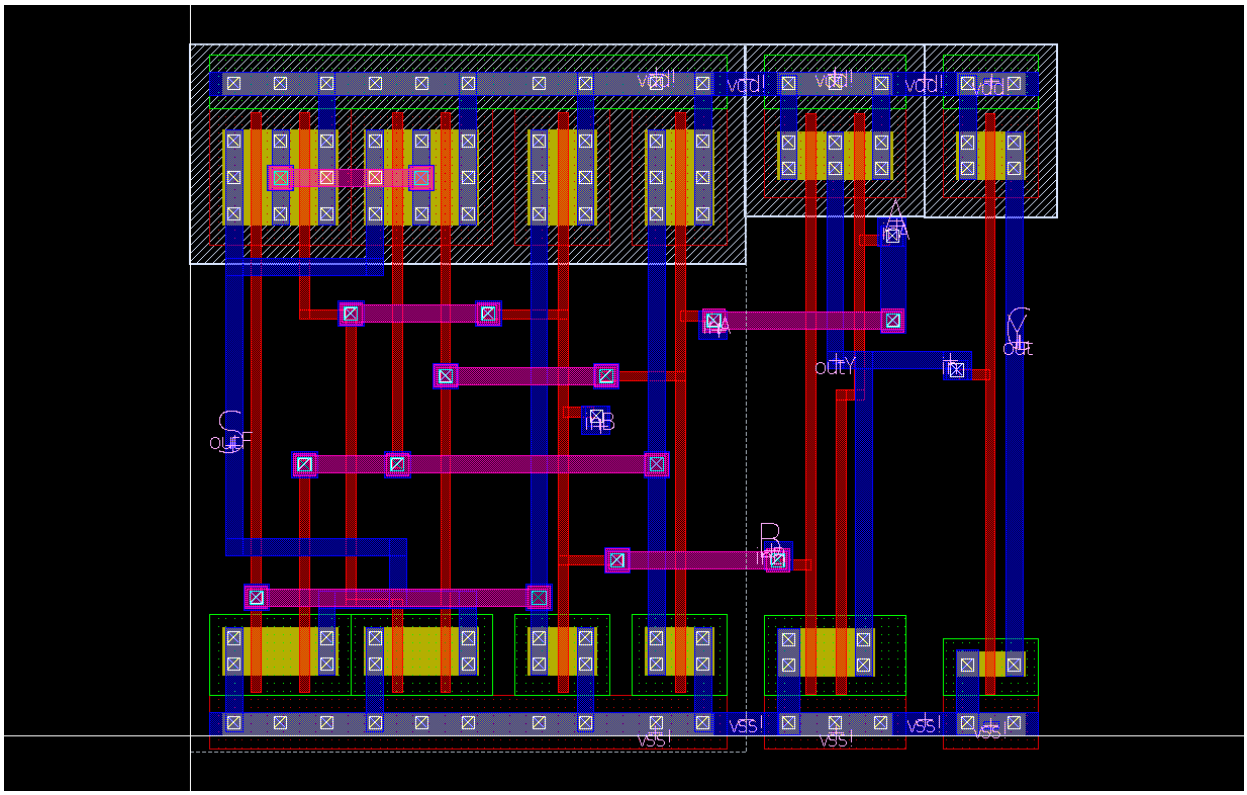


Figure 25: Full adder layout showing internal routing

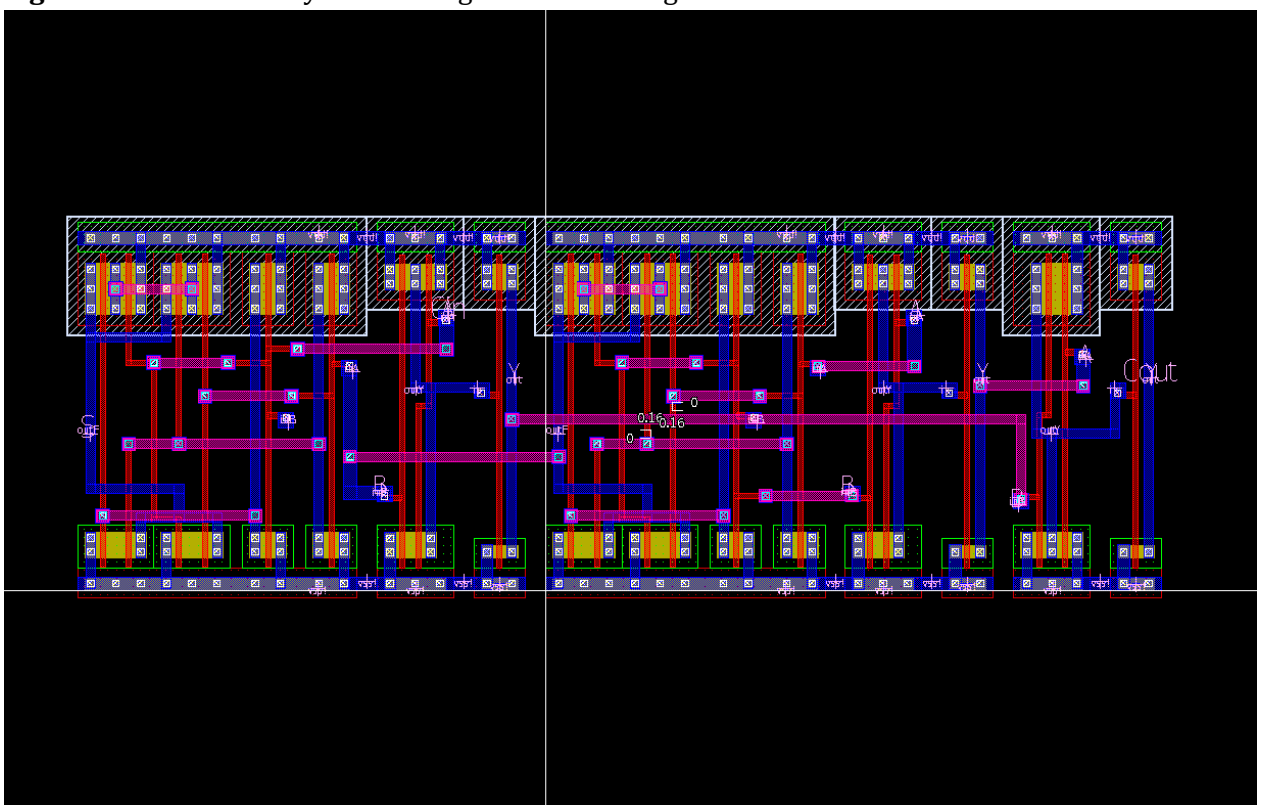
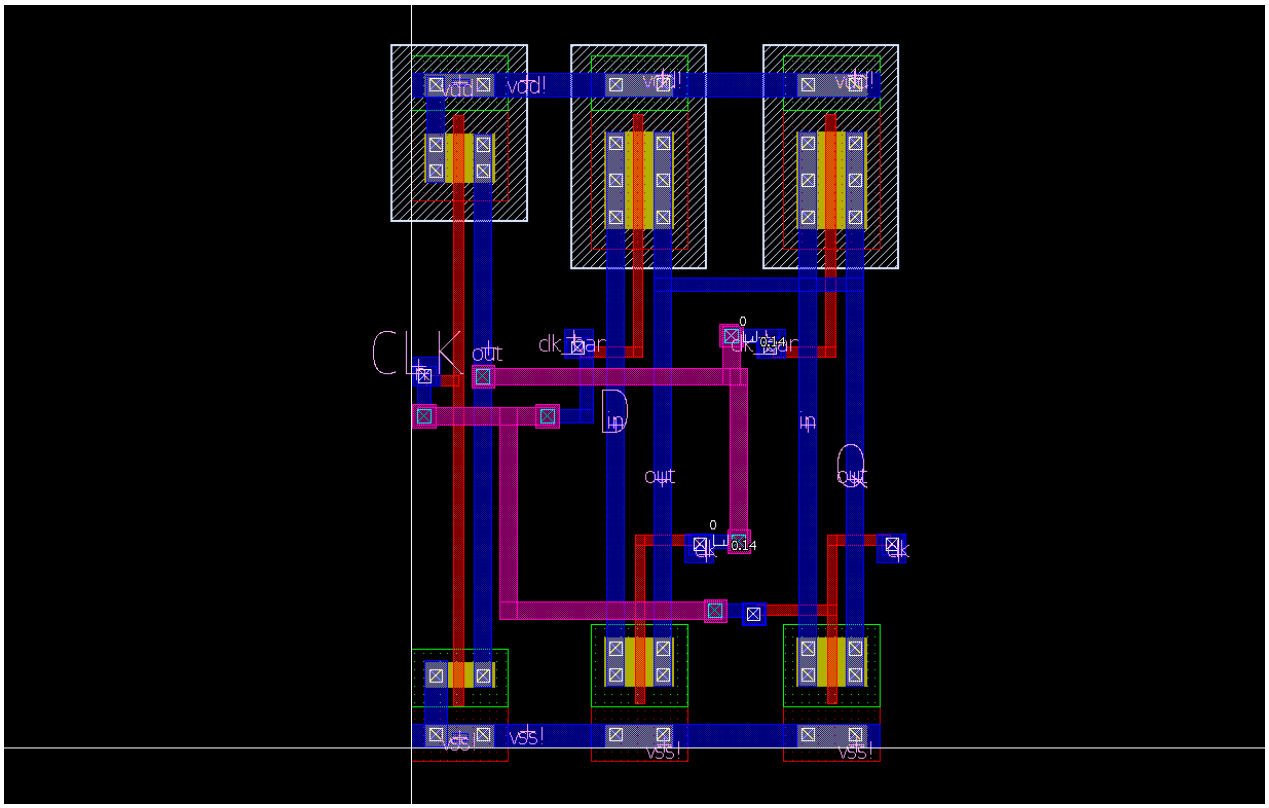


Figure 26: Transmission-Gate D Flip-Flop layout showing internal routing



At this level, routing complexity increased significantly due to the number of internal signals. To manage this complexity:

- Metal 2 (M2) was introduced for vertical signal routing
- Vias were carefully placed to avoid spacing and enclosure violations

Pin placement for these blocks was adjusted to improve accessibility when integrating them into larger arithmetic structures. Lessons learned at this stage directly informed improvements made in higher-level layouts.

4.4 Top-Level Layout Integration

The final layout of the **8×8 pipelined Vedic multiplier** integrates multiple mid-level blocks, including 4×4 Vedic multipliers, pipeline registers(16REG), 8 and 16-bit ripple carry adders.

Figure 27: 8RCA layout

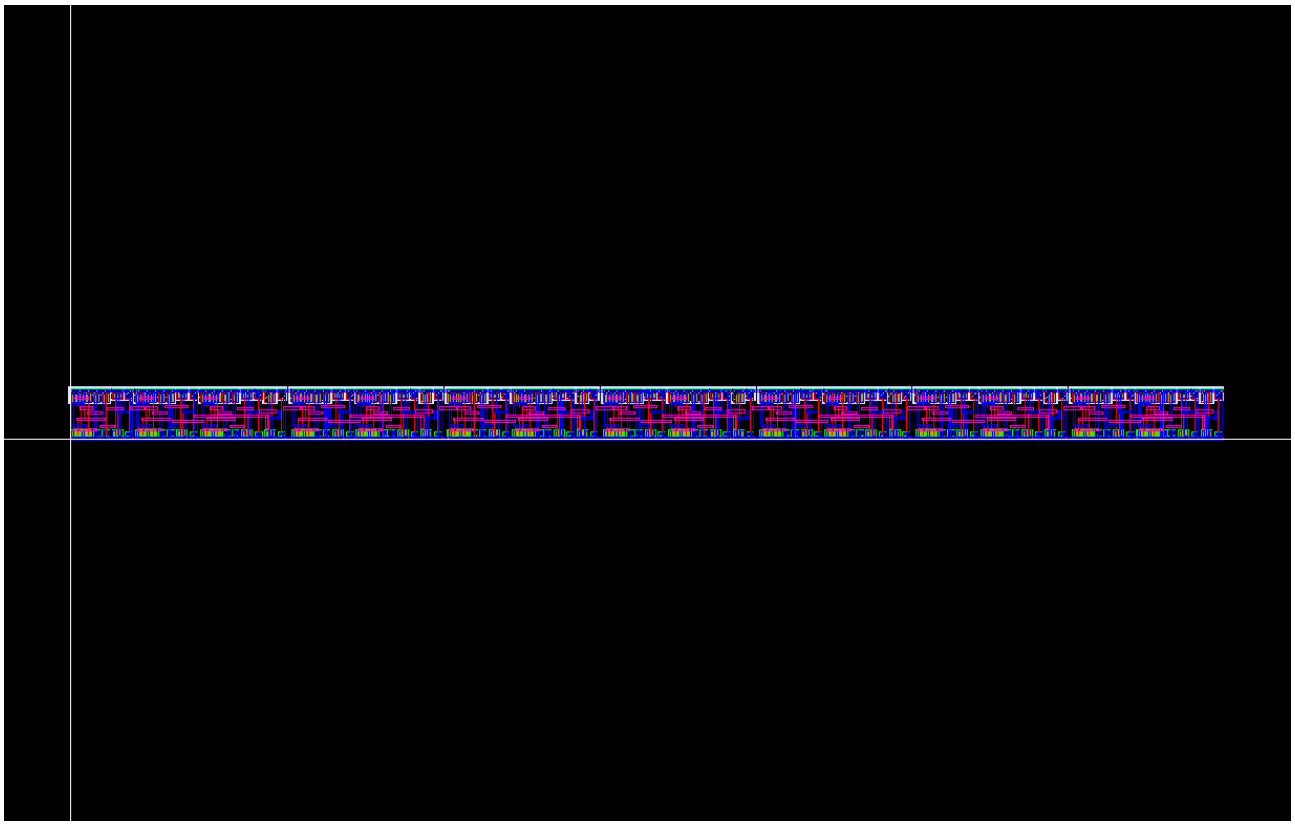


Figure 27b: 8RCA layout (left half)

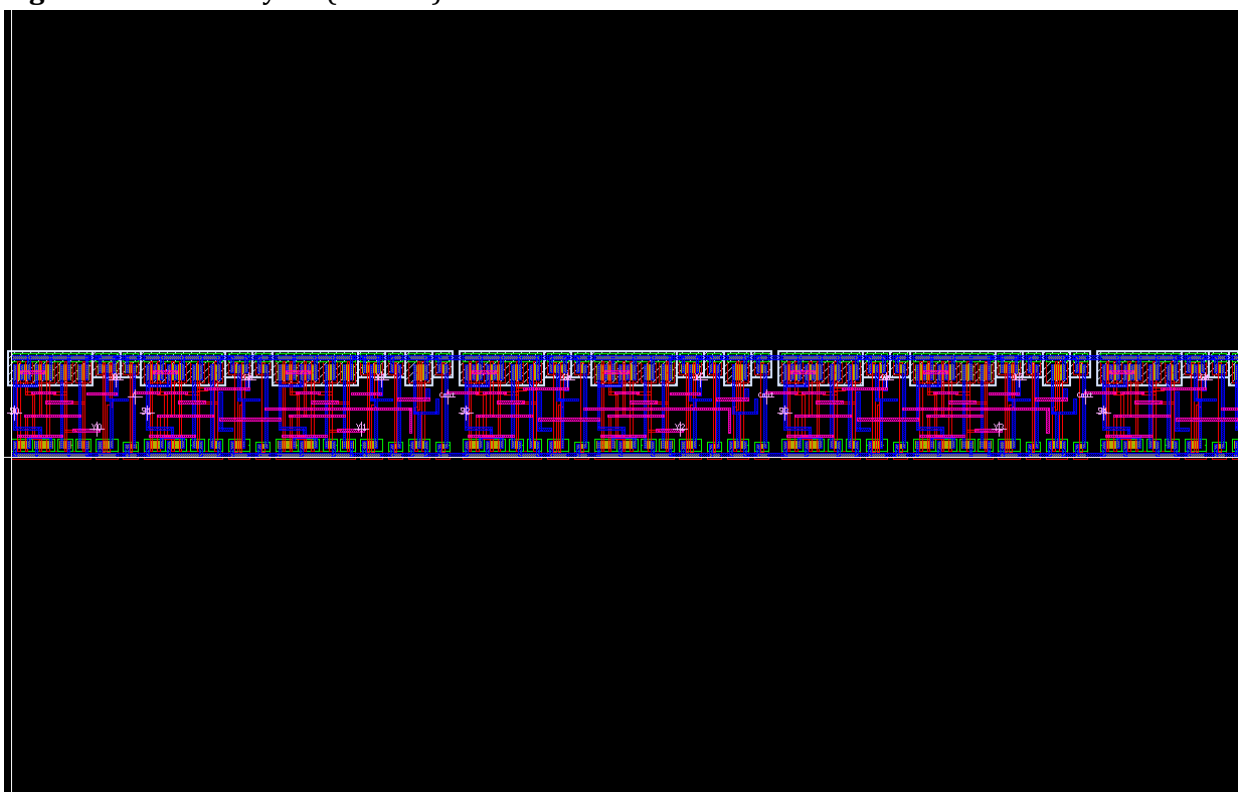


Figure 27c: 8RCA layout (right half)

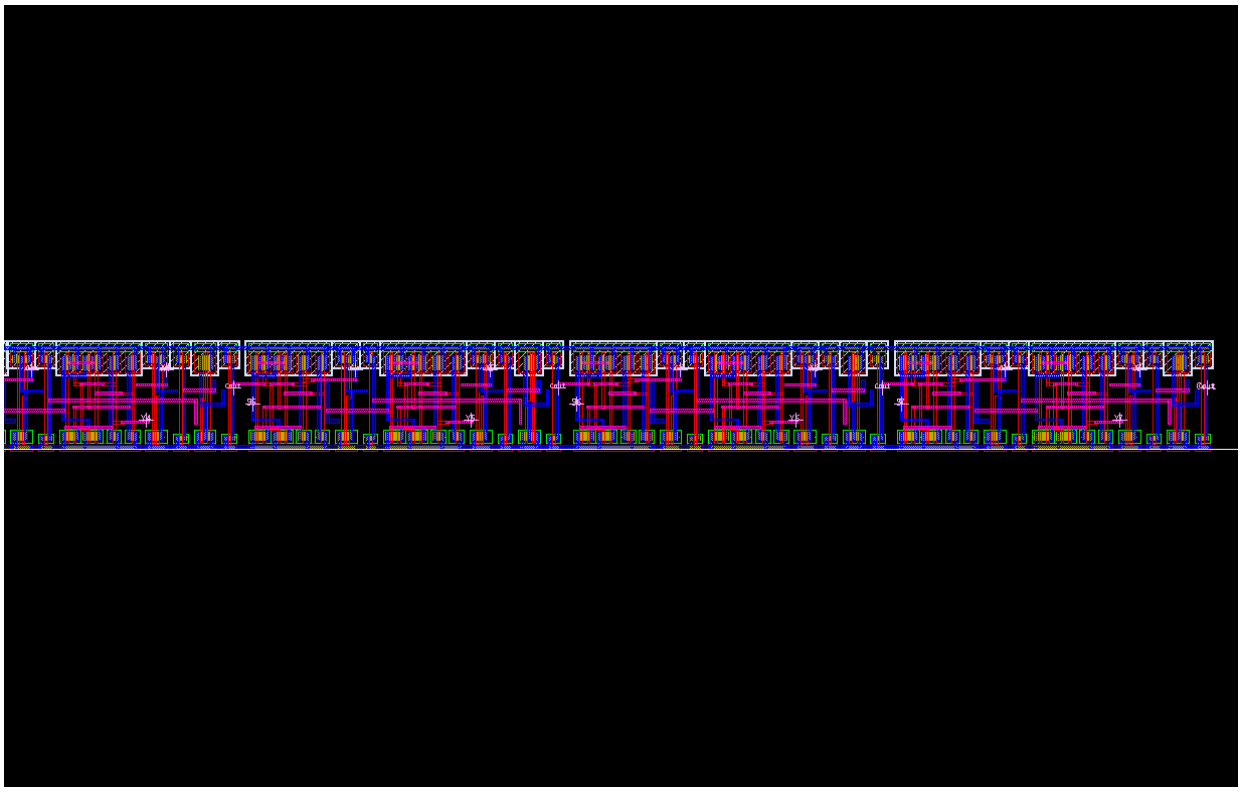


Figure 28a: 16RCA layout

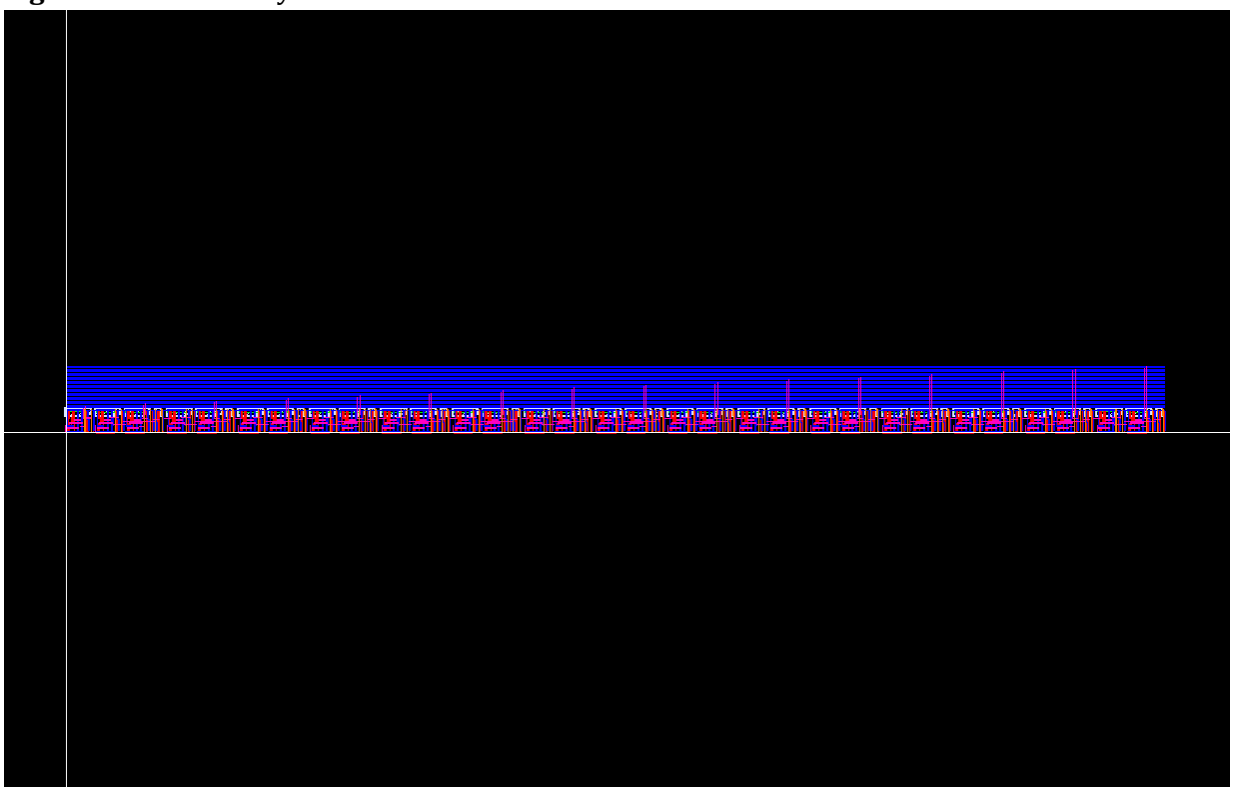


Figure 28b: 16RCA layout (left half)

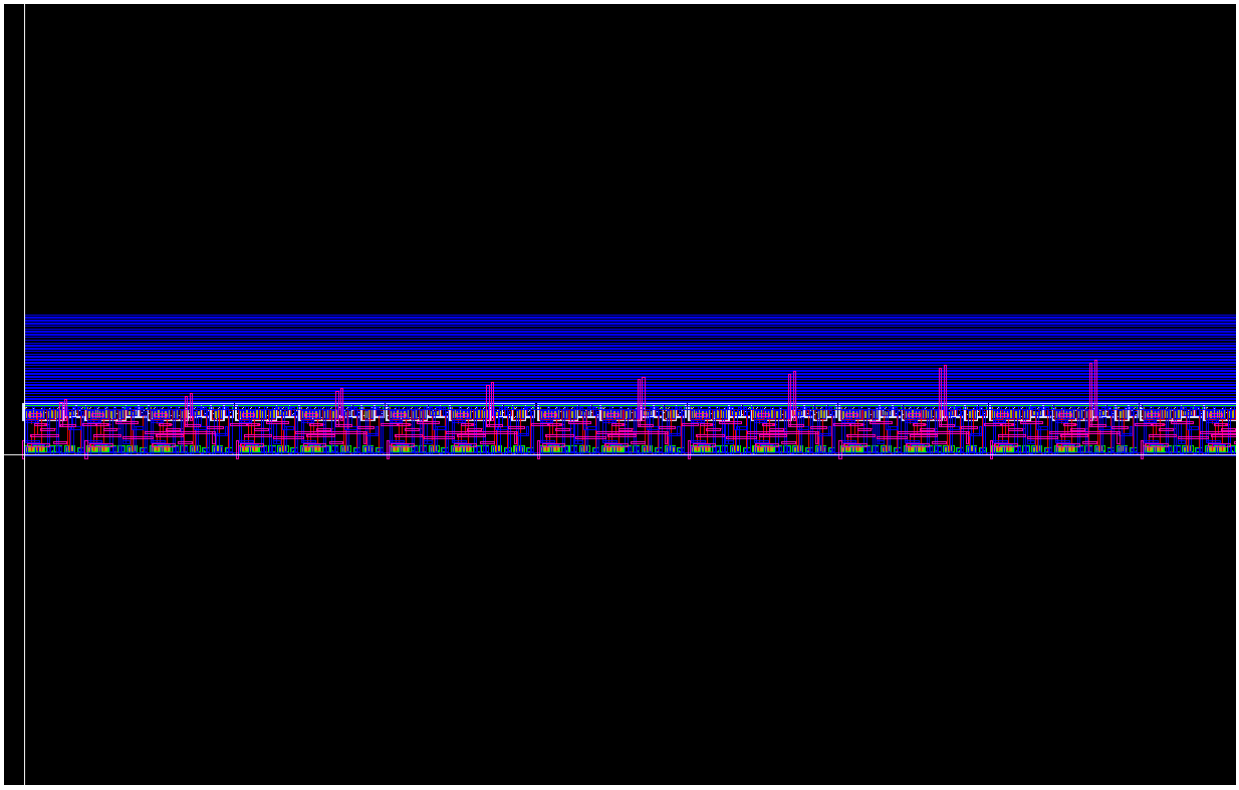


Figure 28c: 16RCA layout (right half)

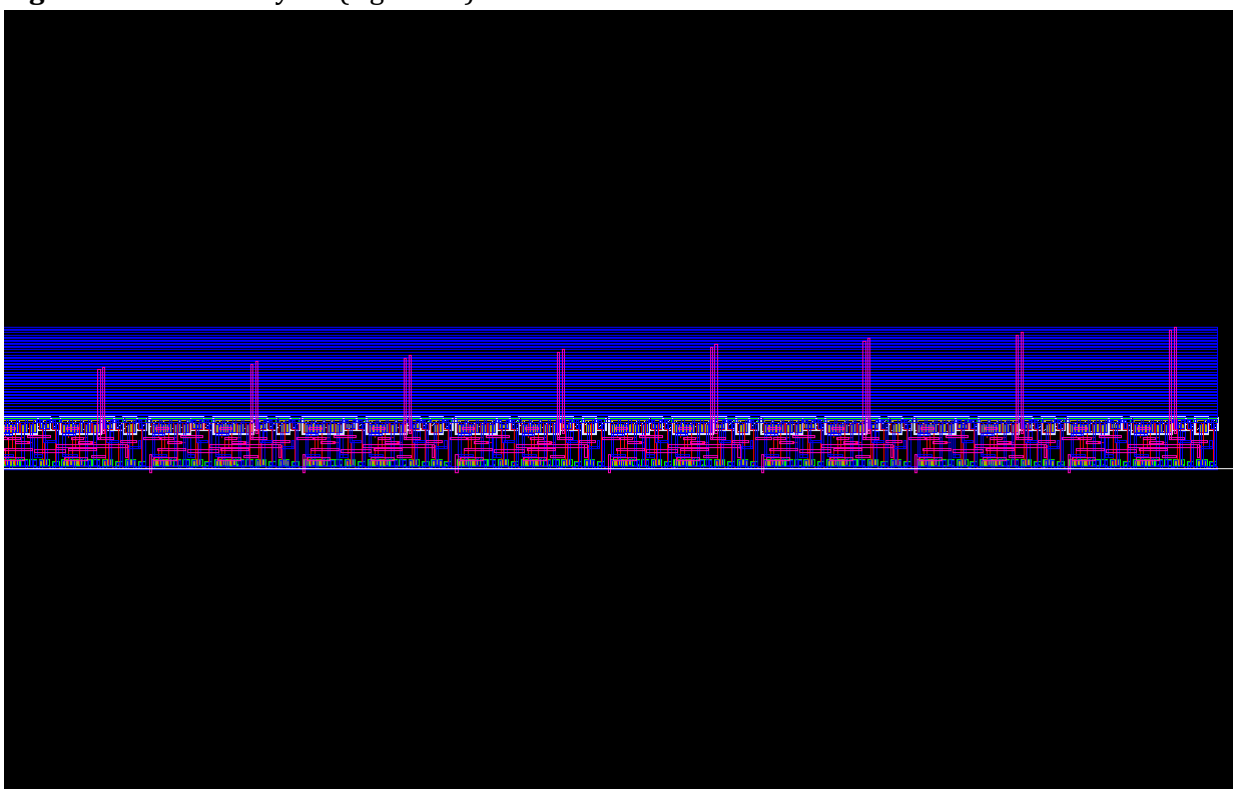


Figure 29a: 16REG layout

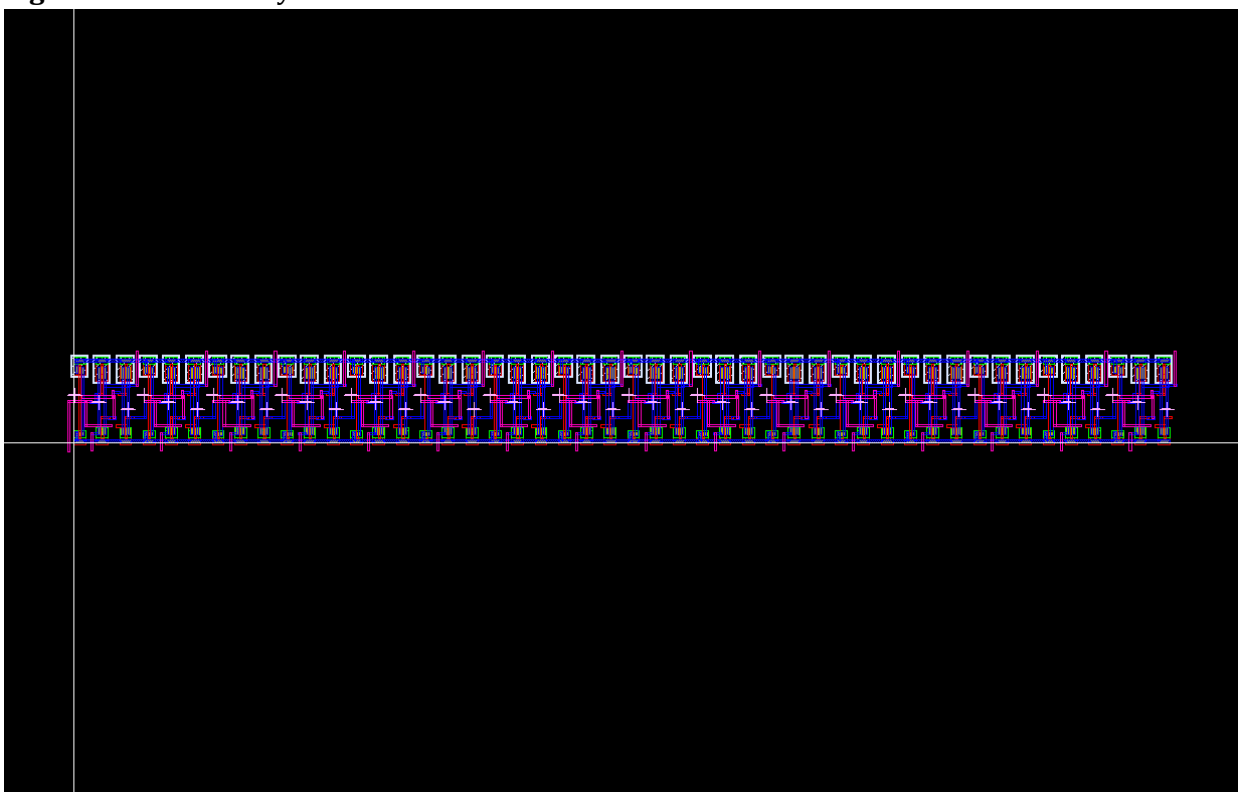


Figure 29b: 16REG layout (left half)

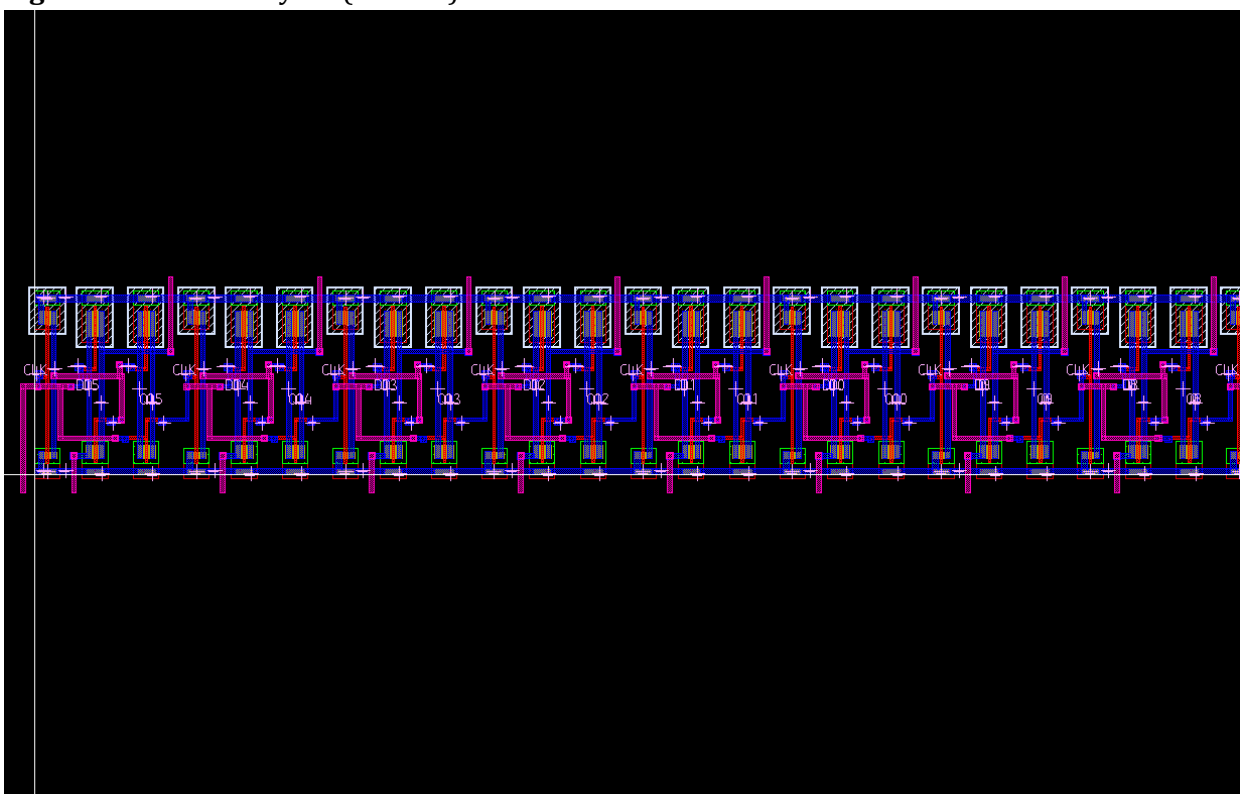


Figure 29c: 16REG layout (right half)

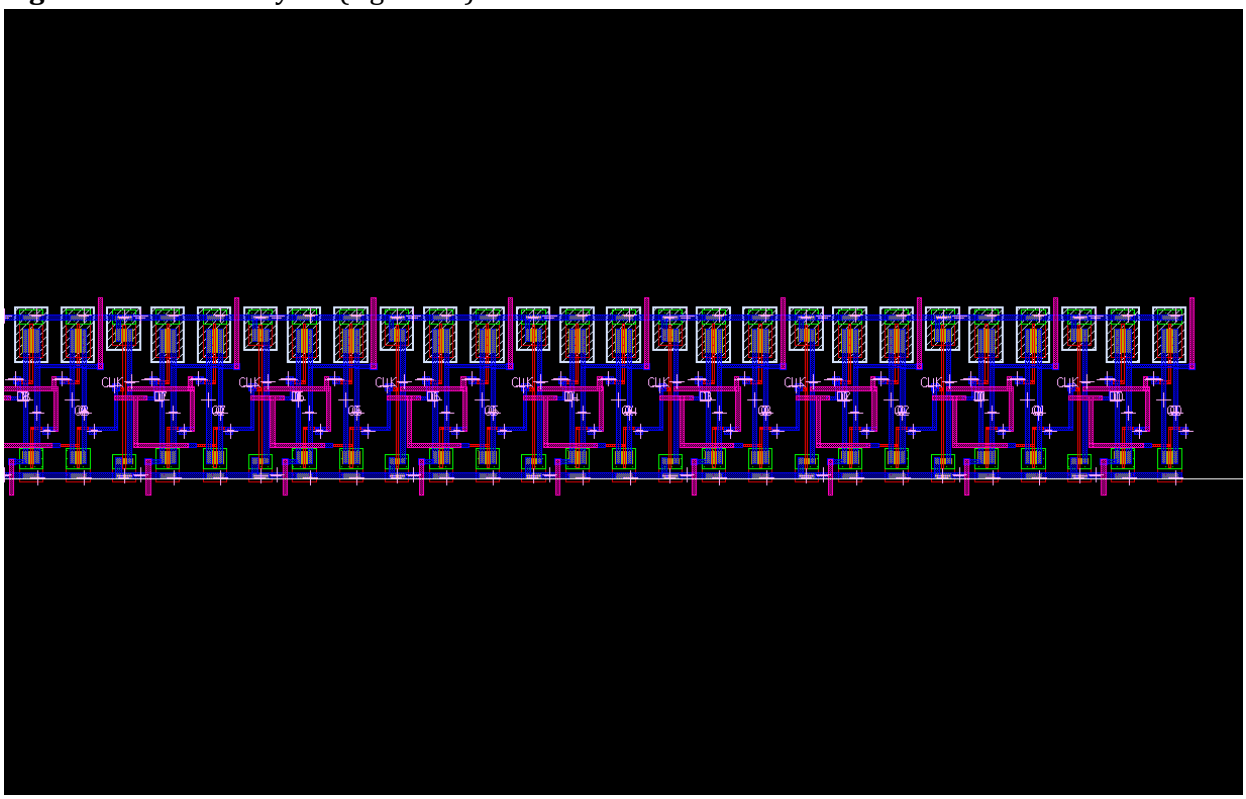


Figure 30: 2X2VEDIC layout

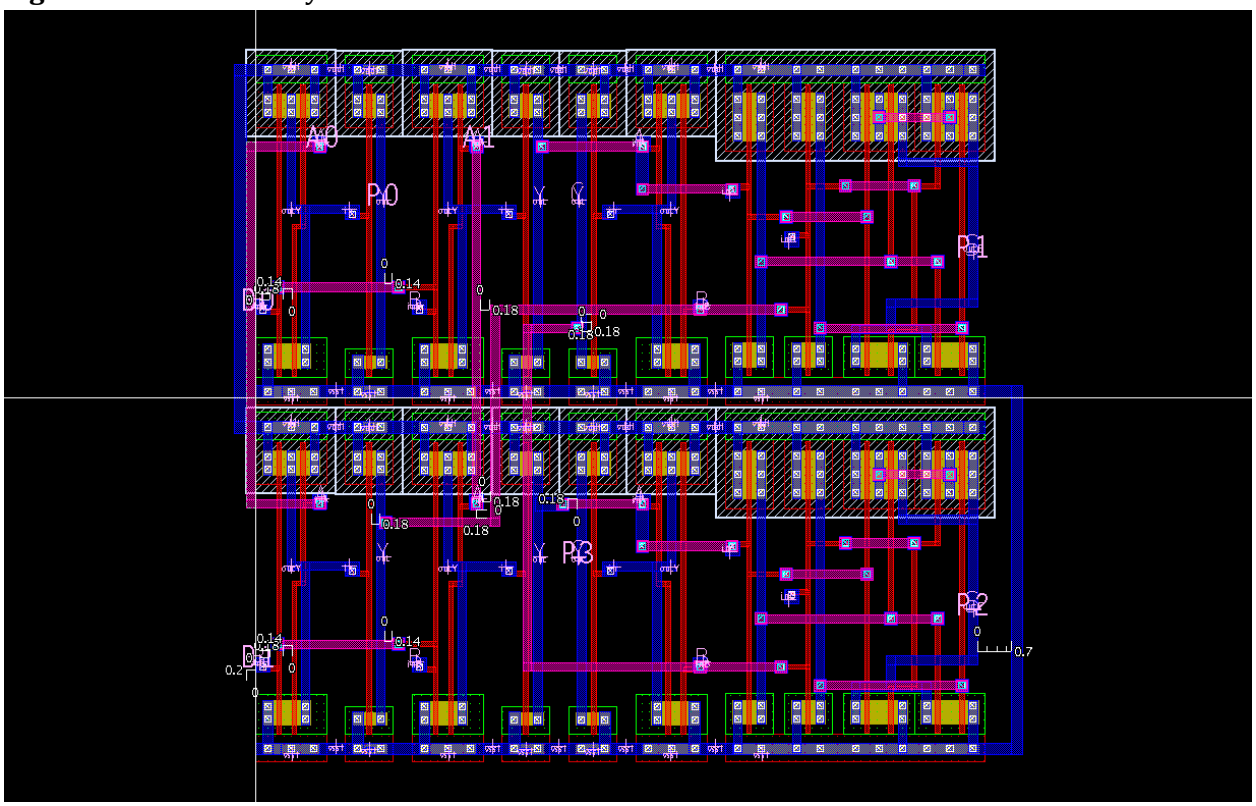


Figure 31a: 4X4VEDIC layout

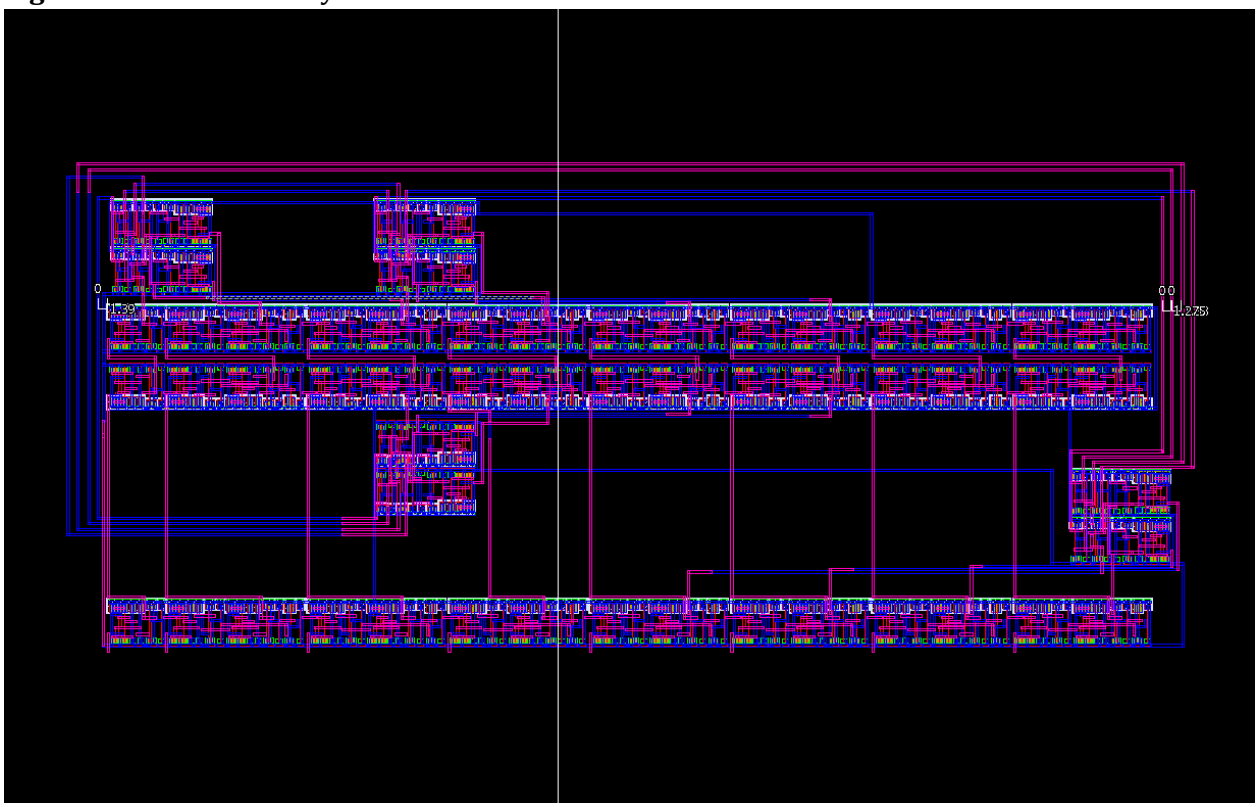


Figure 31b: 4X4VEDIC layout (left half)

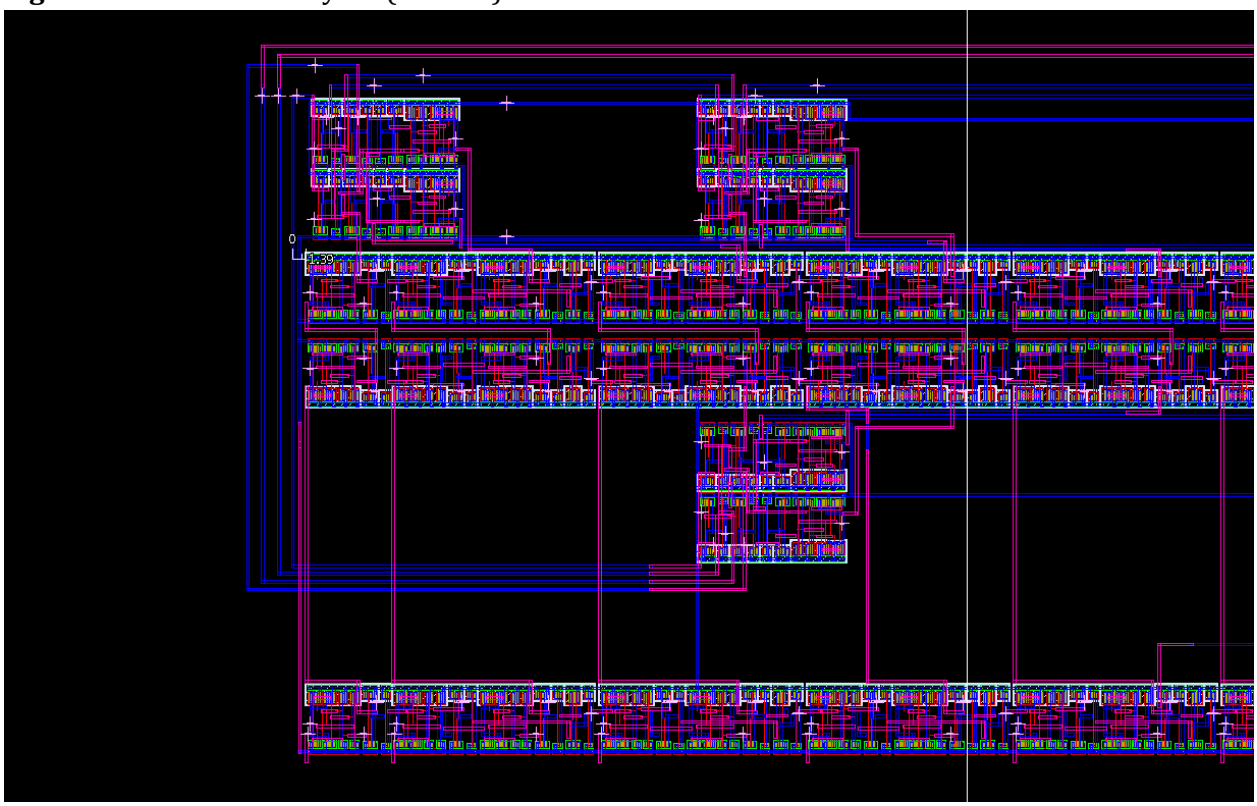


Figure 31c: 4X4VEDIC layout (right half)

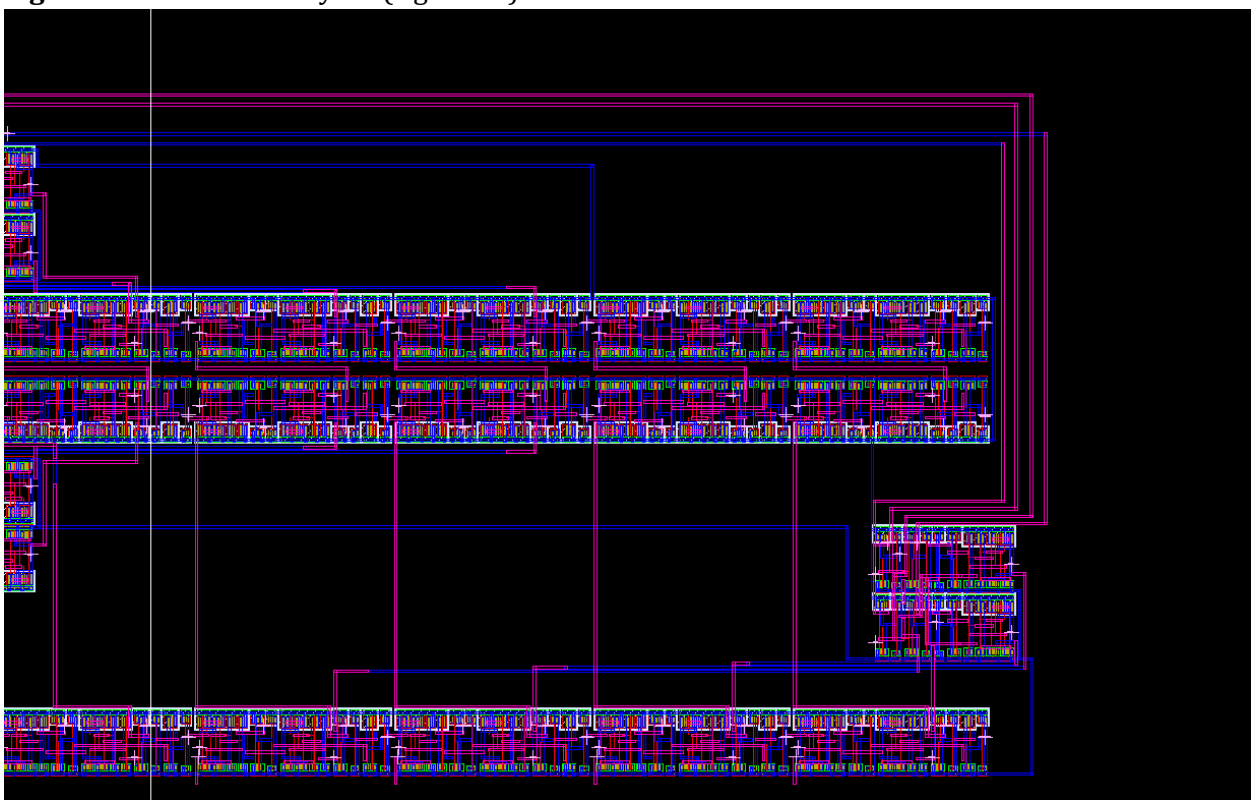


Figure 32a: 8X8VEDIC Pipelined layout

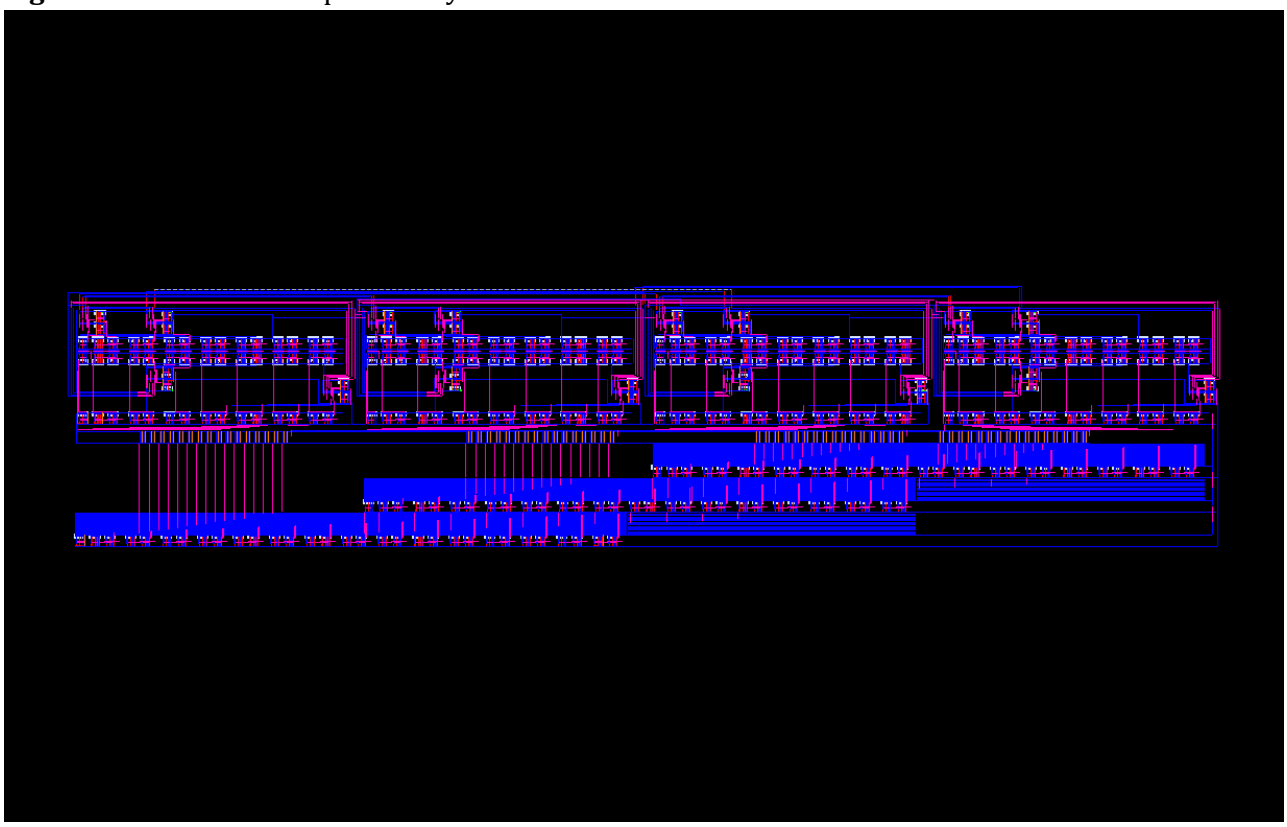


Figure 32b: 8X8VEDIC layout (left half)

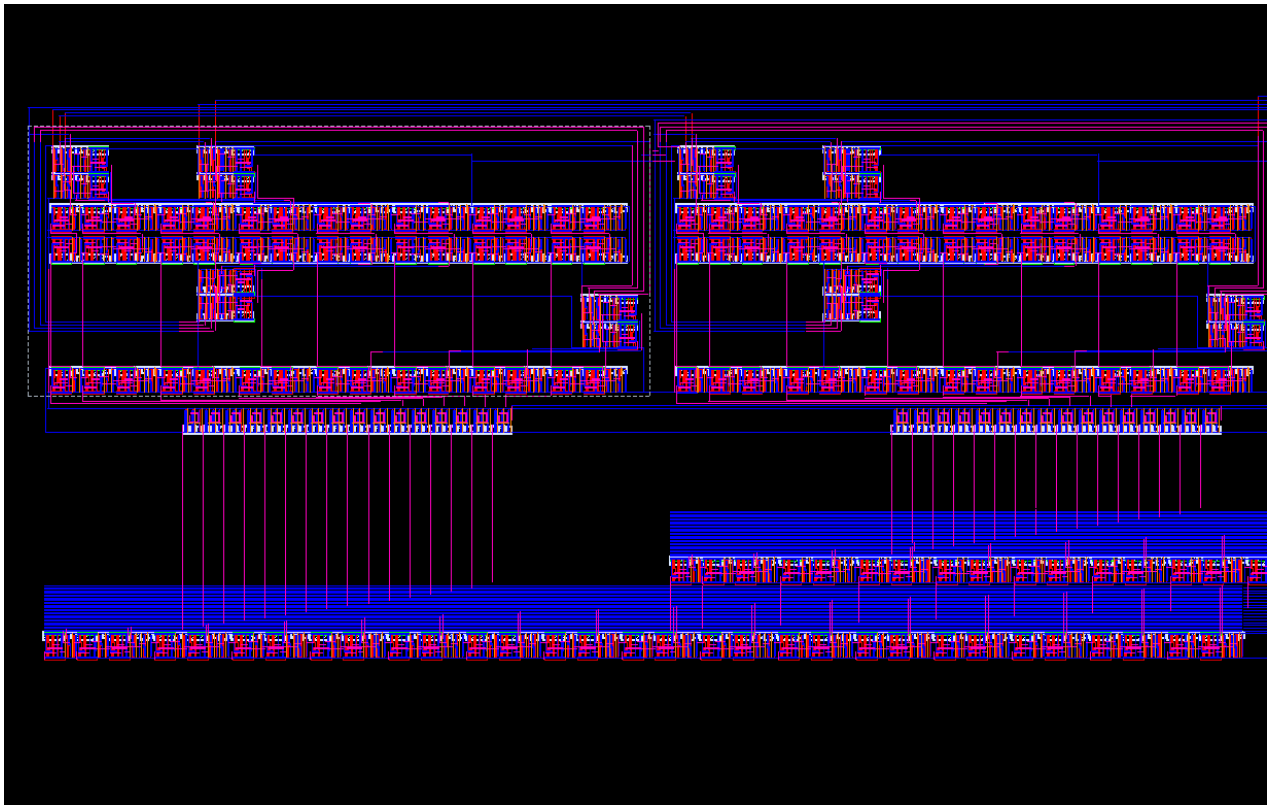
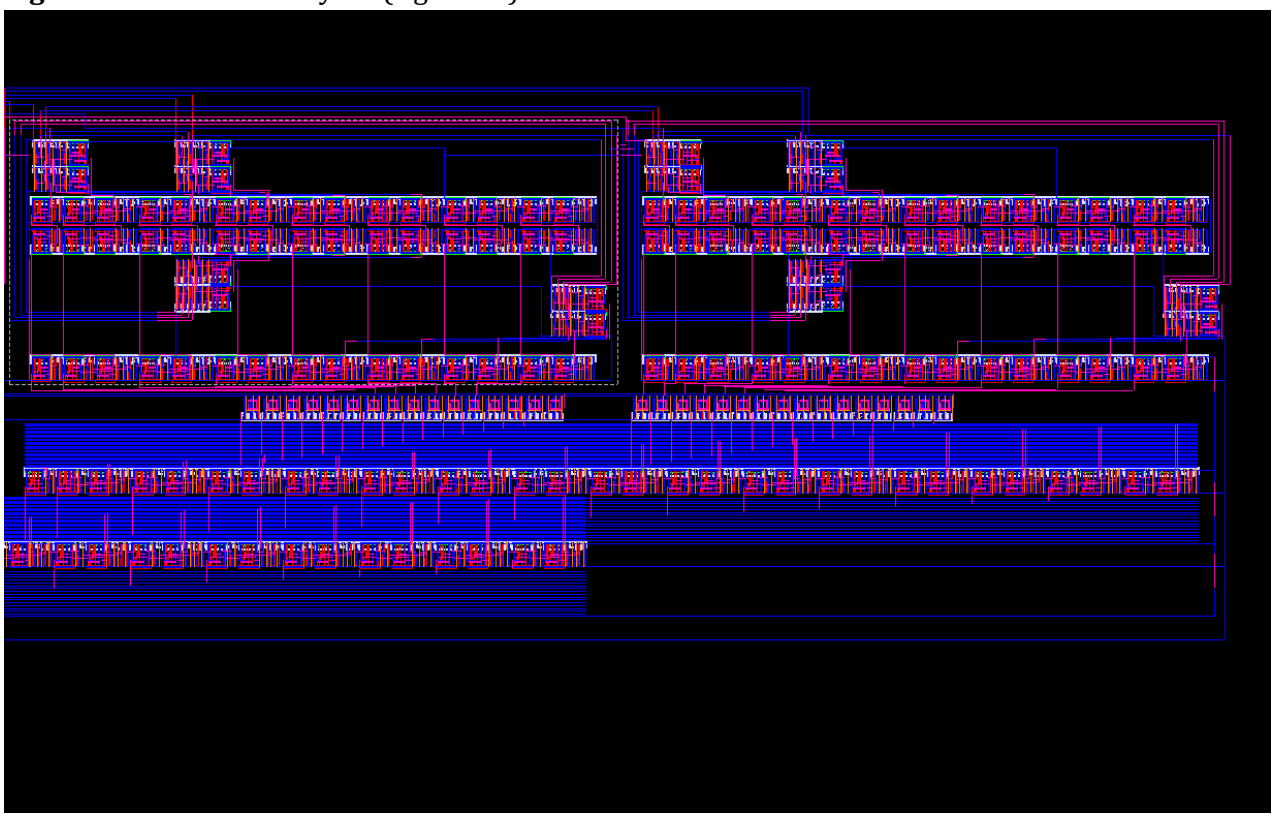


Figure 32c: 8X8VEDIC layout (right half)



Due to the large number of interconnections, routing at the top level was the most challenging aspect of the physical design. To address this:

- M1, M2, M3, and Poly were used for local routing and global signal routing and clock distribution
- Careful layer separation minimized unintended shorts and congestion

Pipeline registers were placed strategically between arithmetic blocks to reflect the logical dataflow and reduce long interconnect runs.

4.5 Pin Placement Strategy

Pin accessibility emerged as a critical factor in successful top-level integration. Early layout iterations revealed that poor pin placement in mid-level blocks significantly increased routing difficulty.

To address this:

- Pins were repositioned toward block edges
- Input and output pins were grouped logically based on dataflow direction
- Clock and control signals were routed using higher metal layers to reduce interference with data signals

These adjustments substantially simplified routing in the 8×8 pipelined multiplier and reduced the number of design iterations required to achieve a clean layout.

4.6 Metal Layer Utilization

Three metal layers were used throughout the design:

Metal 1, 2, and 3 were used for local and intra-cell routing, vertical inter-cell routing and signal crossing, global routing and congestion resolution as needed.

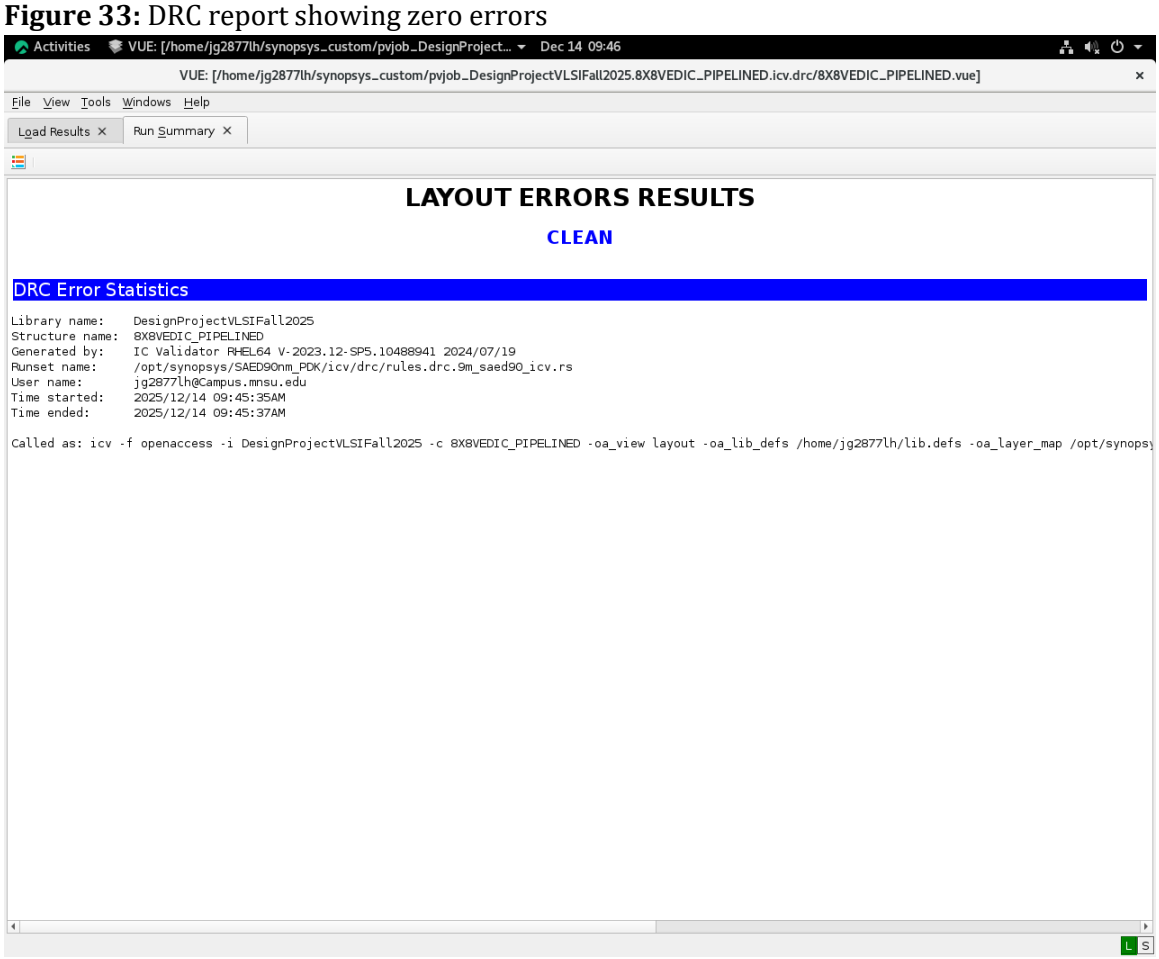
The use of multiple metal layers was essential to manage the high interconnect density inherent in a pipelined multiplier architecture. Higher metal layers were used sparingly and only where necessary to maintain layout clarity and DRC compliance [4].

4.7 Physical Verification

4.7.1 Design Rule Check (DRC)

DRC was run iteratively throughout the layout process rather than only at completion. This incremental verification strategy allowed early detection of spacing, enclosure, and width violations.

Figure 33: DRC report showing zero errors

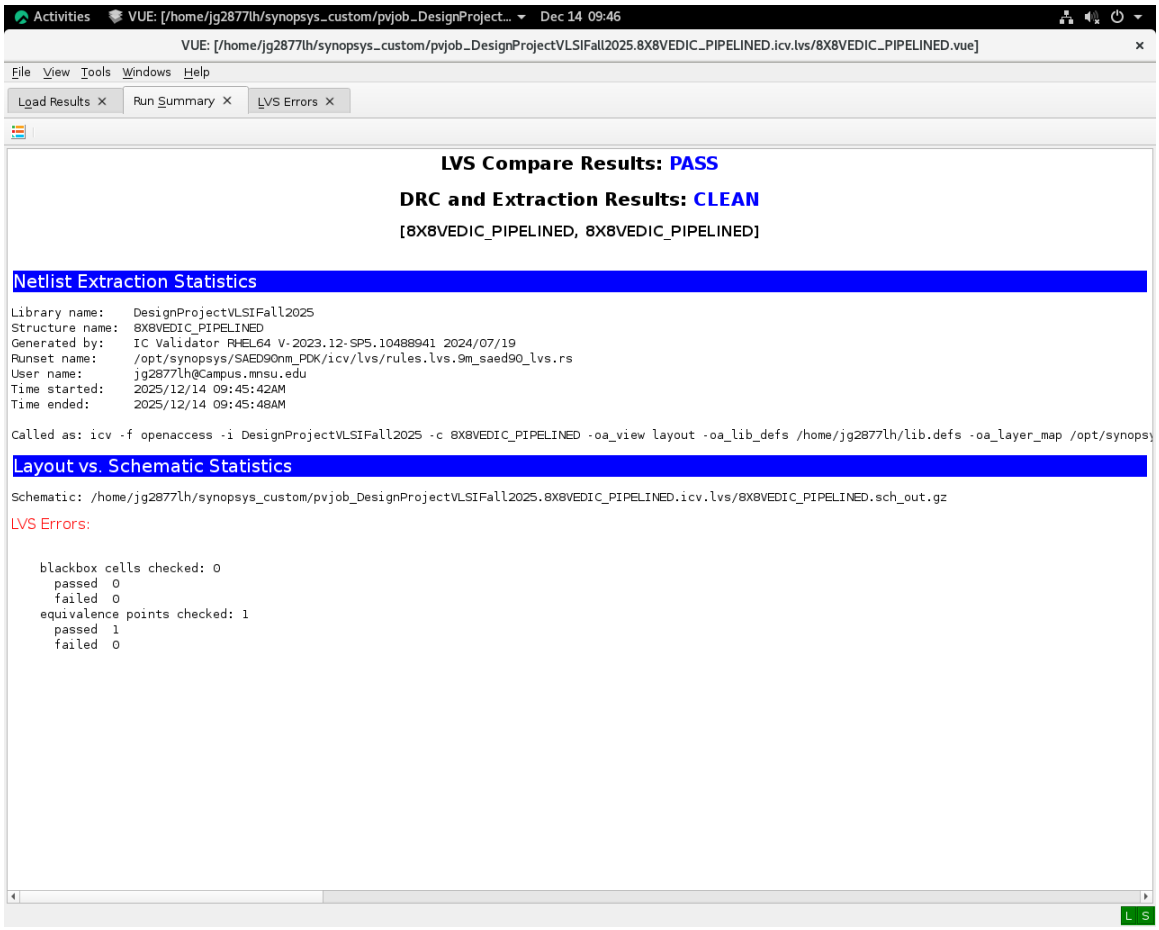


All DRC violations encountered during development were resolved, and the final layout passed DRC with zero errors.

4.7.2 Layout Versus Schematic (LVS)

LVS verification was performed on all major blocks and the final integrated design to ensure that the layout accurately reflected the schematic connectivity and device counts.

Figure 34: LVS report confirming schematic-layout match



Successful LVS matching confirms that the physical implementation preserves the intended circuit behavior.

4.7.3 Parasitic Extraction

Layout Parasitic Extraction (LPE) was not performed for this project. While parasitic effects can significantly impact timing and power in deep-submicron designs, the focus of this work was on functional correctness, architectural demonstration, and disciplined layout practices. The design flow, however, supports straightforward integration of LPE and post-layout simulation in future work.

4.8 Layout Challenges and Design Improvements

The most significant layout challenge encountered was routing congestion, particularly at higher levels of hierarchy where numerous signals overlap. This issue was mitigated through:

- Improved pin placement
- Strategic use of higher metal layers
- Iterative refinement of block placement

An important improvement made during the project was redesigning pin accessibility in the 4×4 Vedic multiplier, which greatly simplified its integration into the 8×8 pipelined architecture.

5. Simulation, Verification, and Testing Methodology

5.1 Verification Strategy Overview

Verification of the 8×8 pipelined Vedic multiplier followed a **progressive, bottom-up testing methodology**, consistent with disciplined VLSI design practice. Each block was verified independently before being integrated into higher levels of hierarchy. This approach minimized debugging complexity and ensured that functional correctness was preserved throughout the design flow.

The verification process consisted of:

1. Pre-layout schematic simulation of primitive cells
2. Functional simulation of mid-level arithmetic blocks
3. System-level simulation of the complete pipelined multiplier
4. Physical verification using DRC and LVS

Post-layout simulation with parasitics was not performed; however, the design flow and testing methodology support straightforward integration of post-layout analysis in future work [3].

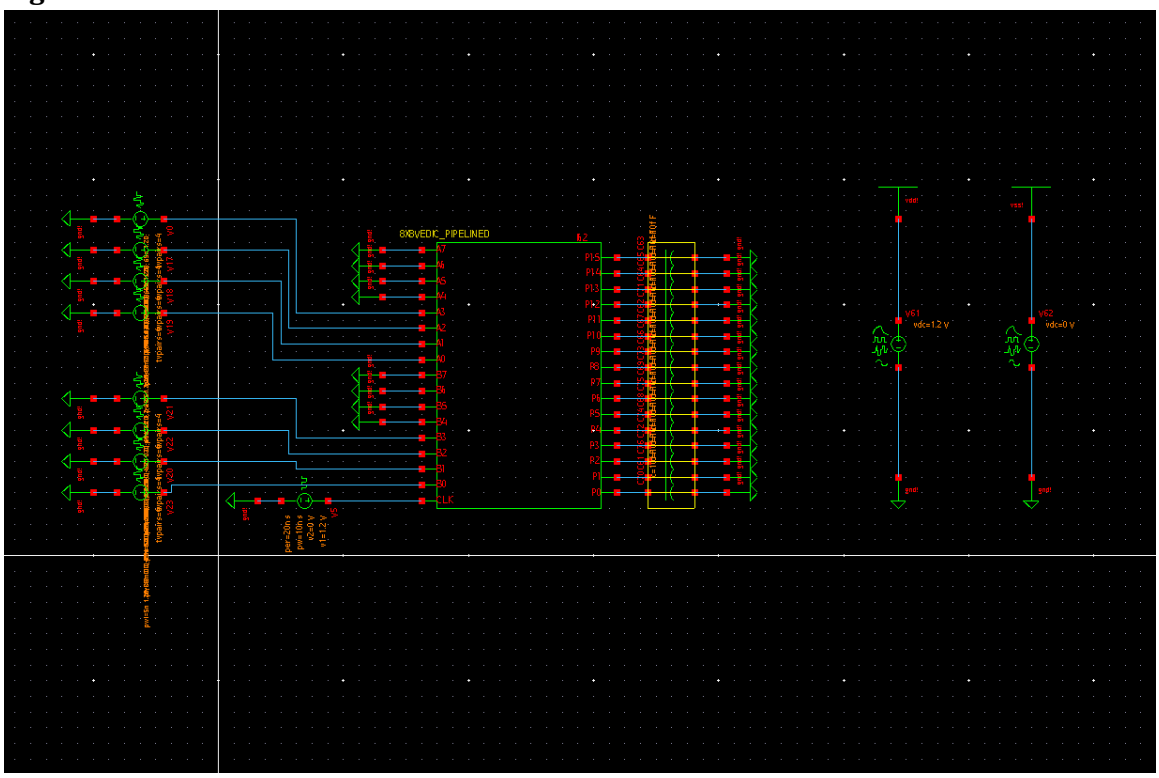
5.2 Simulation Environment and Setup

All schematic-level simulations were performed using a **90 nm CMOS process** at a **1.2 V supply voltage** in a testbench, with Prime Wave with HSPICE. This provides a representative operating condition for functional verification and timing observation.

Inside the TB:

- I instantiated VEDIC8x8_PIPELINED
- Added:
    - VPULSE for CLK
    - 8 VPWL sources and GND for A7 to A0 and B7 to B0
- Outputs: probe P15 to P0 which were connected to 10fF capacitors

Figure 35: 8X8VEDIC PIPELINED TB schematic



5.2.1 Clock Configuration

The pipelined multiplier was driven by a global clock signal with the following parameters:

- High voltage (V1): 1.2 V
- Low voltage (V2): 0 V
- Rise time: 0.1 ns
- Fall time: 0.1 ns
- Pulse width: 10 ns
- Period: 20 ns

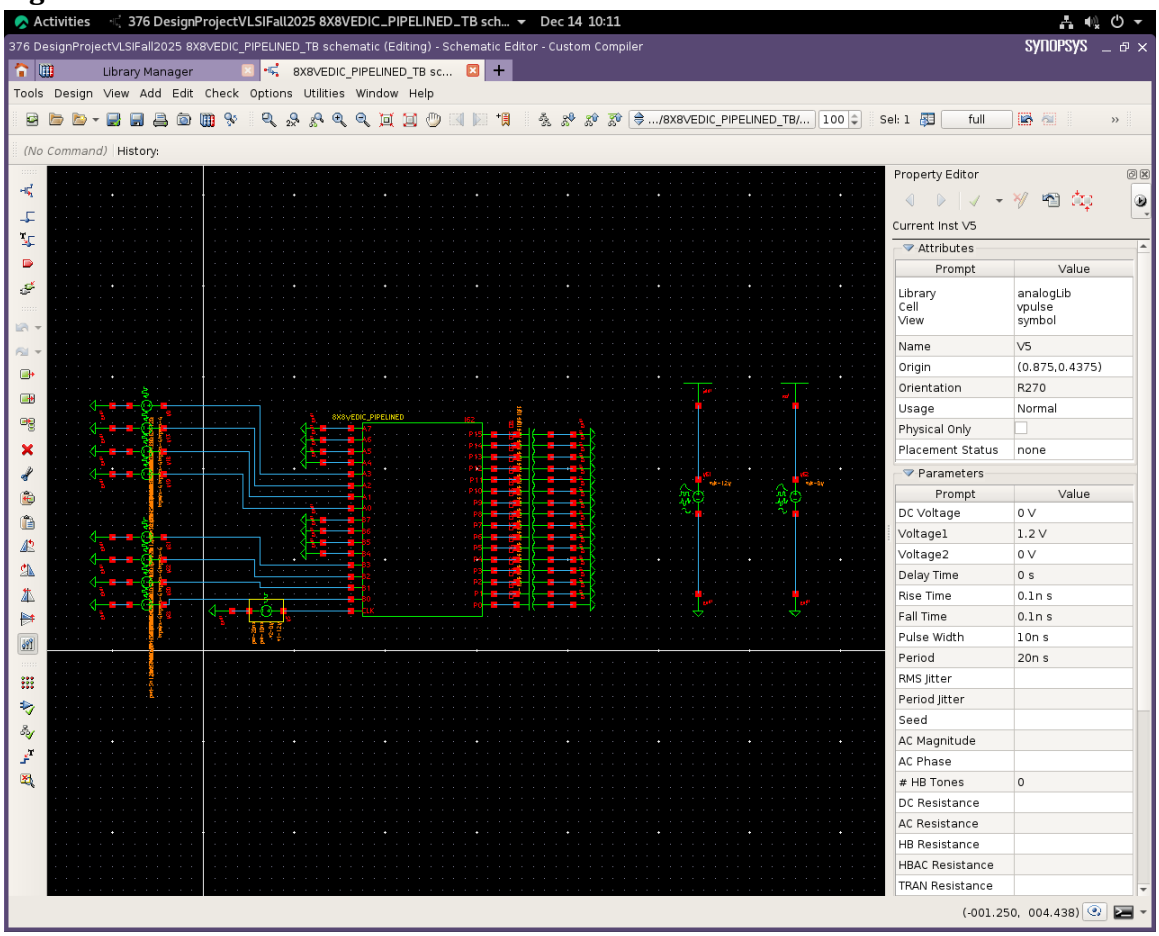
This corresponds to an operating frequency of:

$$f = \frac{1}{20\ \text{ns}} = 50\ \text{MHz}$$

And gives clock edges at: 0ns, 20ns, 40ns, 60ns, 80ns, ...

The selected clock frequency ensures stable operation while clearly illustrating the benefits of pipelining in the multiplier architecture.

Figure 36: 8X8VEDIC PIPELINED TB schematic CLK



5.3 Pre-Layout Functional Simulation

5.3.1 Input and Output Signals

I changed A and B shortly after a falling clock edge and keep them stable until the next falling edge, this ensures proper pipeline loading.

Using VPWL sources that change at specific times; here is my sequence:

- My pipeline has 1-cycle latency (inputs at cycle N = output at cycle N+1).
- Inputs change at: 5ns, 25ns, 45ns, 65ns, 85ns
- Falling clock edges at: 0ns, 20ns, 40ns, 60ns, 80ns

Table 1: Test Cases

Cycle	Time (ns)	A decimal	B decimal	A x B (Expected P)	Output Valid At
0	0-20	3	5	15	cycle 1 (0-20ns)
1	20-40	7	2	14	cycle 2 (20-40ns)
2	40-60	9	9	81	cycle 3 (40-60ns)
3	60-80	12	10	120	cycle 4 (60-80ns)

This gives 4 very different products, including:

- small × small
- odd × odd
- mid-range
- even × even

Exact VPWL Timing for Inputs:

- For A[7:0]:

Time	Value	Binary
5ns	3	00000011
25ns	7	00000111
45ns	9	00001001
65ns	12	00001100

- For B[7:0]:

Time	Value	Binary
5ns	5	00000101
25ns	2	00000010
45ns	9	00001001
66ns	10	00001010

Implement each bit with VPWL:

A4 to A7 and B4 to B7 were grounded for bit 0

Figure 37: 8X8VEDIC PIPELINED TB schematic A0

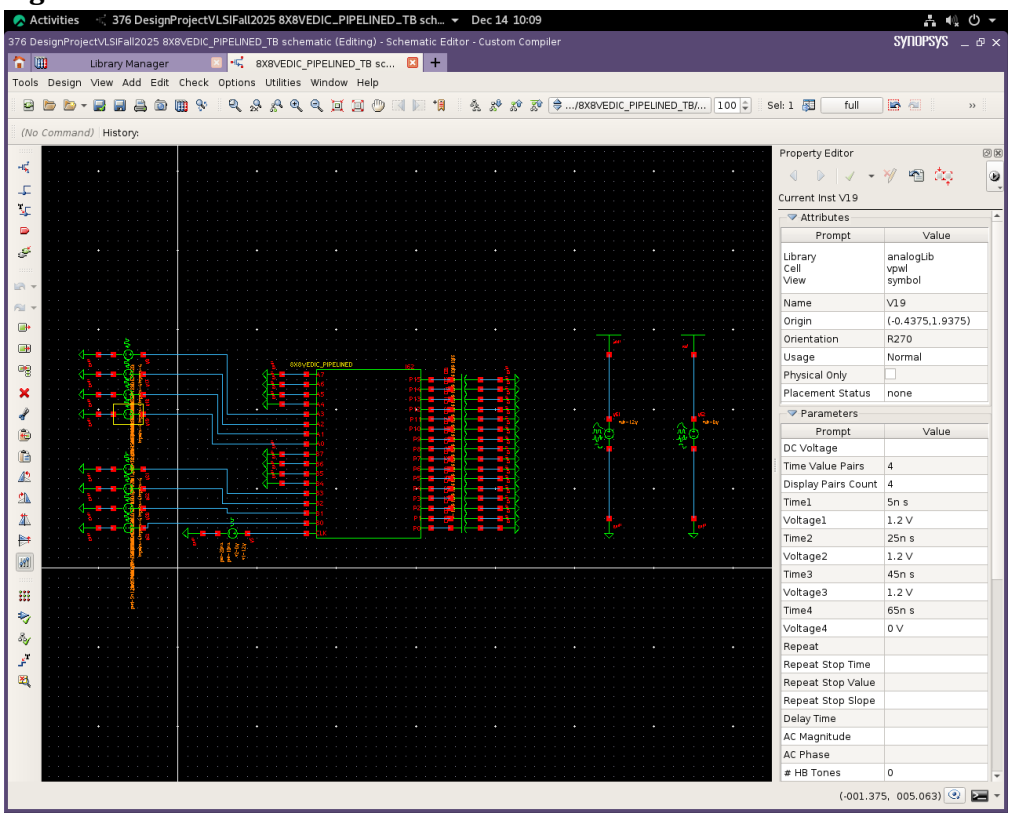


Figure 38: 8X8VEDIC PIPELINED TB schematic A1

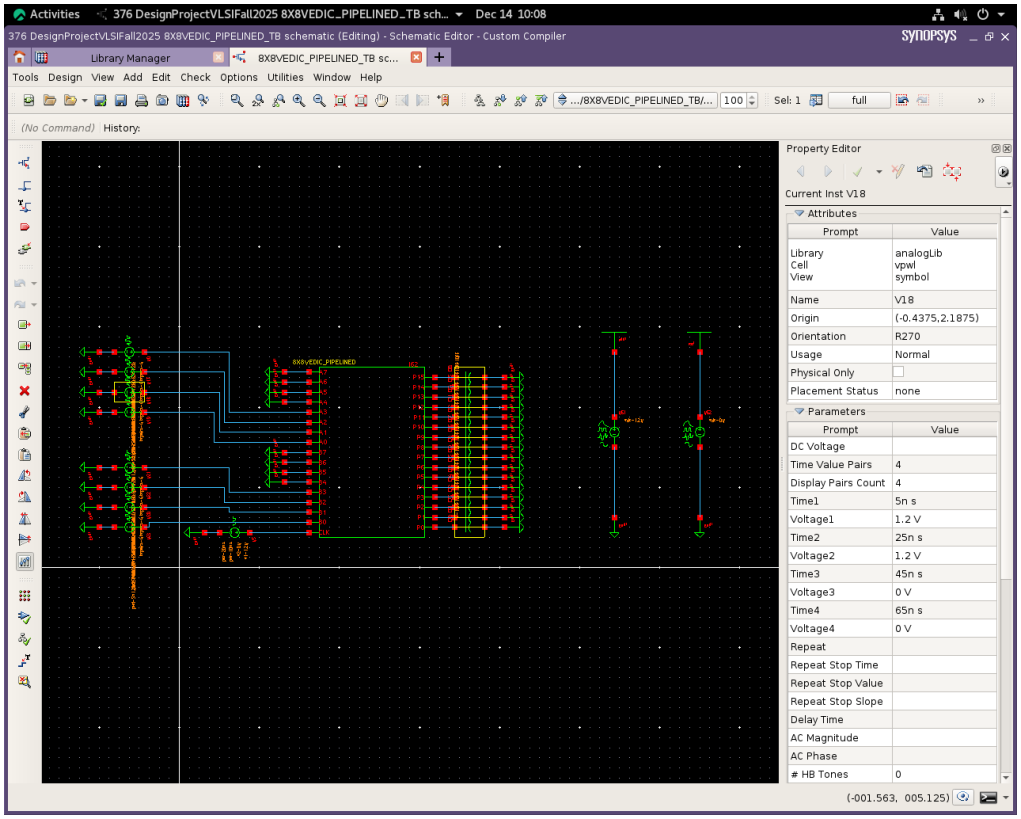


Figure 39: 8X8VEDIC PIPELINED TB schematic A2

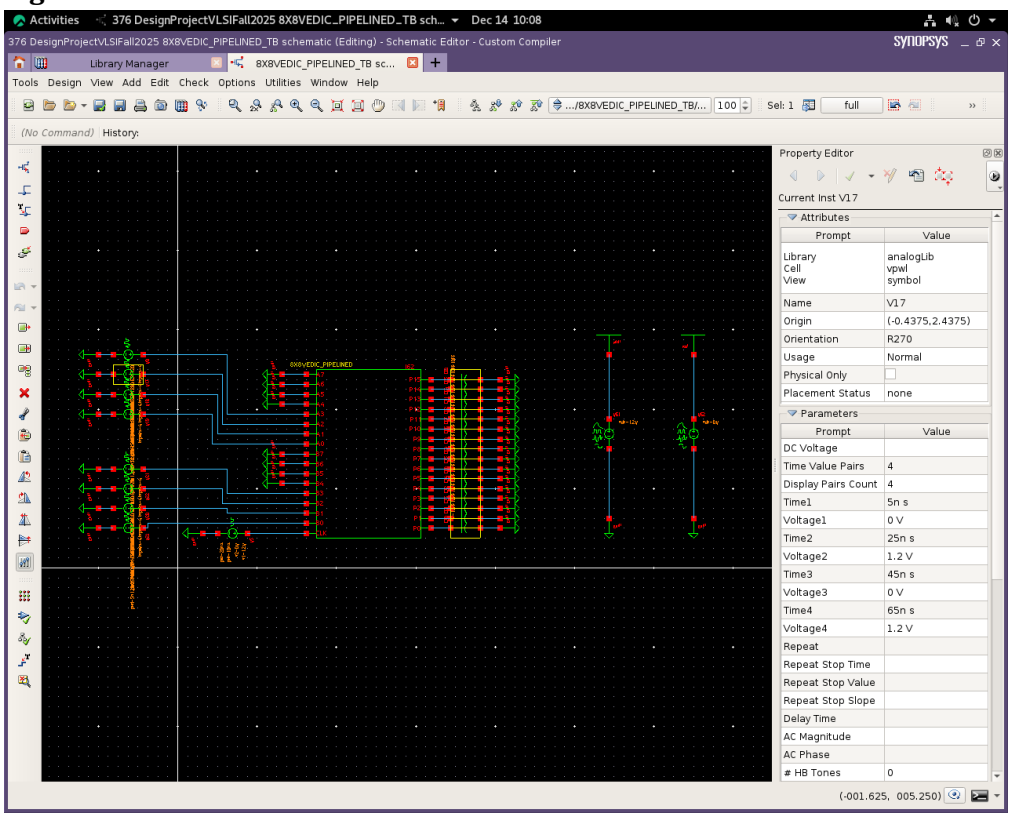


Figure 40: 8X8VEDIC PIPELINED TB schematic A3

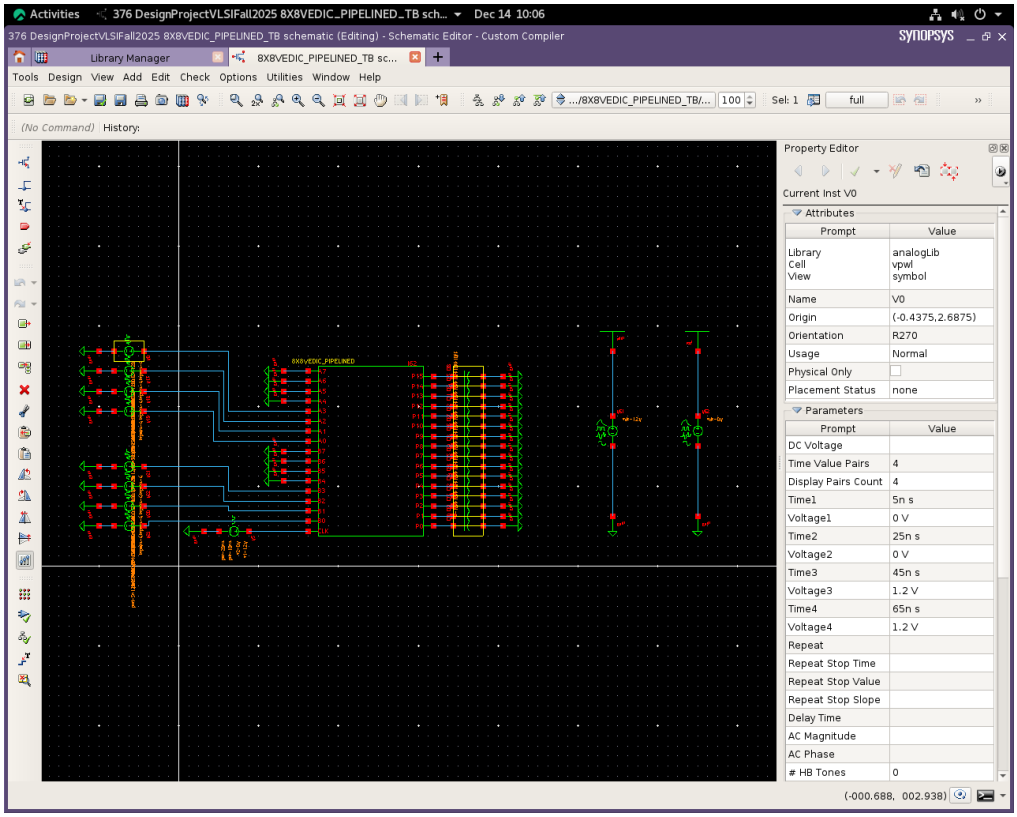


Figure 41: 8X8VEDIC PIPELINED TB schematic B0

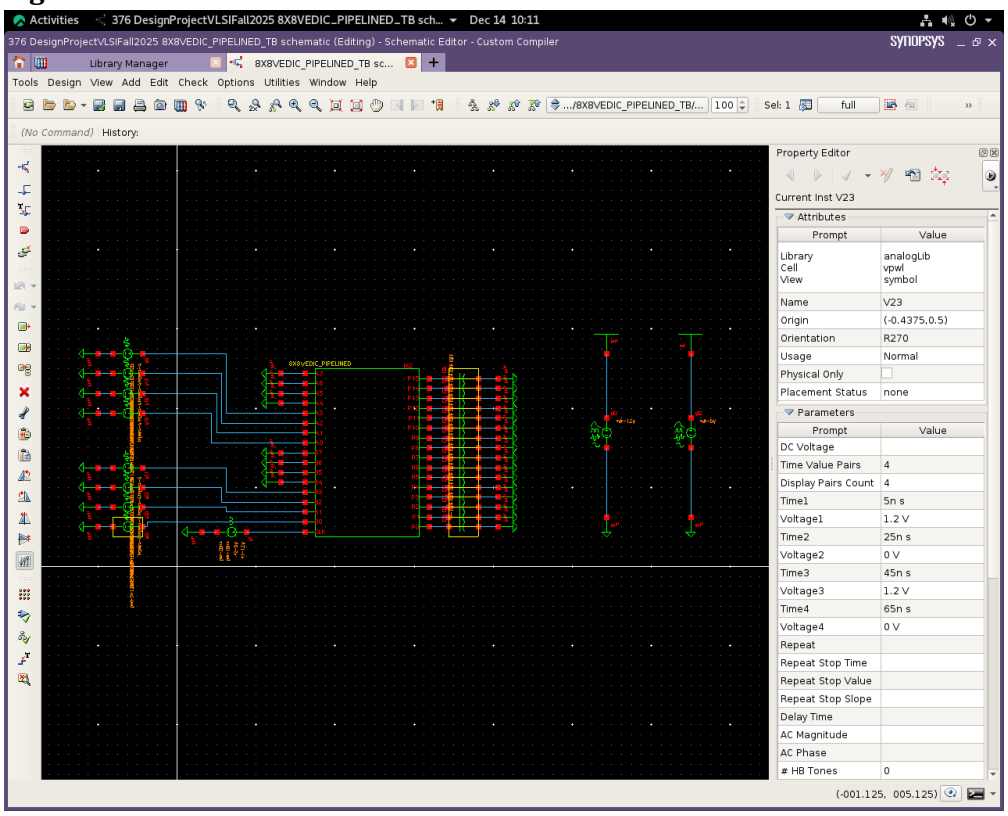


Figure 42: 8X8VEDIC PIPELINED TB schematic B1

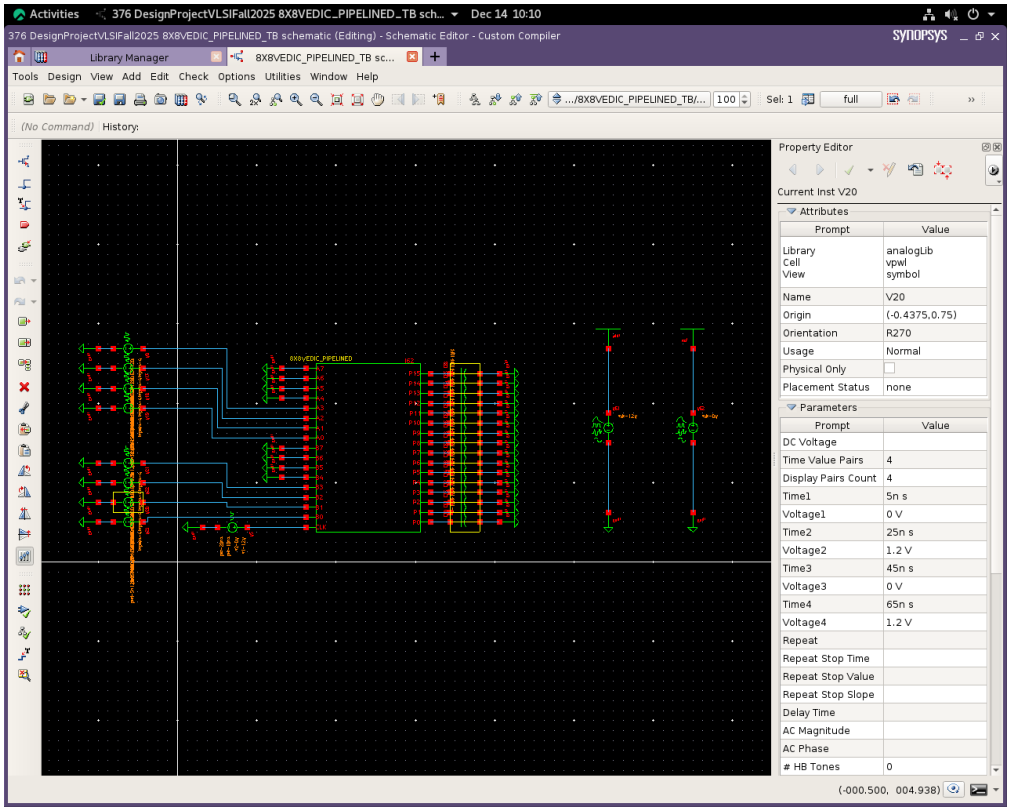


Figure 43: 8X8VEDIC PIPELINED TB schematic B2

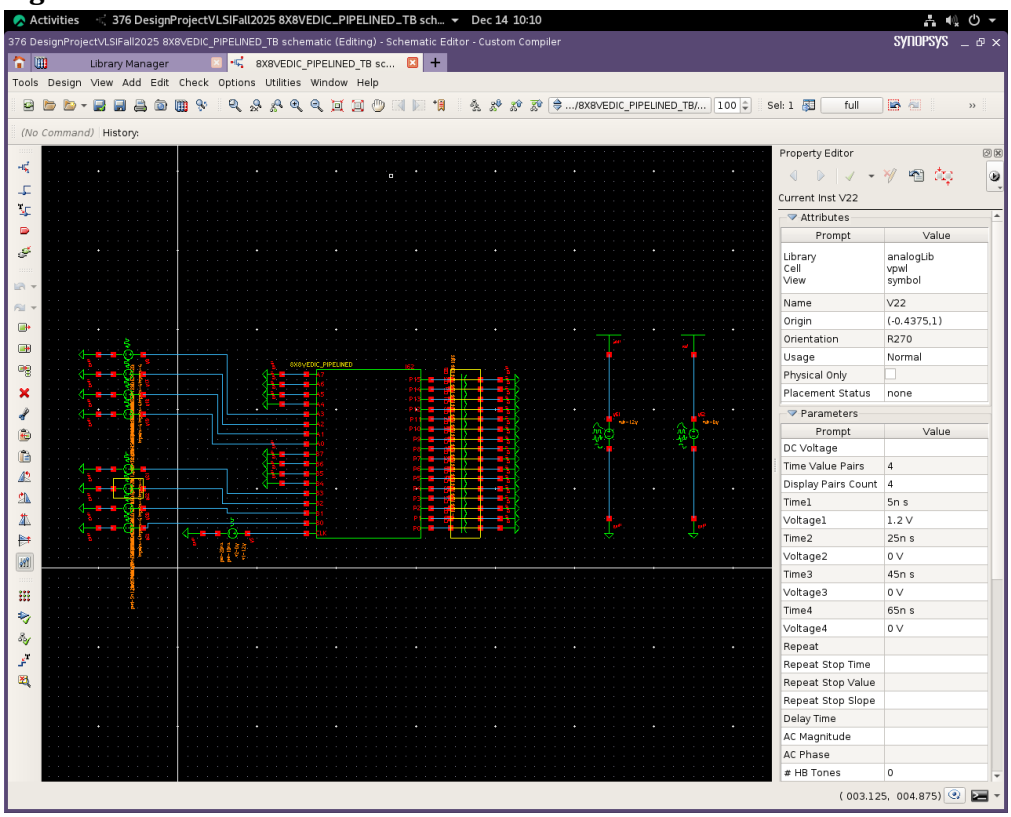
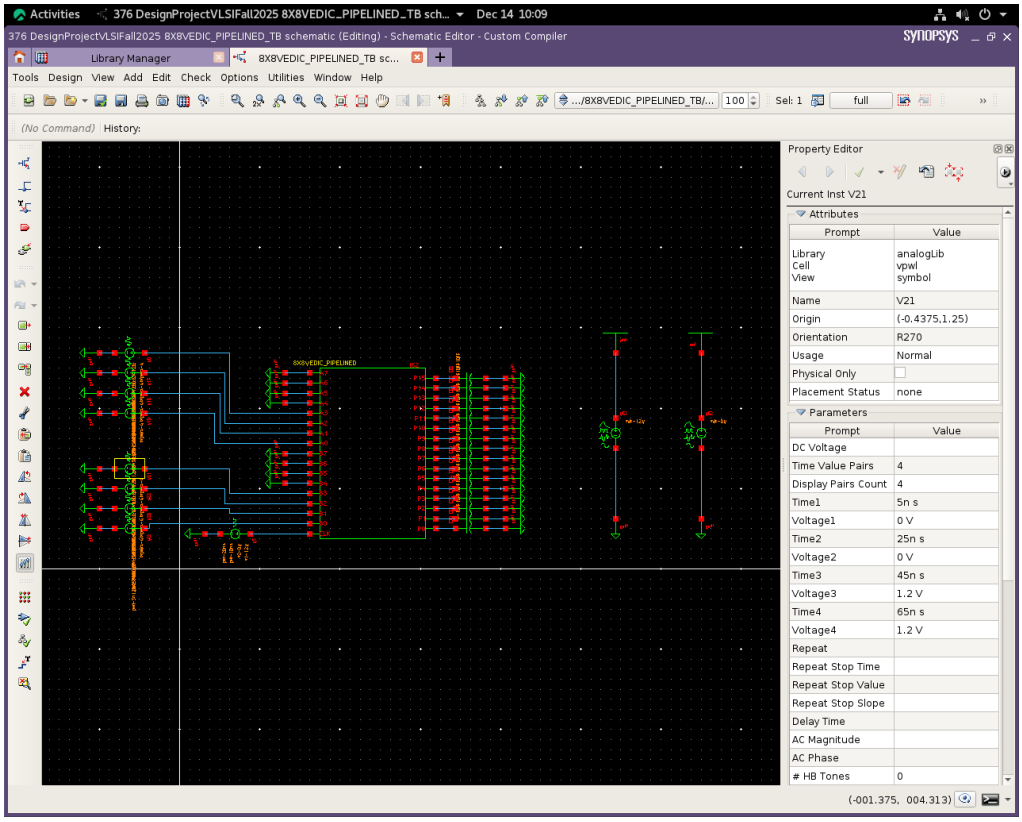


Figure 44: 8X8VEDIC PIPELINED TB schematic B3



The primary signals observed from simulation waveform include:

- Clock (CLK)
- Multiplier output bus: P0: 15

Input operands were applied synchronously with the clock, and output correctness was evaluated after appropriate pipeline latency.

Figure 45: Pre-layout simulation waveform showing CLK and output bits P0: 15

5.3.2 Functional Correctness

Simulation waveforms confirm that the multiplier produces correct 16-bit output results corresponding to the applied input combinations. Due to the pipelined architecture, the output becomes valid only after the pipeline stages are filled. Once steady-state operation is reached, a new valid multiplication result is produced on every clock cycle.

This behavior demonstrates:

- Correct implementation of the Vedic multiplication algorithm
- Proper synchronization of pipeline registers
- Absence of race conditions or timing hazards at the schematic level

The observed outputs match the expected multiplication results for all tested input combinations, confirming functional correctness.

5.4 Pipelining Behavior and Timing Interpretation

One of the key objectives of this project was to demonstrate the effectiveness of pipelining in reducing the critical path. Simulation results clearly show that combinational logic delays are confined within individual pipeline stages.

The longest combinational delay occurs between pipeline registers and is dominated by the 16-bit ripple carry adder stage, where carry propagation determines the worst-case delay. By isolating this logic within a single pipeline stage, the overall system can operate reliably at the selected clock frequency.

The simulation waveforms validate that:

- Register boundaries correctly isolate combinational stages
- No output glitches propagate across clock edges
- Timing behavior is consistent with the intended pipeline structure

5.5 Physical Verification as Part of Testing

While simulation verifies logical correctness, physical verification ensures manufacturability and structural correctness.

- Design Rule Check (DRC): Confirms compliance with all process design rules
- Layout Versus Schematic (LVS): Confirms one-to-one correspondence between schematic and layout

As we saw in **Figure 33 and 34**, DRC report showed zero violations, and LVS report confirmed layout-schematic match
Passing both DRC and LVS provides strong confidence that the layout accurately implements the intended circuit and can be fabricated without rule violations.

5.6 Absence of Post-Layout Simulation

Post-layout simulation with extracted parasitics was not performed in this project. This decision was primarily driven by time constraints and project scope considerations. However, the layout was completed with careful attention to routing discipline, metal layer usage, and signal integrity to minimize parasitic effects.

In future work, parasitic extraction and post-layout simulation would enable:

- More accurate timing analysis
- Quantification of delay degradation due to interconnect capacitance
- Improved power estimation

The current verification methodology nonetheless provides sufficient evidence of functional correctness and architectural validity at the schematic and layout levels.

5.7 Summary of Verification Results

The verification and testing methodology employed in this project demonstrates:

- Systematic bottom-up validation
- Correct functional behavior of the pipelined Vedic multiplier
- Reliable timing behavior at a 50 MHz clock frequency
- Successful physical verification through DRC and LVS

These results confirm that the design meets its stated objectives and is robust from both a logical and physical design perspective.

6. Design Analysis and Performance Evaluation

6.1 Area Analysis

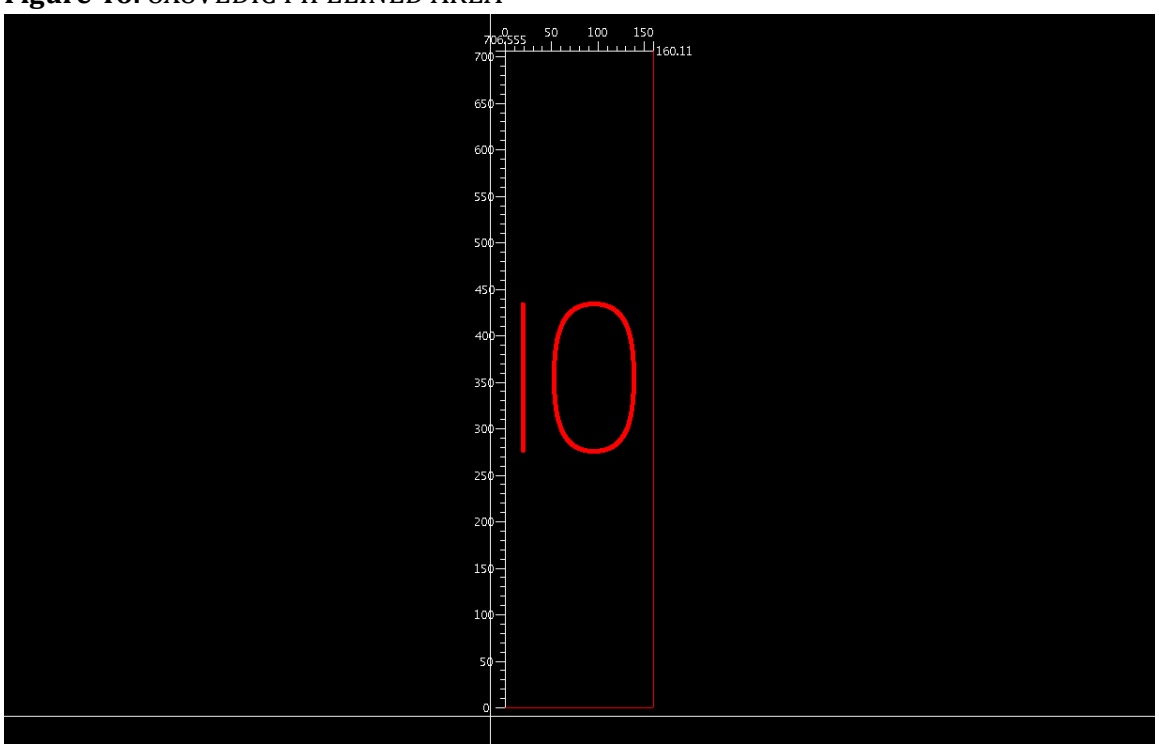
The physical area of the complete **8×8 pipelined Vedic multiplier** was obtained directly from the layout editor after successful integration and verification. The final layout occupies an area of:

$$\text{Area} = 706.55 \times 160.11 = 113125.72\ \mu m\text{^}2$$

This area includes all combinational logic, pipeline registers, routing resources, and power distribution networks. The relatively large footprint reflects the hierarchical nature of the design, the inclusion of multiple pipeline stages, and the routing complexity associated with wide datapaths.

The use of hierarchical layout and repeated sub-blocks (such as 2×2 and 4×4 Vedic multipliers) promotes regularity, but routing congestion at the top level contributes significantly to area growth. In particular, the dense interconnect requirements of the pipelined architecture necessitated the use of multiple metal layers, increasing overall layout dimensions.

Figure 46: 8X8VEDIC PIPELINED AREA



6.2 Timing Analysis

6.2.1 Clock Frequency

The multiplier was simulated using a clock period of **20 ns**, corresponding to an operating frequency of:

$$f_{max} = \frac{1}{20\ \text{ns}} = 50\ \text{MHz}$$

This clock frequency was selected to ensure stable operation and to clearly demonstrate correct pipelined behavior in simulation.

6.2.2 Critical Path Identification

In the pipelined architecture, the critical path is confined to the longest combinational logic stage between pipeline registers. Based on the circuit structure, the dominant contributor to the critical path is the 16-bit ripple carry adder (16RCA) stage.

Ripple carry adders exhibit linear delay due to sequential carry propagation through each full adder stage. Without pipelining, this delay would span multiple arithmetic blocks and severely limit operating frequency. By introducing pipeline registers between major computation stages, the design confines carry propagation to a single pipeline stage, significantly reducing the effective critical path.

This architectural decision enables higher clock frequencies than would be achievable in a purely combinational 8×8 multiplier design [3].

6.2.3 Propagation Delay

Delays were extracted per output (P0–P6). Values below are in picoseconds (ps). Average and worst-case delays are computed from the uploaded spreadsheet.

Outputs	Rise Delay (ps)	Fall Delay (ps)	H-L Prop Delay (ps)	L-H Prop Delay (ps)	Avg Prop Delay (ps)
P0	712	414	156	387	271.5
P1	N/A	370	257	N/A	257
P2	N/A	370	255	N/A	255
P3	870	370	257	641	449
P4	956	N/A	N/A	638	638
P5	892	N/A	N/A	648	648
P6	831	N/A	N/A	654	654
				Total Average:	453.2142857

Below are images of my 8X8 Pipelined Vedic Multiplier Rise, Fall, and Propagation Delays:

Figure 47: P0 Delays

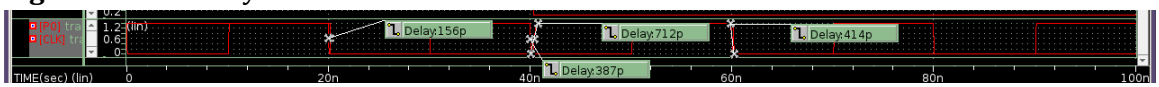


Figure 48: P1 Delays

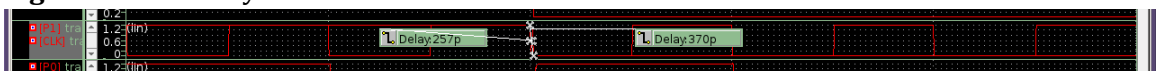


Figure 49: P2 Delays

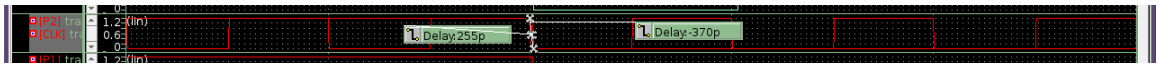


Figure 50: P3 Delays

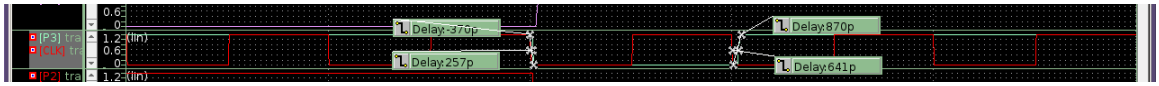


Figure 51: P4 Delays

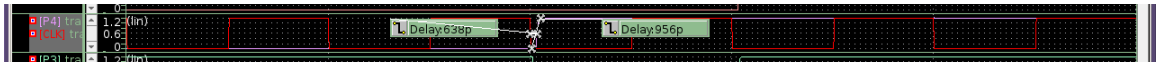


Figure 52: P5 Delays

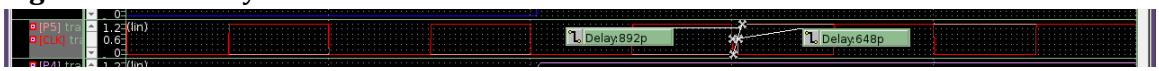


Figure 53: P6 Delays

Average propagation delay (P0–P6): 453.21ps.

Worst-case propagation delay: P6 = 654ps.

6.2.4 Timing Trade-Offs

While ripple carry adders are not the fastest adder architecture available, their simplicity, predictability, and ease of layout integration make them suitable for this pipelined design. More advanced adder architectures, such as carry-lookahead or carry-select adders, could further reduce delay but would introduce additional layout complexity and area overhead [3].

6.3 Power Considerations

6.3.1 Dynamic Power Consumption

Dynamic power consumption dominates in CMOS digital circuits operating at moderate to high frequencies. In the proposed design, dynamic power is primarily influenced by:

- Clock switching activity in pipeline registers
- Charging and discharging of large interconnect capacitances

- Switching activity within arithmetic logic blocks

The inclusion of multiple pipeline registers increases clock-related power consumption; however, this is an expected trade-off for improved throughput and timing performance [4].

6.3.2 Static Power Consumption

Static power consumption is minimal in this design due to the use of complementary CMOS logic with full rail-to-rail signal swing. Leakage currents exist in deep-submicron technologies, but their impact is secondary compared to dynamic power under normal operating conditions [4].

6.3.3 Static Power Consumption

The design illustrates a classic power–performance trade-off:

- Higher throughput achieved through pipelining
- Increased register count leading to higher dynamic power

This trade-off is acceptable for performance-oriented applications where throughput is a priority. Further power optimization techniques, such as clock gating or operand isolation, could be applied in future iterations to reduce unnecessary switching activity [4].

6.4 Scalability Considerations

The hierarchical and modular nature of the Vedic multiplier architecture makes it inherently scalable. Larger operand sizes (e.g., 16×16 or 32×32) can be constructed by extending the same building blocks and pipeline strategy. However, scaling would increase routing complexity and power consumption, emphasizing the importance of advanced interconnect planning and power optimization techniques in larger designs.

6.5 Summary of Performance Evaluation

The design analysis demonstrates that the proposed 8×8 pipelined Vedic multiplier:

- Achieves correct operation at 50 MHz
- Confines the critical path to a single arithmetic stage
- Occupies a physical area of 113125.72 μm^2
- Has an average propagation delay of 453.21 ps, and Worst-case propagation delay of 654 ps
- Balances performance, area, and power through architectural trade-offs

These results confirm that the design meets its stated objectives and provides a practical demonstration of pipelined arithmetic implementation in CMOS technology.

7. Design Decisions, Challenges, and Lessons Learned

7.1 Key Design Decisions

7.1.1 Selection of Vedic Multiplier Architecture

The Vedic multiplier architecture was selected due to its inherent parallelism and regular structure. Unlike conventional array multipliers, the Vedic approach generates partial products concurrently, reducing logical depth and enabling efficient hierarchical decomposition. This made it particularly suitable for an 8×8 implementation built from smaller 2×2 and 4×4 blocks.

Additionally, the regularity of the architecture simplifies layout reuse and scaling, which is advantageous in a full-custom CMOS design environment [1].

7.1.2 Use of Pipelining

Pipelining was introduced to improve throughput by reducing the effective critical path. Without pipelining, the combined delay of multiple arithmetic stages would significantly limit the achievable clock frequency. By inserting register stages between the 4×4 Vedic multipliers and the 16-bit ripple carry adders, the longest combinational delay was confined to a single arithmetic stage.

This design choice reflects a deliberate trade-off:

- Benefit: Increased throughput and improved timing margins
- Cost: Increased register count and higher dynamic power consumption

Given the project's performance-oriented objective, this trade-off was justified.

7.1.3 Choice of Transmission-Gate D Flip-Flops

Transmission-gate-based D flip-flops were selected instead of static master–slave flip-flops. This decision was motivated by several factors:

- Full rail-to-rail signal swing without threshold voltage loss
- Reduced static power consumption
- Robust operation at low supply voltage (1.2 V)

Transmission gates also integrate naturally with CMOS logic styles and avoid ratioed behavior, improving reliability across process variations [5].

7.1.4 Use of Ripple Carry Adders

Ripple carry adders were chosen for both 8-bit and 16-bit addition stages due to their simplicity and predictable timing behavior. While more advanced adder architectures could reduce delay, their increased layout complexity and area overhead were not justified given that pipelining limits carry propagation to a single stage.

This choice prioritizes clarity, correctness, and layout feasibility over aggressive micro-optimization.

7.2 Major Design Challenges

7.2.1 Complexity of the 4×4 Vedic Multiplier Layout

The 4×4 Vedic multiplier proved to be the most challenging block to design, primarily due to initial pin placement decisions. Early versions of the layout had pins that were not easily accessible when integrating the block into the 8×8 pipelined architecture. This significantly increased routing complexity at higher levels of hierarchy.

The issue was resolved by redesigning the pin placement to align inputs and outputs along the block boundaries, greatly simplifying top-level routing.

7.2.2 Routing Congestion in Top-Level Layout

Routing congestion emerged as the dominant physical design challenge, particularly in the top-level integration of the pipelined multiplier. The high number of overlapping data paths, control signals, and clock lines required careful use of multiple metal layers.

This challenge was mitigated by:

- Strategic use of Metal 2 and Poly for signal crossings
- Improved block placement to shorten interconnect lengths
- Iterative refinement of routing paths

These adjustments reduced congestion and enabled the final layout to pass DRC successfully.

7.3 Verification-Driven Design Refinement

Throughout the project, verification results directly informed design refinements. Incremental DRC and LVS checks identified layout issues early, preventing error propagation into higher-level designs. This iterative verification approach proved essential in managing the complexity of a full-custom, hierarchical design.

The absence of post-layout simulation was acknowledged as a limitation; however, the disciplined verification process provides confidence in the correctness and manufacturability of the design.

7.4 Lessons Learned

Several important lessons emerged from this project:

1. Pin placement is critical: Poor pin accessibility at lower levels can significantly complicate higher-level integration.

2. Hierarchy simplifies complexity: Building and verifying small blocks before integration reduces debugging effort.
3. Routing dominates area: In large arithmetic designs, interconnect planning can be more critical than transistor count.
4. Pipelining is a powerful tool: Strategic register insertion can dramatically improve performance with manageable overhead.

7.5 Opportunities for Improvement

Given additional time, the design could be further improved by:

- Optimizing pin placement across all blocks from the outset
- Performing parasitic extraction and post-layout simulation
- Exploring faster adder architectures within pipeline stages
- Applying power optimization techniques such as clock gating

These enhancements would further improve performance, power efficiency, and robustness in production-level implementation.

8. Conclusion

This project presented the complete design and implementation of an 8×8 pipelined Vedic multiplier using full-custom CMOS techniques in a 90 nm process technology. The primary objective which was to demonstrate the effectiveness of combining Vedic multiplication principles with pipelining for improved throughput, was successfully achieved through careful architectural planning, circuit-level design, disciplined layout practices, and systematic verification.

The multiplier architecture leveraged the inherent parallelism of the Vedic *Urdhva Tiryakbhyam* algorithm and decomposed the design hierarchically into reusable 2×2 and 4×4 multiplier blocks. The introduction of pipeline registers between major arithmetic stages significantly reduced the effective critical path, enabling reliable operation at a clock frequency of 50 MHz. Transmission-gate-based D flip-flops provided robust sequential storage with full rail-to-rail signal swing at a 1.2 V supply.

At the physical design level, the project demonstrated a structured layout methodology using multiple metal layers to manage routing complexity. All layouts successfully passed Design Rule Check (DRC) and Layout Versus Schematic (LVS) verification, confirming both manufacturability and schematic equivalence. The final design occupied an area of 113125.72 μm^2, reflecting the trade-offs between pipelining, routing density, and modular hierarchy.

Although post-layout simulation with parasitic extraction was not performed, the project established a solid foundation for future timing and power refinement. The verification strategy, architectural decisions, and layout discipline collectively demonstrate a strong understanding of modern VLSI design principles.

Overall, this project provides a practical and scalable demonstration of pipelined arithmetic design in CMOS technology. The techniques employed; hierarchical decomposition, pipelining, and full-custom layout, are directly applicable to larger and more complex digital systems, making this work a valuable learning exercise and a strong representation of disciplined VLSI design practice.

9. References

[1] J. K. Patel, V. S. Patel, and S. J. Patel, "Design of High Speed Vedic Multiplier Using Urdhva Tiryakbhyam Sutra," *International Journal of Computer Applications*, vol. 21, no. 6, pp. 26–30, May 2011.

[2] N. H. E. Weste and D. M. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed., Boston, MA, USA: Pearson, 2011.

[3] J. M. Rabaey, A. Chandrakasan, and B. Nikolić, *Digital Integrated Circuits: A Design Perspective*, 2nd ed., Upper Saddle River, NJ, USA: Prentice Hall, 2003.

[4] S. M. Kang and Y. Leblebici, *CMOS Digital Integrated Circuits: Analysis and Design*, 3rd ed., New York, NY, USA: McGraw-Hill, 2003.

[5] R. Zimmermann and W. Fichtner, "Low-Power Logic Styles: CMOS Versus Pass-Transistor Logic," *IEEE Journal of Solid-State Circuits*, vol. 32, no. 7, pp. 1079–1090, Jul. 1997.